

Linux Commands Summary

Prepared by : Amit

29/06/2026

Contents

1	Miscellaneous commands	3
1.1	py : evaluate python command from command line	3
1.2	shutdown : shutdown the system in a safe way	3
1.3	minicom : RS232 communication on serial devices	4
1.4	google-chrome : the web browser from Google	4
1.5	mktemp : Create a temporary file or directory	5
1.6	clamscan : antivirus	5
1.7	reboot : Restart or reboot the system.	6
1.8	Zeal : Offline documentation browser	6
1.9	wine : run microsoft software in Linux.	6
1.10	yes : accept everything...	6
1.11	uptime : tells how long the system is running	7
1.12	sleep : delay for a specified amount of time	7
1.13	Linux..O.S..version..check	7
1.14	bc : "Basic Calculator" - simplified man pages	7
1.15	cal : "CALendar" - basic calendar	8
1.16	date : show current date with format	8
1.17	hdate : Hebrew DATE	9
2	System administration basics	9
2.1	lspci : List all PCI devices	9
2.2	kernel..modules : Linux drivers	10
2.3	dmesg..modules : Show kernel messages	10
2.4	strace : trace system calls of a program	12
2.5	uname : print system information	14
2.6	service..systemctl : service / daemon	14
3	Bash Features and Scripting	15
3.1	only..bash..features	15
3.2	run..BASH..script../bin/bash	15
3.3	"hashbang"... "shebang"	16
3.4	stop..bash..script..at...error	16
3.5	bash..debug	17
3.6	shopt : SHell OPTions manage	18
3.7	set : Change shell attributes / display shell vars	18
3.8	chaining..operators explanation	20
3.9	: pipe - redirect stdout of one cmd to stdin of another cmd.	21
3.10	;; : send several commands in BASH	22
3.11	: logical OR	22
3.12	&& : logical AND	23
3.13	backslash : Split commands in BASH	23
3.14	& symbol : start process in the background immediately	24
3.15	bash : Start an interactive shell session	24
3.16	SHELL.variables..vs..Env.variables	26
3.17	"=" : assign value for Shell/Env. variables.	29
3.18	examples..of..system..vars	29
3.19	\$PATH : path system variable	30
3.20	\$SHELL : current shell type)	30
3.21	\$SHLVL : SHell LeVeL check shell level	30

3.22	\$PS1 : Prompt String 1	31
3.23	\$BASH_VERSION : result of last command execution	31
3.24	\$? : result of last command execution	31
3.25	\$_ : PID of last executed program	31
3.26	\$\$: current bash Process ID	32
3.27	\$PROMPT_DIRTRIM : PROMPT DIRectory TRIMmer	32
3.28	which : Locate a program in the user's PATH.	32
3.29	export : set or present exported variables	33
3.30	.bashrc : (BASH Run Comm.) runs every time new bash opened	33
3.31	.profile vs. .bash_profile vs. .bashrc	34
3.32	alias : Creates aliases	35
3.33	unalias : Remove aliases	36
3.34	printenv : Print values of all/specific Env. variables.	36
3.35	env : Show and set the env. / run a prog. in a modified environment	36
3.36	declare : Declare variables and give them attributes	39
3.37	"`" : Quoting	40
3.38	"expansion" : made in echo command	41
3.39	`cmd` (backtick) equals \$(cmd): run cmd and substitute its text	43
3.40	math.\$[...]\$((...))...((...)) : command to do simple arithmetic	44
3.41	let : math - command to do simple arithmetic	46
3.42	true : Returns a successful exit status code of 0: \$? = 0	47
3.43	false : Returns a non-zero exit code: \$? = 1	47
3.44	tab : autocomplete	47
3.45	type : Display the type of command the shell will execute.	47
3.46	Comments in BASH	48
3.47	source/"." : Execute commands from a file in the current shell	48
3.48	read : retrieving data from stdin	48
3.49	gnome-terminal : open new linux terminal emulator	49
3.50	arguments..of..process	49
3.51	clear :Equivalent to CTRL+L : clears the screen of the terminal.	50
3.52	process..output..to..stderr	50
3.53	exit : exit the process (script) , exit the shell	51
3.54	if : if command	51
3.55	test..[...]: command that can be used as test in if command	53
3.56	for : loop command	57
3.57	case : case command	61
3.58	while : while command	62

4	Processes/Jobs Control	62
4.1	process..input..output	62
4.2	arguments..of..process	63
4.3	Ctrl+Z : SIGTSTP - an inTeractive SToP signal	64
4.4	Ctrl+C : SIGINT <Signal INTerrupt> interrupting a process	65
4.5	Ctrl+D : End of File / exit shell	66
4.6	ps : "Process Status" - snapshot of the current processes	66
4.7	pstree : display a tree of processes	71
4.8	pgrep : ps+grep	71
4.9	top : "Table Of Processes"	72
4.10	htop : similar as top, but more user friendly	72
4.11	nice : run a program with a custom priority (niceness)	73

4.12	renice : alters priority of already running process	74
4.13	eval : combines arguments into a single string and runs	74
4.14	jobs : Execute arguments as a single command in shell	75
4.15	kill : send a signal to a process	76
4.16	pkill : Signal process by name	78
4.17	xkill : Kill a window interactively in a graphical session	79
4.18	bg : resume suspended process to run it in the background.	79
4.19	& symbol : start process in the background immediately	80
4.20	disown : Signal process by process's name	81
4.21	time : Run the command and print the time measurements	82
4.22	fg : resume a specific suspended job by its job number.	82
4.23	\$? : result of last command execution	83
5	Commands envistigation	83
5.1	type : Display the type of command the shell will execute.	83
5.2	which : Locate a program in the user's path.	84
5.3	whereis : locate the source, and manual page for a command.	84
5.4	command : locate the source, and manual page for a command.	84
5.5	history : command-line history	85
5.6	man : display manual pages	85
5.7	tldr : "Too Long; Didn't Read" - simplified man pages	86
5.8	whatis : breaf command description	86
5.9	apropos : command to search the man page files	87
6	Text and text files manipulations	87
6.1	pandoc : Convert documents between various formats	87
6.2	tab : autocomplete	87
6.3	: pipe - redirect stdout of one cmd to stdin of another cmd.	87
6.4	Redirections of I/O	88
6.5	echo : display line of text	91
6.6	printf : display line of text	94
6.7	grep : "Global Regular Expression search and Print"	94
6.8	cat : "ConcATenite" : concatenate and print.	97
6.9	more : present text file in parts.	97
6.10	less : present text file in parts and allows search	98
6.11	head : Display n rows at the head of file	98
6.12	tail : Display n rows at the end of file	98
6.13	sort : Display lines in sorted order	98
6.14	wc : "Word Count" - standard input statistics	99
7	Compressing and archiving	100
7.1	gzip/gunzip : shrink huge files and save space	100
7.2	bzip2/bunzip2 : better compression than gzip	101
7.3	tar : Tape ARchive - back up set of files	101
7.4	xz/ : relative new, better compression	104
7.5	7zip/ : better compression than bzip2	104
8	File comparison and file integrity	106
8.1	file.integrity..check	106
8.2	crc32 : calculate crc32	106

9	Find files and check directory size	106
9.1	du : "Disk Usage" - file and dir usage space	106
9.2	locate : command	108
9.3	find : command	108
9.4	tree : show curr. dir as tree.	109
10	Dirs and files manipulation	110
10.1	directory.permissions.in.Linux	110
10.2	basename : store the name of dir based on full path	110
10.3	nautilus : file explorer in Ubuntu	111
10.4	chmod : CHange file MODe (permissions and spec. modes)	111
10.5	ln : Creates links to files and directories	115
10.6	find : Search for a file	117
10.7	touch : Create files or update access times	119
10.8	stat : display file (inod) and file system info	120
10.9	ls : "LiSt" files and dirs	120
10.10	du : "Disk Usage" - file and dir usage space	122
10.11	ncdu : "Norton Commander and Disk Usage" - file and dir usage space	123
10.12	fdisk : partitions in hard disk	123
10.13	lsblk : Lists information about block(storage) devices	123
10.14	df : overview of the filesystem disk space usage	124
10.15	cd : "Change Dir"	124
10.16	pwd : "Print Working Directory"	124
10.17	mkdir : "MaKe DIRectory"	124
10.18	cp : "CoPy" file to file, file to dir, dir to dir	125
10.19	mv : "MoVe" files or dirs from one place to another.	126
10.20	rm : "ReMove" removes references(!) to objects from the filesystem	126
10.21	rmdir : "ReMove DIR" remove empty(!) dir from filesystem.	127
10.22	tree : show curr. dir as tree.	127
10.23	file : determine file type.	128
11	User Groups	129
11.1	CreateNewUser - steps to create new user	129
11.2	visudo - edit sudoer file	129
11.3	passwd : Change a user's PASSWorD.	130
11.4	useradd : add user to the system	130
11.5	\$SHELL : env. variable (check current shell type)	132
11.6	adduser : perl script, that uses useradd	132
11.7	userdel : delete user from the system	133
11.8	deluser : perl script, that uses userdel	133
11.9	/etc/group - file with list of groups	133
11.10	/etc/passwd : conteins info what users are in the system	134
11.11	groups : present all groups of the user	134
11.12	chown : CHange OWNer (user/group) of files and dirs	134
11.13	members : present all users of the group	135
11.14	groupadd : Add user groups to the system	135
11.15	passwd : change password	136
11.16	whoami : currently logged-in user	136
11.17	su : "Substitute User"	136
11.18	exit : exit the shell	137

11.19	usermod : MODifies a USER account/add user to group.	137
11.20	usermod : add Tal to sudo group and remove tal from sudo group.	138
12	Special Device files	139
12.1	Device..files with special behaviour	139
12.2	/dev/null and /dev/zero : data sink files	140
12.3	/dev/random file - random numbers generator file	141
12.4	/dev/full - file for device full simulation	141
12.5	/etc/group - file with list of groups	142
12.6	/etc/passwd - info what users are in the system	142
12.7	/etc/skel - skeleton files for new user	143
13	Binary files treatment	143
13.1	binary files compare	143
13.2	nm : "Name Mangling" damp symbol table from object files.	144
13.3	readelf : ELF file info.	145
13.4	od : "Octal Damp" - display binary file.	145
14	Editors, text operations	146
14.1	nano : run text editor.	146
14.2	vim :v improved editor.	146
14.3	tilde : intuitive text editor.	146
15	Install applications	147
15.1	dpkg : Debian PaKaGe install	147
15.2	apt : "Advanced Package Tool"	147
16	GCC and binary files operations	149
17	Ethernet	149
17.1	Wireshark install	149
17.2	packet.names.at.different.levels.of.TCP/IP	150
17.3	TCP.vs.UDP	150
17.4	sysctl : change kernel Networking params	150
17.5	IP..netmask..explanation	151
17.6	DHCP : Dynamic Host Configuration Protocol	154
17.7	dhclient : obtain/release the IP address drom DHCP server	158
17.8	SSH : OpenSSH remote login client	158
17.9	fail2ban : security intrusion prevention software framework	161
17.10	who : display who is logged in	161
17.11	SCP : OpenSSH secure file copy	161
17.12	pscp : transfer files between Windows and linux	162
17.13	ping : check connection	163
17.14	iperf : measure quality and bandwidth of link	165
17.15	TTL : Time To Leave	166
17.16	Gate Way : Time To Leave	166
17.17	traceroute : shows hopes(jumps) between devices	166
17.18	route : set/display the routing table	167
17.19	ip/ip route : ip command	168
17.20	DNS..server	171
17.21	arp : manage Address Resolution Protocol	171

17.22	ifconfig : InerFace CONFIGurator	172
17.23	ping : ping command	174
17.24	hostname : system's hostname	175
17.25	wget : network downloader	175
18	TextToSpeech	176
18.1	TextToSpeech..apps	176
18.2	spd-say : text-to-speech output	177
19	Appendix.	177
19.1	LaTeX : editor	177
19.2	Shell..built..in..commands : shell built in commands	178
19.3	shift+PgUp/PgDn : page scroll in terminal	178
19.4	niceness : process priority	178
19.5	VT : virtual terminal	179
19.6	ctrl+shift+arrowup/down : scroll in terminal	179
19.7	Additional...shells	179
19.8	Special...files	180
19.9	inode : Create files or set access/modif. times	180
19.10	Disc..Partitions	180
19.11	Globbering	181

1 Miscellaneous commands

1.1 py : evaluate python command from command line

-c : python command run

Example :

```
$ py -c "print(2**32-1)"
4294967295
```

1.2 shutdown : shutdown the system in a safe way

Shutdown the system in a safe way. You can shutdown the machine immediately, or schedule a shutdown using 24 hour format. It brings the system down in a secure way. When the shutdown is initiated, all logged-in users and processes are notified that the system is going down, and no further logins are allowed.

Only root user can execute shutdown command

- r : Requests that the system be rebooted after it has been brought down.
- P : Requests that the system be powered off after it has been brought down.
- k : Only send out the warning messages and disable logins, do not actually bring the system down.

Shut down :

```
$ sudo shutdown
```

How to shutdown the system at a specified time

```
$ sudo shutdown 05:00
```

System shutdown in 20 minutes from now:

```
$ sudo shutdown +20
```

How to shutdown the system immediately

```
$ sudo shutdown now
```

How to make shutdown power-off machine

Although this is by default, you can still use the -P option to explicitly specify that you want shutdown to power off the system.

```
$ shutdown -P
```

How to broadcast a custom message

```
$ sudo shutdown +10 "System upgrade"
```


1.3 minicom : RS232 communication on serial devices

minicom is a communication on serial devices, supports script language interpreting, capturing to file, multiple users with individual configurations, and more.

```
$ sudo apt install minicom
```

```
$ minicom -s # for settings, will edit /etc/minirc.dfl file
serial port settings setup:
```

```
+-----+
| A -   Serial Device       : /dev/modem   # device name   |
| B - Lockfile Location    : /var/lock     |
| C -   Callin Program     :                |
| D -   Callout Program    :                |
| E -   Bps/Par/Bits       : 115200 8N1    |
| F - Hardware Flow Control : Yes          |
| G - Software Flow Control : No          |
| H -   RS485 Enable       : No           |
| I -   RS485 Rts On Send  : No           |
| J -   RS485 Rts After Send : No         |
| K -   RS485 Rx During Tx : No           |
| L -   RS485 Terminate Bus : No          |
| M - RS485 Delay Rts Before: 0            |
| N - RS485 Delay Rts After : 0            |
|                                     |
|   Change which setting?              |
+-----+
```

after end of settings setuo we can choose
"save as dflt"

minicom will control the terminal.
to choose options : ctrl+a+o : options
ctrl+a+q : exit

1.4 google-chrome : the web browser from Google

```
$ google-chrome --new-window https://rt-ed.co.il/
```

--new-window

If PATH or URL is given, open it in a new window.

1.5 mktmp : Create a temporary file or directory

```
- Create an empty temporary directory and print its absolute path:  
$ mktmp -d  
/tmp/tmp.Ubw3bJBvR2
```

1.6 clamscan : antivirus

```
Scan home directory ("~"):  
$ clamscan -r --bell -i -v -o -a ~
```

```
-v: Be verbose.  
-a: Show filenames while scan  
-r: Recursive  
-i: Only print infected files.  
-o: Skip printing OK files
```

```
To view the version of the scan engine  
$ clamscan -V # Capital V  
ClamAV 0.103.11/27191/Tue Feb 20 11:25:13 2024
```

```
# Base Format  
$ clamscan [options] [directory or file to scan]  
# To scan the current directory  
$ clamscan  
# To view the version of the scan engine  
$ clamscan -V # Capital V  
# To scan a specific folder or file  
$ clamscan /path or file  
# To scan a specific path and its sub-folders  
$ clamscan -r /path  
# To scan and display only the infected files  
$ clamscan -r -i /path  
# To save the scan report  
$ clamscan -l /path/log-file-name /path  
# Move the infected files to a different folder  
$ clamscan --move /directory /folder-or-file-to-scan  
# Delete the infected files  
$ clamscan --remove yes /folder-or-file-to-scan  
# Notify the user with a bell sound on identifying infected files  
$ clamscan --bell /folder-or-file-to-scan  
# Different options in single go  
$ clamscan -r --bell --remove yes -l /path/log-file-name \  
/folder-or-file-to-scan
```

Freshclam runs as a background service and updates the virus signature database in regular intervals. If you would like to manually perform an update, execute the below commands in the terminal.

This would stop the Freshclam service and updates the db and finally starts the service back.

```
$ sudo systemctl stop clamav-freshclam.service
$ sudo freshclam
$ sudo systemctl start clamav-freshclam.service
See : man clamscan
```

1.7 reboot : Restart or reboot the system.

Using 'reboot' command to restart our Linux system

```
$ sudo reboot
```

Force Immediate reboot in Linux system:

```
$ sudo reboot -f
```

1.8 Zeal : Offline documentation browser

Zeal : Offline documentation browser
for C,C++,Python,BASH etc.

1.9 wine : run microsoft software in Linux.

Wine is a free and open-source compatibility layer that aims to allow application software and computer games developed for Microsoft Windows to run on Unix-like operating systems.

Wine also provides a software library, named Winelib, against which developers can compile Windows applications to help port them to Unix-like systems

1.10 yes : accept everything...

- Repeatedly output "message":
yes {{message}}
- Repeatedly output "y":
yes
- Accept everything prompted by the apt-get command:
yes | sudo apt-get install {{program}}

1.11 uptime : tells how long the system is running

Tell how long the system has been running

- Print current time, uptime, number of logged-in users and other information:

```
$ uptime
```

- Show only the amount of time the system has been booted for:

```
$ uptime --pretty
```

- Print the date and time the system booted up at:

```
$ uptime --since
```

1.12 sleep : delay for a specified amount of time

`sleep NUMBER[SUFFIX]...`

Pause for NUMBER seconds.

SUFFIX may be

- 's' for seconds (the default),
- 'm' for minutes,
- 'h' for hours or
- 'd' for days.

NUMBER need not be an integer.

Given two or more arguments, pause for the amount of time specified by the sum of their values.

- Delay in seconds:

```
sleep {{seconds}}
```

Example: `sleep 3`

- Execute a specific command after 20 seconds delay:

```
sleep 20 && {{command}}
```

1.13 Linux..O.S..version..check

- `lsb_release -a` #lsb is for LinuxStandardBase:
- `cat /etc/os-release`
- `hostnamectl`
- `cat /proc/version`

1.14 bc : "Basic Calculator" - simplified man pages

[https://en.wikipedia.org/wiki/Bc_\(programming_language\)](https://en.wikipedia.org/wiki/Bc_(programming_language))

bc, for basic calculator, is "an arbitrary-precision calculator language" with syntax similar to the C programming language.

bc is typically used as either a mathematical scripting language or as an interactive mathematical shell.

```
scale = 5 : set precision to 5 numbers after the ".".
```

bc can be used non-interactively, with input through a pipe.

This is useful inside shell scripts:

```
$ result=$(echo "scale=2; 5 * 7 /3;" | bc)
$ echo $result
```

1.15 cal : "CALendar" - basic calendar

[https://en.wikipedia.org/wiki/Cal_\(command\)](https://en.wikipedia.org/wiki/Cal_(command))

see ncal command with slightly advanced options

Prints calendar information, with the current day highlighted

- Display a calendar for the current month:
cal
- Display previous, current and next month:
cal -3
- Display a calendar for a specific year (4 digits):
cal 2024
- Display a calendar for a specific month and year:
cal 4 2024
- Display calendar with working list numbers (example):
cal -W 2024

1.16 date : show current date with format

Return current date

```
$ date          #return current date
Wed 13 Sep 2023 15:31:52 IDT
```

```
$ date "+%Y"    # year long format
2023
```

```
$ date "+%y"    # year short format
23
```

```
$ date "+%m"    # month
03
```

```
$ date "+%b"    #
Mar
```

```
$ date "+%d"    # day
13
```

```
$ date "+%S"    # seconds ?
33
```

```
$ date "+%H"    # hours
15
```

```
$ date "+%M"    # minutes
35
```

```
$ date "+%Y-%m-%d %H:%M"
2023-09-13 15:36
```

Example:

```
$ x='date +%Y' # command substitution
```

```
$ echo $x
2024
$ x=$(date +%Y)
$ x=$(date +%Y)
$ echo $x
2024
```

1.17 hdate : Hebrew DATE

Display Hebrew date.

```
-r --parasha      print weekly reading for Shabbat.
-h --holidays     print holidays.
-c               print Shabbat start/end times.
```

```
Location of Haifa : Haifa      32, 34, 2
hdate -sRH 02 2024 -l32 -L34 -z2 : sun rize+sunset (Haifa)+
                                weekly reading+holidays
```

2 System administration basics

2.1 lspci : List all PCI devices

display information about PCI buses in the system and devices connected to them.

```
-v - Be verbose, Include the Kernel Drivers for each device and Modules
    Modules - drivers in Linux.
-vv - Be more verbose. Show a more detailed version of the last command.
-vvv - Be as verbose as possible. Show all the details.

-t - Show a tree-like diagram containing all buses, bridges, devices
    and connections between them.

-b - Bus-centric view. Show all IRQ numbers and addresses as seen by the
    cards on the PCI bus instead of as seen by the kernel
```

Example :

```
$ lspci -t -v
+-[10000:e0]--+-17.0 Intel Corporation Alder Lake-P SATA AHCI Controller
|               +-1c.0 Intel Corporation Device 09ab
|               \-1c.4-[e1]----00.0 Intel Corporation Device f1aa
\-[0000:00]--+-00.0 Intel Corporation Device 4601
              +-02.0 Intel Corporation Device 46a8
```

```

+-04.0 Intel Corporation Alder Lake Innovation Platform
       Framework Processor Participant
+-08.0 Intel Corporation 12th Gen Core Processor Gaussian &
       Neural Accelerator
+-0e.0 Intel Corporation Volume Management Device NVMe RAID
       Controller
+-14.0 Intel Corporation Alder Lake PCH USB 3.2 xHCI Host
       Controller
+-14.2 Intel Corporation Alder Lake PCH Shared SRAM
+-14.3 Intel Corporation Alder Lake-P PCH CNVi WiFi
+-15.0 Intel Corporation Alder Lake PCH Serial IO I2C Controller
       #0
+-15.1 Intel Corporation Alder Lake PCH Serial IO I2C Controller
       #1
+-16.0 Intel Corporation Alder Lake PCH HECI Controller
+-17.0 Intel Corporation Device 09ab
+-1d.0-[01]----00.0 Realtek Semiconductor Co., Ltd.
       RTL8111/8168/8411 PCI Express Gigabit Ethernet Controller

```

2.2 kernel..modules : Linux drivers

```

module - Linux drivers
drivers are in files named *.ko (Kernel Object)
Add Code \Functionality to the Linux kernel while it is running.
$ lsmod                # list kernel modules (drivers)
$ insmod module_name.ko # load kernel modules
$ rmmod module_name     # unload kernel modules

```

Example

```

$ lsmod
Module                Size  Used by
rfcomm                98304  4
ccm                   20480  6
cmac                  12288  3
algif_hash            12288  1
algif_skcipher        12288  1
spi_intel_pci         12288  0
i2c_hid               40960  1 i2c_hid_acpi
realtek               36864  1
i2c_smbus             16384  1 i2c_i801
idma64                20480  0
xhci_pci_renesas      20480  1 xhci_pci
vmd                   24576  0
spi_intel             32768  1 spi_intel_pci

```

2.3 dmesg..modules : Show kernel messages

```

dmesg : Show kernel drivers messages

```

dmesg is used to examine or control the kernel ring buffer.

```
$ dmesg [ -c ] [ -n level ] [ -s bufsize ]
```

```
$ sudo dmsg -c # -c is for clear after previous messages.
after USB flash/ USB developement connection we will see
USB messages.
```

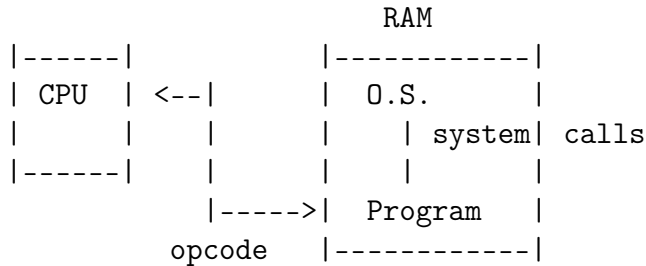
- Show kernel messages:
\$ dmesg
- Show kernel error messages:
\$ dmesg --level err
- Show kernel messages and keep reading new ones, similar to tail -f (available in kernels 3.5.0 and newer):
\$ dmesg -w
- Show how much physical memory is available on this system:
\$ dmesg | grep -i memory
- Show kernel messages 1 page at a time:
\$ dmesg | less
- Show kernel messages with a timestamp (available in kernels 3.5.0 and newer):
\$ dmesg -T
- Show kernel messages in human-readable form (available in kernels 3.5.0 and newer):
\$ dmesg -H
- Colorize output (available in kernels 3.5.0 and newer):
\$ dmesg -L

The program helps users to print out their bootup/device driver messages.

- c: Clear the ring buffer contents after printing.
- sbufsize: Use a buffer of size bufsize to query the kernel ring buffer. This is 16392 by default.
- nlevel : Set the level at which logging of messages is done to the console.
For example, -n 1 prevents all messages, except panic messages, from appearing on the console.

All levels of messages are still written to /proc/kmsg

2.4 strace : trace system calls of a program



Program communicates with CPU through opcode and with O.S. with system calls, within program there are opcodes and system calls.

With this assembly code (file *.s) there are opcodes and system calls, for example program hello_world.c:

```
-----  
# include <stdio.h>
```

```
int main(void) {  
    printf ("Hello world");  
}
```

```
-----  
present assembly source file:  
$gcc -S hello_world.c
```

```
$ cat hello_world.s  
.file "hello_world.c"  
.text  
.section .rodata  
.LC0:  
.string "Hello world"  
.text  
.globl main  
...  
movl $0, %eax  
call printf@PLT # here call is SYSTE CALL for Linux Kernell  
movl $0, %eax  
popq %rbp
```

```
-----  
$ gcc hello_world.c -o hello  
$ ./ hello
```

```
$ strace ./hello  
execve("./hello", [ "./hello" ], 0x7ffd79c58fc0 /* 46 vars */) = 0  
brk(NULL) = 0x64c5259e8000  
arch_prctl(0x3001 /* ARCH_??? */, 0x7ffe171e4d90) = -1 EINVAL  
(Invalid argument)  
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0)  
= 0x743fb8c0c000  
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or
```

```

directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=93767, ...},
AT_EMPTY_PATH) = 0
mmap(NULL, 93767, PROT_READ, MAP_PRIVATE, 3, 0) = 0x743fb8bf5000
close(3)                                     = 0

```

Used for tracing the system calls of a program.
when it is run in conjunction with a program,
it outputs all the calls made to the kernel by the program.

In many cases, a program may fail because it is
unable to open a file or because of insufficient memory.
And tracing the output of the program will clearly show the
cause of either problem.

Usage:

```
$ strace <name of the program>
```

Example:

```
$ strace ls
```

this will output a great amount of data on to the screen.
If it is hard to keep track of the scrolling mass of data, then there is
an option to write the output of strace to a file instead which is done
using the -o option:

```
$ strace -o strace_ls_output.txt ls
```

write all the tracing output of 'ls' to the 'strace_ls_output.txt' file.

```

$ strace echo "Hello World"
execve("/usr/bin/echo", ["echo", "Hello World"], 0x7ffc1265a308
/* 45 vars */) = 0
brk(NULL)                                     = 0x643239cd2000
arch_prctl(0x3001 /* ARCH_??? */, 0x7ffc5285b970) = -1 EINVAL
(Invalid argument)
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS,
-1, 0) = 0x75c5d1538000
access("/etc/ld.so.preload", R_OK)           = -1 ENOENT
(No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=93767, ...},
AT_EMPTY_PATH) = 0
mmap(NULL, 93767, PROT_READ, MAP_PRIVATE, 3, 0) =
0x75c5d1521000
close(3)                                     = 0
....

```

....

2.5 uname : print system information

print all information:

```
$ uname -a
Linux amits-vostro-3520 6.5.0-25-generic #25~22.04.1-Ubuntu SMP
PREEMPT_DYNAMIC Tue Feb 20 16:09:15 UTC 2 x86_64 x86_64 x86_64
GNU/Linux
```

```
-a | all info
-s | kernel name
-n | network node hostname
-r | kernel release
-v | kernel version
-m | machine hw name
-p | processor type
-i | hardware platform
-o | operating system
```

print the kernel release

```
$ uname -r
6.5.0-25-generic
```

2.6 service..systemctl : service / daemon

A "service" is a program that starts automatically when you start your computer, and runs in the background.

For example, the "network" service sets up your connection to the Internet and keeps it running correctly.

```
service <service-name> status - Check if a service is running:
service <service-name> start  - Starting a service:
service <service-name> stop   - Stopping a service:
```

For example:

```
$ service network [start |stop| restart]
or
$ systemctl [start |stop| restart] network)
```

```
$ sudo systemctl status ssh    #check the status
$ sudo systemctl stop ssh      #stop ssh for this session
```

```
$ sudo systemctl start ssh    #start ssh for this session
$ sudo systemctl disable ssh  #stop ssh from boot to computer
$ sudo systemctl enable ssh   #start ssh from boot to computer
```

you can also modify services by `/etc/init.d/service-name action_taken` (status, stop and start too).

look in `/etc/init.d/` for the services available.

3 Bash Features and Scripting

3.1 only..bash..features

```
# (( )) $(( )) ${ }
above are features of BASH only
not other terminals as ZASH ... ("bashizm")
```

3.2 run..BASH..script../bin/bash

```
#! ----> named "hash bang" or in short form "shebang"
which bash
/usr/bin/bash
```

By default, every text file can be run as BASH script (file permissions must be executable):

```
$ cat hi555
echo hi
$ ./hi555
hi
$ ls -l hi555
-rwxrw-r-- 1 amit amit 8 Oct  8 22:01 hi555
```

What happens if we add our own command "cat" that prints "miay" at the top of the PATH: our "cat" now is the first in search order.

Prepare following script named "cat" :

```
$ nano cat
-----
#!/bin/bash
echo miay !!!
-----
$ PATH=$PWD:$PATH
$ cat
miay!!!!
```

3.3 "hashbang"... "shebang"

[https://en.wikipedia.org/wiki/Shebang_\(Unix\)](https://en.wikipedia.org/wiki/Shebang_(Unix))

For bash script:

```
$ which bash
/usr/bin/bash
```

```
#!/ symbol in named "hash bang" or shortly "shebang"
#!/usr/bin/bash
```

For Python:

<https://en.wikipedia.org/wiki/Env>

env may also be used in the hashbang line of a script to allow the interpreter to be looked up via the \$PATH.

For example, here is the code of a Python script:

```
#!/usr/bin/env python3
print("Hello, World!")
```

In this example, /usr/bin/env is the full path of the env command. The environment is not altered, i.e. the original \$PATH will be used. Note that it is possible to specify the interpreter without using env, by giving the full path of the python interpreter. A problem with that approach is that on different computer systems, the exact path may be different. By instead using env as in the example, the interpreter is searched for and located at the time the script is run

3.4 stop..bash..script..at...error

<https://ioflood.com/blog/bash-exit-on-error/>

Using 'set -e'

The simplest way to make a bash script exit when an error occurs is by using the 'set -e' command at the beginning of your bash script. This command instructs the bash script to stop execution if any command it runs returns a non-zero exit status, which is the convention for indicating an error in bash.

```
-----
#!/bin/bash
set -e
-----
```

Using 'set -o pipefail'

The 'set -o pipefail' command ensures that your script exits

if any command in a pipeline fails, not just the last one.
This can be useful in scripts where you're chaining together multiple commands using pipes.

```
-----  
#!/bin/bash  
set -o pipefail  
-----
```

Using 'set -u'

The 'set -u' command causes your script to exit if it tries to use an uninitialized variable. This can help catch typos and other bugs related to variables.

```
-----  
#!/bin/bash  
set -u  
-----
```

- Execute a specific script and stop at the first [e]rror:
\$ bash -e {{path/to/script.sh}}

3.5 bash..debug

Execute a specific script while printing each command before executing it:
Usefull for debug !

```
$ bash -x {{path/to/script.sh}}
```

Print verbose information for each step (For Debug).

```
$ env -v echo hello world  
executing: echo  
arg[0]= 'echo'  
arg[1]= 'hello'  
arg[2]= 'world'  
hello world
```

Get warning on undefined variable use :

```
$ set -u # To  
$ echo [$b]  
[]  
$ set -u  
$ echo [$b]  
bash: b: unbound variable # warning on undefined var.  
$ set +u  
$ echo [$b]  
[]
```

3.6 shopt : SHell OPTions manage

List shell definitions.
set/unset options

Options:

-p print each shell option with an indication of its status
-s enable (set) each OPTNAME
-u disable (unset) each OPTNAME

- s : set
- u : unset

Examples :

```
$ shopt -s globstar
```

to return failure if glob did not get result::

```
$ shopt -s failglob
$ echo [j-z].txt
bash: no match: [j-z].txt
$ shopt -u failglob
$ echo [j-z].txt
[j-z].txt
```

to enable ** recursive glob :

```
$ shopt -s globstar // set recursive glob on
$ shopt globstar
globstar      on
$ echo **/*.txt # recursive glob:
                # find in the current dir (!)
                # and all sub dirs under it
1.txt 2.txt 3.txt 4.txt 5.txt # files from current dir also
temp1/a.txt temp1/b.txt temp1/c.txt temp1/d.txt
temp2/10.txt temp2/1.txt temp2/2.txt temp2/3.txt
temp2/4.txt temp2/5.txt temp2/6.txt temp2/7.txt
temp2/8.txt temp2/9.txt
$ shopt -u globstar
$ shopt globstar
globstar      off
$ echo **/*.*   # now it will search only in sub dirs
temp1/a.txt temp1/b.txt temp1/c.txt temp1/d.txt
temp2/10.txt temp2/1.txt temp2/2.txt temp2/3.txt
temp2/4.txt temp2/5.txt temp2/6.txt temp2/7.txt
temp2/8.txt temp2/9.txt
```

3.7 set : Change shell attributes / display shell vars

<https://phoenixnap.com/kb/linux-set>

```
$ type set
set is a shell builtin
```

Change the value of shell attributes and positional parameters, or display the names and values of shell variables.

set command: warnings for undefined variables.
set command is similar to shopt.

The set command is a built-in Linux shell command that displays and sets the names and values of shell and Linux environment variables.

On Unix-like operating systems, the set command functions within the Bourne shell (sh), C shell (csh), and Korn shell (ksh).

If we type set without any additional parameters, we will get a list of all shell variables, environmental variables, local variables, and shell functions:

```
$ set
```

This is usually a huge list. You probably want to pipe it into a pager program to more easily deal with the amount of output:

```
$ set | less
```

```
-----
Options to set/unset warning on undefined variable.
```

```
-u Treat unset variables as an error when substituting.
```

```
+u Return warning on undefined variable.
```

```
-----
set -e to exit immediately when there is an error in script
and don't try to continue run
-----
```

```
$ set -u # To get warning on undefined variable use
```

```
$ echo [$b]
```

```
[]
```

```
$ set -u
```

```
$ echo [$b]
```

```
bash: b: unbound variable      # warning on undefined var.
```

```
$ set +u
```

```
$ echo [$b]
```

```
[]
```

```
$ help set
```

```
set: set [-abefhkmnptuvxBCHP] [-o option-name] [--] [arg ...]
```

```
Set or unset values of shell options and positional parameters.
```

```
Change the value of shell attributes and positional parameters, or display the names and values of shell variables.
```

```
-u Treat unset variables as an error when substituting.
```

```
+u Return warning on undefined variable.
```


Split Strings

Use the set command to split strings based on spaces into separate variables. For example, split the strings in a variable called myvar, which says "This is a test".

```
$ myvar="This is a test"
$ set -- $myvar
$ echo $1
$ echo $2
$ echo $3
$ echo $4
This
is
a
test
```

3.8 chaining..operators explanation

<https://www.geeksforgeeks.org/chaining-commands-in-linux/>

Chaining commands in Linux allows us to execute multiple commands at the same time and directly through the terminal.

It's like short shell scripts that can be executed through the terminal directly. Linux command chaining is a technique of merging several commands such that each of them can execute in sequence depending on the operator that separates them and these operators decide how the commands will get executed. It allows us to run multiple commands in succession or simultaneously.

Some commonly used chaining operators are as follows:

Operators	Function
& (Amperсанд)	This command sends a process/script/command to the background.
&& (Logical AND)	The command following this operator will only execute if the command preceding this operator has been successfully executed.
; (Semi-colon)	The command following this operator will execute even if the command preceding this operator is not successfully executed.
(Logical OR)	The command succeeding this operator is only executed if the command preceding it has failed.
(Pipe)	The output of the first command acts as input to the second command.
! (NOT)	Negates an expression within a command. It is

	much like an “except” statement.
>,>>,<(Redirection)	Redirects the output of a command or a group of commands to a file or stream.
&&- (AND-OR)	It is a combination of AND OR operator and is similar to the if-else statement.
\ (Concatenation)	Used to concatenate large commands over several lines in the shell.
() (Precedence)	Allows command to execute in precedence order.
{ } (Combination)	The execution of the command succeeding this operator depends on the execution of the first command.

3.9 | : pipe - redirect stdout of one cmd to stdin of another cmd.

- “Pipe” redirects standard output of one command to standard input of another command
- t1, t2, tL are “temporary files” in this explanation.
- “|” acts as temporary file.
- Pipe redirects only REGULAR input/output, not error

```

keyboard --> command1 ---> | ---> command2 ---> |--> command3---> display
command1 > t1
command2 <t1 >t2
command3 > tL

```

- Example 1:
Redirect output of (print of all files *.log)
to (grep -i (insensitive) that search lines with “error”)
and sort all this lines:
\$ cat *log|grep -i error|sort
- Example 2:
Redirect (result of recursive insensitive search ,
starting with current directory) to (search of lines that not
include word “ignored”), the (sort the results with “unic”)
and send the result to file serious_errors.log:
\$ grep -ri error .|grep -v “ignored”|sort u> serious_errors.log
- Example 3 (YES command usage):
\$ cat dir1_rm_inputs.txt
y
y
n
n
y
y

```

y
$ cat dir1_rm_inputs.txt | rm ./dir1/*.* -i

$ rm -i dir2/*
rm: remove regular empty file 'dir2/6.txt'? n
rm: remove regular empty file 'dir2/7.txt'? n
rm: remove regular file 'dir2/errors2.log'? n
rm: remove regular file 'dir2/errors3.log'? n
rm: remove regular file 'dir2/errors.log'? n
$ yes | rm -i dir2/*
rm: remove regular empty file 'dir2/6.txt'? rm: remove regular
empty file 'dir2/7.txt'? rm: remove regular file 'dir2/
errors2.log'? rm: remove regular file 'dir2/errors3.log'? rm:
remove regular file 'dir2/errors.log'?

```

- Example: automation to enter “yes” when install:

```
$ yes | sudo apt install some_programs
```

- apt has -flag to allow install without interrupts.

3.10 ; : send several commands in BASH

run several commands one after another:

```
$ echo hello ; sleep 1;echo world
hello
world
```

```
$ echo A; echo B; echo C
A
B
C
```

This will output A, wait 3 sec., clear line and output B:

```
$ printf "A"; sleep 3; printf "\rB\n"
B
```

3.11 || : logical OR

The command succeeding this operator will only execute if the execution of the command preceding it fails.

Exit status: success: 0, others - fail

Exit status of grep command:

Normally the exit status is
0 if a line is selected,

1 if no lines were selected,
2 if an error occurred.
However, if the -q or --quiet or --silent is used and a line is selected, the exit status is 0 even if an error occurred.

Short circuiting:

x || x || executed : stops on first success;

Example: Do the same task with several alternatives automatically.

Stop in case of first success in one of tasks.

Example:

```
$ ls
  latex-document-prototype.tex  try.docx  try.html  try.pdf  try.tex
$ ll try.pdf || echo "2-nd" || echo "3 -rd"
-rw-rw-r-- 1 amits amits 256176 Feb 28 12:11 try.pdf
$ ll 222.txt || echo "2-nd" || echo "3 -rd"
ls: cannot access '222.txt': No such file or directory
2-nd
```

Example of "turnery if":

```
$ ls
  latex-document-prototype.tex  try.docx  try.html  try.pdf  try.tex
$ ll try.pdf && echo "OK !" || echo "FAILED !"
-rw-rw-r-- 1 amits amits 256176 Feb 28 12:11 try.pdf
OK !
$ ll 222.txt && echo "OK !" || echo "FAILED !"
ls: cannot access '222.txt': No such file or directory
FAILED !
```

3.12 && : logical AND

Exit status: success: 0, others - fail.

The command succeeding this operator will only execute if the command preceding it executed successfully

v && v && executed : stops on first fail

Example: run chain of commands automatically, where we interested that all the commands in the chain will run successfully.

Stop at first fail.

Example:

```
$ echo "1-st" && ll 222.txt && echo "3-rd"
1-st
ls: cannot access '222.txt': No such file or directory
```

3.13 backslash : Split commands in BASH

To split the command in command line into number of lines we use back slash symbol \ :

```
$ ls\  
> -l
```

3.14 & symbol : start process in the background immediately

& Symbol

Another method to run a process in the background is by appending the ampersand symbol & to the command you execute.

This symbol tells the shell to start the process in the background immediately.

To just start the command in the background at the beginning: all you need to do is put an ampersand at the end of any Linux command. This will allow to use terminal along with running the command:

Example:

```
$ gzip largefile.txt &
```

In this example, the gzip command starts running in the background as soon as you execute it.

The process ID (PID) will be displayed on the terminal, allowing you to continue using the shell while the process completes silently.

It's important to note that when using the & symbol, the process may still produce output, which can interfere with the prompt on the terminal.

Run several commands in background :

```
$ xed & nemo & geany &  
[1] 19420  
[2] 19421  
[3] 19422
```

Note:

To avoid this, it's recommended to redirect the output to a file or to /dev/null, which discards the output. You can do this by appending > filename or > /dev/null to the command, respectively.

3.15 bash : Start an interactive shell session

when the variable is defined in following form

it is used in internal bash !

\$ x=123 bash # x will be defined in internal bash

\$ echo \$x

123

\$ exit

exit

\$ echo \$x

2024

- Execute a specific script:

Allows to run script without execution permissions !

\$ bash {{path/to/script.sh}}

bash command allows to run additional bash session within parent bash window.
To exit it use exit command.

- Start an interactive shell session:

\$ bash

- Start an interactive shell session without loading startup
configs:

\$ bash --norc

- Execute a specific script while printing each command before executing it:
Usefull for debug !

\$ bash -x {{path/to/script.sh}}

- Execute a specific script and stop at the first [e]rror:

\$ bash -e {{path/to/script.sh}}

Exit bash session :

\$ exit

\$SHLVL presents shell level:

\$ bash

\$ echo \$SHLVL

2

\$ bash

\$ echo \$SHLVL

3

\$ exit

exit

\$ echo \$SHLVL

```

2
$ exit
exit
$ echo $SHLVL
1

```

3.16 SHELL.variables..vs..Env.variables

[https://www.digitalocean.com/community/tutorials/how-to-read-and-set-environmental-](https://www.digitalocean.com/community/tutorials/how-to-read-and-set-environmental-variables-on-linux)

<https://www.digitalocean.com/community/tutorials/how-to-read-and-set-environmental-and-shell-variables-on-linux>

By convention, Env. and Shell variables are usually defined using all capital letters. This helps users distinguish environmental variables within other contexts

for example command grep uses environmental variables:

```

$ man grep
...
ENVIRONMENT
The behavior of grep is affected by the following environment
variables.... LC_ALL ... LANG ... GREP_COLOR ...
...

```

Shell variables :

variables that are contained exclusively within the shell in which they were set or defined. They are often used to keep track of ephemeral data, like the current working directory. Shell variable shouldn't be passed on to any child processes

Environmental variables :

variables that are defined for the current shell and are inherited by any child shells or processes. Environmental variables are used to pass information into processes that are spawned from the shell.

Env variables are local to the program/process that use by them and they children and are deleted after the parent program run finish.

For example, environ for C programs allows to use env. variables. Java uses CLASS PATH and JAVA HOME variables that define where

to find executable files to run them.

Creating shell variable.

```
$ TEST_VAR='Hello World!'
```

```
$ echo $TEST_VAR
```

```
    Hello World!
```

We can see this by grepping for our new variable within the set output:

```
$ set | grep TEST_VAR
```

```
    TEST_VAR='Hello World!'
```

We can verify that this is not an environmental variable by trying the same thing with printenv:

```
$ printenv | grep TEST_VAR
```

```
    # No output should be returned.
```

Accessing the value of shell or environmental variable:

```
$ echo $TEST_VAR
```

```
    Hello World!
```

As you can see, reference the value of a variable by preceding it with a \$ sign. The shell takes this to mean that it should substitute the value of the variable when it comes across this.

So now we have a shell variable TEST_VAR.

It shouldn't be passed on to any child processes.

We can spawn a new bash shell from within our current one to demonstrate:

```
$ bash # open child bash session inside the existing one
```

```
$ echo $TEST_VAR
```

If we type bash to spawn a child shell, and then try to access the contents of the variable, nothing will be returned. This is what we expected.

Get back to our original shell by typing exit:

```
$ exit
```

```
$ TEST_VAR="current BASH var" # must be without spaces  
                                around !!!!
```

```
$ echo $TEST_VAR
```

```
    current BASH var
```

```
$ bash
```

```
$ echo $TEST_VAR
```

```
    # no output
```

```
$ exit # return to parent BASH shell
```

```
    exit
```

```
$ echo $TEST_VAR
```

```
    current BASH var
```


Creating Environmental Variables.

Now, let's turn our shell variable into an environmental variable. We can do this by exporting the variable. The command to do so is appropriately named:

```
$ export TEST_VAR
```

This will change our variable into an environmental variable.

We can check this by checking our environmental listing again:

```
$ printenv | grep TEST_VAR
TEST_VAR=Hello World!
```

Or

```
$ export | grep TEST_VAR
declare -x TEST_VAR="current BASH var"
```

This time, our variable shows up.

Let's try our experiment with our child shell again:

```
$ bash
$ echo $TEST_VAR
Hello World!
```

Great! Our child shell has received the variable set by its parent.

Before we exit this child shell, let's try to export another variable.

We can set environmental variables in a single step like this:

```
$ export NEW_VAR="Testing export"
```

Test that it's exported as an environmental variable:

```
$ printenv | grep NEW_VAR
NEW_VAR=Testing export
```

Now, let's exit back into our original shell:

```
$ exit
```

Let's see if our new variable is available:

```
$ echo $NEW_VAR
$
```

Nothing is returned.

This is because environmental variables are only passed to child processes. There isn't a built-in way of setting environmental variables of the parent shell. This is good in most cases and prevents programs from affecting the operating environment from which they were called.

Example of shell vars vs. env. vars:

```
$ a=5
$ b=7
$ export b
$ bash
$ echo $a
$
$ echo $b
```

```

7
$ exit
exit
$ echo $a
5
$ echo $b
7

```

3.17 "=" : assign value for Shell/Env. variables.

Every shell/env variable is string !

Example of define:

```

$ TEST_VAR1 = "current BASH var"
TEST_VAR1: command not found # because of spaces
                                around "=" !!!
$ TEST_VAR1="current BASH var" # must be entered without
                                spaces around "=" !!!!
$ echo $TEST_VAR1
current BASH var
$ TEST_VAR2="previous and $TEST_VAR1"
$ echo $TEST_VAR2
previous and current BASH var
$ TEST_VAR3='previous and $TEST_VAR1'
$ echo $TEST_VAR3
previous and $TEST_VAR1
$ a=5
$ echo $a
5

```

3.18 examples..of..system..vars

```

$ echo `whoami`
amit
$ echo $(whoami)
amit
$ echo aaa $(whoami) bbb
aaa amit bbb
$ echo aaa $USER bbb
aaa amit bbb
$ echo [$(pwd)]
[/home/amit]
$ echo [$PWD]
[/home/amit]
$ echo [$HOME]
[/home/amit]

```

```
$ echo $HOSTNAME
amits-vostro-3520
$ echo $SHELL # current shell name
/bin/bash
```

3.19 \$PATH : path system variable

If the command is not built in command in bash, bash performs the search for the command by using path in \$PATH variable. The search is done according to the order of directories in \$PATH variable.

It might be a good idea to create a directory ~/scripts to hold your scripts. Add the directory to the contents of the PATH variable:

```
$ export PATH="$PATH:~/scripts"
```

3.20 \$SHELL : current shell type)

<https://unix.stackexchange.com/questions/43499/difference-between-echo-shell-and-which-bash>

```
$ echo $SHELL
/bin/zsh
$ which bash
/bin/bash
```

The first command tells me which shell will be executed by login when you log in. In my case, /bin/zsh.

The second command tells me the first occurrence in my \$PATH the bash command can be found.

3.21 \$SHLVL : SHell LeVeL check shell level

Allows to check the shell's level
bash command runs additional bash session within parent bash window.
To exit it use exit command.

\$SHLVL presents shell level:

```
$ bash
$ echo $SHLVL
2
$ bash
$ echo $SHLVL
3
$ exit
exit
$ echo $SHLVL
```

```
2
$ exit
exit
$ echo $SHLVL
1
```

3.22 \$PS1 : Prompt String 1

```
$ echo $PS1
$ echo $PS1 >> ~/.profile
```

3.23 \$BASH_VERSION : result of last command execution

```
$ echo "Bash version $BASH_VERSION"
Bash version 5.1.16(1)-release
```

3.24 \$? : result of last command execution

```
$ ls -la
total 772
drwxrwxr-x 2 amits amits 4096 Feb 28 12:12 .
drwxrwxr-x 8 amits amits 4096 Feb 28 11:42 ..
-rw-rw-r-- 1 amits amits 256176 Feb 28 12:11 try.pdf
-rw-rw-r-- 1 amits amits 148037 Feb 28 11:41 try.tex
$ echo $?
0
https://www.baeldung.com/linux/check-if-command-executed-successfully#:~:text=We%20can%20use%20a%20binary%20comparison%20operator%20eq,the%20pwd%20command%20displays%20the%20present%20working%20directory.
```

We can use a binary comparison operator eq to check the outcome of the previous command, stored in the \$? variable:

```
#!/bin/bash
pwd
if [ $? -eq 0 ]; then
    echo "Command Executed Successfully"
else
    echo "Command Failed"
fi{verbatim}
```

3.25 \$! : PID of last executed program

Present PID of last program:

```
$ geany 1.txt & disown
```

```
[1] 8816
$ echo $!
8816
```

3.26 \$\$: current bash Process ID

```
$ echo $$      # present current BASH pid
6425
```

3.27 \$PROMPT_DIRTRIM : PROMPT DIRectory TRIMmer

For people looking for a much simpler solution and don't need the name of the first directory in the path, Bash has built-in support for this using the `PROMPT_DIRTRIM` variable. For example:

```
$ PROMPT_DIRTRIM=2
```

If set to a number greater than zero, the value is used as the number of trailing directory components to #retain when expanding the `\w` and `\W` prompt string escapes (see [Printing a Prompt](#)). Characters removed are replaced with an ellipsis.

3.28 which : Locate a program in the user's PATH.

```
$ type -a which
which is /usr/bin/which
which is /bin/which
```

which is not BASH built in
which locates a program in the user's `$PATH`.
Search the `$PATH` env. variable and display the location of any matching executables.

- Search the `$PATH` environment variable and display the location of any matching executables:

```
$ which {{executable}}
```

- If there are multiple executables which match, display all:

```
$ which -a {{executable}}
```

Example :

```
$ which -a time
/usr/bin/time
/bin/time
```

3.29 export : set or present exported variables

Export command is used to add system variable.
It is session specific i.e., when Bash window is closed, it became not valid. If you want your variable to be set always in your shell, add it to .bashrc file:

1. Make sure your .bashrc exists
`ls -A ~/.bashrc`
2. Add your variable at the bottom
`export VARIABLE=value`
3. Save, then open a new terminal and verify the variable is set
`$ echo $VARIABLE`
value

```
$ type export
export is a shell builtin
```

To present all exported variables :

```
$ export
```

Example of set env variable:

```
$ NEW_VAR="Testing export"
$ export NEW_VAR
```

We can set environmental variables in a single step like this:

```
$ export NEW_VAR="Testing export"
```

If no names are supplied, or if the -p option is given, a list of names of all exported variables is displayed:

```
$ export | head
declare -x COLORTERM="truecolor"
declare -x DESKTOP_SESSION="ubuntu"
declare -x DISPLAY=":0"
declare -x GDMSESSION="ubuntu"
$ export | grep TEST_VAR
declare -x TEST_VAR="current BASH var"
```

also see declare that can set exported variable and non exported variable.

3.30 .bashrc : (BASH Run Comm.) runs every time new bash opened

.bashrc file location: File bash.bashrc
is located in /etc/ folder.

For the current user it must be located at home dir:

```
~/.bashrc
/home/amits/.bashrc
```

The `.bashrc` file is a script file that's executed when a user logs in.

The file itself contains a series of configurations for the terminal session. This includes setting up or enabling: coloring, completion, shell history, command aliases, exported variables and more.

Export command when entered in bash terminal is used to add system variable. It is session specific i.e., when Bash window is closed, it became not valid. If you want your variable to be set always in your shell, (as opposed to the superuser's), add it to `.bashrc` file:

1. Make sure your `.bashrc` exists
`ls -A ~/.bashrc`
2. Add your variable at the bottom
`export VARIABLE=value`
3. Save, then open a new terminal and verify the variable is set
`$ echo $VARIABLE`
value

`.bashrc` contains also alias commands.
see <https://linuxhandbook.com/linux-alias-command/>
for example add to `~/.bashrc` file following text:

```
alias lilearn='/home/amits/Documents/RTC/linux_administration/
linux_learn_apps/linux_learn_apps_in_C/
tex_file_data_extract'
```

3.31 `.profile` vs. `.bash_profile` vs. `.bashrc`

<https://www.baeldung.com/linux/bashrc-vs-bash-profile-vs-profile>

The order after login:

1. `/etc/profile`
2. `.bash_profile`
3. `.bash_login` and `.profile` (add: `echo "Hello .profile"`)

On every run of bash terminal:

1. `.bashrc` # runs every time we open new bash
(add `echo "Hello .bashrc"`)

Startup files contain commands that are to be executed on shell startup. As a result, the shell executes commands present in these files automatically to set up the shell.

This happens even before displaying the command prompt.

1. /etc/profile file

In an interactive login shell, Bash first looks for the /etc/profile file. If found, Bash reads and executes it in the current shell. As a result, /etc/profile sets up the environment configuration for ALL users.

2. .bash_profile file

Bash then checks if .bash_profile exists in the home directory. If it does, then Bash executes .bash_profile in the current shell. Bash then stops looking for other files such as .bash_login and .profile.

3. .bash_login and .profile

If Bash doesn't find .bash_profile, then it looks for .bash_login and .profile, in that order, and executes the first readable file only.

4. .bashrc

runs at every terminal run.

3.32 alias : Creates aliases

alias command.

ll is alias to ls -alF , see .bashrc file

alias is defined at terminal level.

to allow run at sessin level, add alias to .bashrc file.

Note: alias does not work in bash script !!!

To see the list of aliases:

```
$ alias
```

```
alias alert='notify-send --urgency=low -i "$([ $? = 0 ] && echo
terminal || echo error)" "$(history|tail -n1|sed -e '\''s/^\s*[0-9]\+
\s*//;s/[\;&|]\s*alert$//'\'' )"'
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias l='ls -CF'
alias la='ls -A'
alias liln='/home/amits/Documents/RTC/linux_administration/
linux_learn_apps/linux_learn_apps_in_C/tex_file_data_extract'
```



```
alias ll='ls -alF'
alias ls='ls --color=auto'
```

To define alias:

```
$ alias x='echo '
$ x "hello world"
hello world
```

To remove alias:

```
$ unalias x
$ x hello world
x: command not found
```

3.33 unalias : Remove aliases

Define alias:

```
$ alias x='echo '
$ x "hello world"
hello world
```

Remove alias:

```
$ unalias x
$ x hello world
x: command not found
```

3.34 printenv : Print values of all/specific Env. variables.

Print values of all or specific environment variables

- Display key-value pairs of all environment variables:
\$ printenv | less
- Display the value of a specific variable:
\$ printenv USER
amits
- Display the value of a variable and end with NUL instead of newline:
\$ printenv --null USER
amits \$

3.35 env : Show and set the env. / run a prog. in a modified environment

<https://www.geeksforgeeks.org/env-command-in-linux-with-examples/>

<https://stackoverflow.com/questions/43793040/how-does-usr-bin-env-work-in-a-linux->

1. env is used to either print environment variables.
2. It is also used to run a utility or command in a custom environment.
3. Important : In practice, env has another common use.
It is often used by shell scripts to launch the correct interpreter.
In this usage, the environment is typically not changed (i.e. used current environment).

<https://en.wikipedia.org/wiki/Env>

env may also be used in the hashbang line of a script to allow the interpreter to be looked up via the \$PATH.
For example, here is the code of a Python script:

```
#!/usr/bin/env python3
print("Hello, World!")
```

In this example, /usr/bin/env is the full path of the env command.
The environment is not altered, i.e. the original \$PATH will be used.
Note that it is possible to specify the interpreter without using env, by giving the full path of the python interpreter. A problem with that approach is that on different computer systems, the exact path may be different.
By instead using env as in the example, the interpreter is searched for and located at the time the script is run

- a. Without any argument : print out a list of all environment variables.
\$ env
- b. With -i or -ignore-environment or only - : Ignores the inherited environment and starts with a clean slate :
\$ env -i your_command
This completely clear out the environment, but it does not prevent your_command setting new variables.
- c. With -u or -unset: Unsets a variable.Remove variable from the environment
\$ env -u variable_name
- d. With -0 or -null: End each output line with NULL, not newline
\$ env -0
- e. With -v: Prints verbose information for each step (For Debug).
\$ env -v echo hello world
executing: echo
arg[0]= 'echo'
arg[1]= 'hello'

```
arg[2]= 'world'
hello world
```

```
env [OPTION]... [-] [NAME=VALUE]... [COMMAND [ARG]...]
```

Script Debugging

-x Print commands and their arguments as they are executed.

The set command is especially handy when you are trying to debug your scripts. Use the -x option with the set command to see which command in your script is being executed after the result, allowing you to determine each command's result.

Script Exporting

-a Mark variables which are modified or created for export.

Automatically export any variable or function created using the -a option. Exporting variables or functions allows other subshells and scripts to use them.

Exit When a Command Fails

-e Exit immediately if a command exits with a non-zero status.

Use the -e option to instruct a script to exit if it encounters an error during execution. Stopping a partially functional script helps prevent issues or incorrect results.

Prevent Data Loss

-C If set, disallow existing regular files to be overwritten by redirection of output.

The default Bash setting is to overwrite existing files. However, using the -C option configures Bash not to overwrite an existing file when output redirection using >, >&, or <> is redirected to that file.

Report Non-Existent Variables

-u Treat unset variables as an error when substituting.

The default setting in Bash is to ignore non-existent variables and work with existing ones. Using the -u option prevents Bash from ignoring variables that don't exist and makes it report the issue.

Using + rather than - causes these flags to be turned off.

```
$ type env
env is hashed (/usr/bin/env)
```

- Show the environment variables:
`$ env`
- Run a program.
 Often used in scripts after the shebang (!) for looking up the path to the program:
`$ env {{program}}`
- Clear the environment and run a program:
`$ env -i {{program}}`
- Remove variable from the environment and run a program:
`$ env -u {{variable}} {{program}}`
- Set a variable and run a program:
`$ env {{variable}}={{value}} {{program}}`
- Set multiple variables and run a program:
`$ env {{variable1}}={{value}} {{variable2}}={{value}}
 {{variable3}}={{value}} {{program}}`

3.36 declare : Declare variables and give them attributes

Declare different types of vars

see: <https://tldp.org/LDP/abs/html/declareref.html>

allows to cancell "export" variable

when using -x to set export , or +x to cancell export

- Declare a string variable with the specified value:
`$ declare {{variable}}="{{value}}"`
- Declare an integer variable with the specified value:
`$ declare -i {{variable}}="{{value}}"`

`declare -i number`

The script will treat subsequent occurrences of "number" as an integer.

```
$ number=3
```

```
$ echo "Number = $number"      # Number = 3
```

```
$ number=three
```

```
$ echo "Number = $number"      # Number = 0
```

Tries to evaluate the string "three" as an integer.

Certain arithmetic operations are permitted for

declared integer variables without the need for expr or let.

```
$ n=6/3
```

```
$ echo "n = $n"                # n = 6/3
```

```

$ declare -i n
$ n=6/3
$ echo "n = $n"          # n = 2

- Declare an array variable with the specified value:
  $ declare -a {{variable}}={{item_a item_b item_c}}

-x export declares a variable as available for exporting outside
  the environment of the script itself.
  $ declare -x var3
  $ declare -x var3=373

```

The declare command permits assigning a value to a variable in the same statement as setting its properties.

```

$ type declare
declare is a shell builtin

```

```

To set exported variable :
$ TEST_VAR="current BASH var"
$ declare -x TEST_VAR    # set exported - env var
or
$ declare -x TEST_VAR="current BASH var"

```

```

To unset exported variable :
$ declare + x TEST_VAR    # set non exported - shell var
$ export | grep TEST_VAR
$

```

```

$ echo Enter some text
$ read myvar
$   Hello world
$ echo '$myvar' now equals $myvar
  $myva now equals Hello world

```

3.37 ".vc'.' : Quoting

Normally BASH interprets spaces as argument separators:

```

$ echo Hello          World
Hello World
# Argument 0 is : echo
# Argument 1 is : Hello
# Argument 2 is : World

```

" " used to prevent the shell from interpreting spaces as argument separators. The whole expression within quotes "..." is considered as one argument. All characters between " " are the same argument.

Also " " disables globbing, but does not disable ! and \$:

```
$ echo "You are logged as $USER"
    You are logged as amits
$ echo *
    latex-document-prototype.tex try.docx try.html try.pdf try.tex
$ echo "*"
    *
$ echo *.pdf
    try.pdf
$ echo "*.pdf"
    *.pdf
```

Single quotes preserve all special characters, while double quotes allow some special characters to be interpreted:

```
$ echo "You are logged as $USER"
    You are logged as amits
$ echo 'You are logged as $USER'
    You are logged as $USER
```

see : <https://saturncloud.io/blog/difference-between-single-and-double-quotes-in-bash/>

3.38 "expansion" : made in echo command

echo	\$USER
v	v
echo	shmuel
V	v

argument0 argument1
command !

1. check ALIAS
2. check PATH
3. check BUILT IN

```
$ a=Hello
$ b=World
$ echo "$a $b"
    Hello World
```

"echo" is not path - search for command named "echo".
 Firstly expansion is made.
 After it Hello is substituted as first argument
 World is substituted as second argument.

```
$ echo $a$b
    ABCXYZ
$ echo "$a"$b"
```

```
ABCXYZ
$ echo "$a"123"$b"
ABC123XYZ
```

```
$ a=ec
$ b=ho
$ c=hel
$ d=lo
$ $a$b $c$d
hello
```

```
$ some_path="aaa/bbb/ccc"
$ extended_path="$some_path"/ddd
$ echo $extended_path
aaa/bbb/ccc/ddd
$ extended_path1=ddd/"$some_path"
$ echo $extended_path1
ddd/aaa/bbb/ccc
```

```
$ # print undefined variable (default - NOT ERROR, empty value):
$ echo [$yyyy]
[]
$ a=
$ echo [$a]
[]
$ a=""
$ echo [$a]
[]
```

Problematic example:

```
$ a=123
$ a=$aABC      # problematic: $a here is not separate variable !
$ echo [$a]     # BASH thinks that $aABC is not defined
$ []
```

Fixed example :

```
$ a=123
$ a=${a}ABC     # now it is fixed !
$ echo $a       # BASH thinks that $aABC is not defined
123ABC
```

```
$ echo "hello $USER"
hello amits
$ echo 'hello $USER'
hello $USER
```

Problematic example:

```
$ a="Hello      World"
$ echo $a
Hello World    # echo gets Hello as first argument
```

```

# and Word as second argument
Fixed example :
$ a="Hello      World"
$ echo "$a"      # now the result of expansion is
                  # considered as one alone string
$ Hello      World

```

3.39 'cmd'(backtick) equals \$(cmd): run cmd and substitute its text

Back tick (key beyond ~) 'command' or \$(command) means:
 firstly, run command that is within `` or within \$() and put its
 output text here.

The expression \$(command) is a modern synonym for 'command' which
 stands for command substitution;

```

$ echo whoami
whoami

```

```

$ echo `whoami`
amits

```

```

$ x=`whoami`
$ echo $x
amits

```

```

$ x=$(whoami)
$ echo $x
amits

```

```

$ echo `whoami`
amits

```

```

$ x=$(tldr ls)
$ echo $x
ls List directory contents. More information: https://www.gnu.org/software/coreutils/ls. - List files one
per line:  ls -l  - List all files, including hidden
files:  ls -a  - List all files, with trailing / added
to directory names:  ls -F  - Long format list
(permissions, ownership, size, and modification date)
of all files:  ls -la  - Long format list with size
displayed using human-readable units (KiB, MiB, GiB):  ls -

```

In the example below output of command 2 will not be presented on
 the screen, it will act as input of command 1:

```

$ # store in x the
$ # PID of ping command that is running :

```



```
$ pgrep ping
46034
$ x='pgrep ping'
$ echo $x
46034
$ x=$(pgrep ping)
46034
```

Use of ‘ ‘ vs \$() :

```
$ # COMMAND1 $(COMMAND2 $(COMMAND3))
```

Print hello to the user:

```
$ echo hello to $(whoami)
hello to amit
$ echo hello to 'whoami'
hello to amit
$ echo "You are at 'pwd'"
You are at /home/amits/Desktop
```

Example with problem:

```
-----
$ my_var=Hi there
$ echo '$my_var' # after expansion it is 'Hi there'
                  # and "Hi" is the first argument
Hi: command not found
```

Example without problem:

```
$ var=123
$ echo 'echo $var'
123
$ echo $(echo $(echo $var))
123
```

3.40 math..\$[...]\$((...))...((...)) : command to do simple arithmetic

\$(()) is the same as \$[] : performs integer math operation

```
$ echo $[ 1 + 2 ] # it is not a command
                  # but part of bash features !!!
3
$ echo ${1+2}
3
$ echo $((1+2)) # it is not a command,
                # only part of bash features !!!
3
$ echo $((2*4))
8
$ echo $((5/2))
2
```

```
$ echo $((5>2))
1 # NOTE : in this case true is 1,
  # it is different than test command.
$ echo $((5<2))
0
```

Operator	Operation
+, -, *, /	addition, subtraction, multiply, divide
%	Modulus (Return the remainder after division)
>, <, >=, <=	Note : the result logic in this case is similar to C language.

```
$ x=4
$ y=5
$ ((z=x+y))
$ echo $z
9
$ ((x+=y))
$ echo x
9
$ ((x++))
$ echo x
10
```

```
Result of (( )) returns status code:
$ ((5>17))          # this is COMMAND !!!
$ echo $?
1      # regular "bash" logic
$ ((5<17))
$ echo $?
0
```

```
((5>17)) logic of status code is      : 0 - true, 1 - false
-----
$((5>17)) logic of output expression is : 1 - true, 0 - false
-----
note: if command in bash uses status code !!!
Two following expressions are equivalent :
```

```
if [ "$x" -gt "$y" ]; then echo YES; else echo NO; fi
if ((x>y))                ; then echo YES; else echo NO; fi
```

```
Additional explanation:
((0)) --> fault    ---> status = 1
((1)) --> success  ---> status = 0
```

Recommendation:

```

-----
use (( )) only for boolean conditions : if (( )) ...
use $(( )) only for math : echo $((3+4)) ; x=$((5+8))

use $(( )) only for math operations

-----
$ x=6;y=10
$ ((y=++x))
$ echo "x=$x, y=$y"

$ echo "x=$x, y=$y"
  x=7, y=7

$ x=6;y=10
$ ((y=x++))
$ echo "x=$x, y=$y"

$ echo "x=$x, y=$y"
  x=7, y=6      # y gets x value before increment of x

-----
((x--))
((--x))

```

3.41 let : math - command to do simple arithmetic

let is a builtin function of Bash that allows us to do simple arithmetic. It follows the basic format:

```
$ let <arithmetic expression>
```

Here is a table with some of the basic expressions you may perform. There are others but these are the most commonly used.

Operator	Operation
+, -, *, /	addition, subtraction, multiply, divide
var++	Increase the variable var by 1
var--	Decrease the variable var by 1
%	Modulus (Return the remainder after division)

```

$ let a=5+4
$ echo $a # 9
$ let "a = 5 + 4"
$ echo $a # 9
$ let a++
$ echo $a # 10
$ let "a = 4 * 5"
$ echo $a # 20
$ let "a = $1 + 30"

```

```
$ echo $a # 30 + first command line argument
```

3.42 true : Returns a successful exit status code of 0: \$? = 0

- Return a successful exit code:

```
$ true
```

Example :

```
$ ! true
```

```
$ echo $?
```

```
1
```

3.43 false : Returns a non-zero exit code: \$? = 1

- Return a non-zero exit code:

```
$ false
```

Example :

```
$ ! false
```

```
$ echo $?
```

```
0
```

3.44 tab : autocomplete

Tab : automatic completion

Tab Tab : all options for complete

3.45 type : Display the type of command the shell will execute.

The 'type' is a built-in shell command used to identify the kind of command. For example it checks if it is shell built in or external command.

type without any key :

```
if NAME is an alias,  
shell reserved word,  
shell function,  
shell builtin,  
disk file  
or not found, respectively
```

-t output a single word which is one of

```
'alias',  
'keyword',  
'function',  
'builtin',  
'file'
```

- p Returns the disk file that would be executed
- a Returns all locations containing an executable.

For example:

```
$ type echo
echo is a shell builtin
$ type cat
cat is /usr/bin/cat
$ type code
code is /usr/bin/code
$ type -a time
time is a shell keyword
time is /usr/bin/time
time is /bin/time
```

3.46 Comments in BASH

comments in BASH :

```
#comment
```

3.47 source/"." : Execute commands from a file in the current shell

Execute commands from a file in the current shell

- Evaluate contents of a given file:
\$ source {{path/to/file}}
- Evaluate contents of a given file
(alternatively replacing source with .):
\$. {{path/to/file}}

3.48 read : retrieving data from stdin

Read a line from the standard input and split it into fields.

```
$ read a b c d
aaa bbb ccc ddd
$ echo $a
aaa
$ echo $b
bbb
$ echo $c
ccc
$ echo $d
ddd
$ bash # if parameters are not exported ,
```

```

# the child process does not recognize them
$ echo "x=$x, a=$a, b=$b, c=$c"
  x=456, a=, b=, c=
$ exit
  exit
$ echo "x=$x, a=$a, b=$b, c=$c"
  x=456, a=aaa, b=bbb, c=ccc
$

```

Shell builtin for retrieving data from stdin

```

$ type read
  read is a shell builtin
$ help read  # man read is something else
  -a array assign the words read to sequential indices of the array
  -p prompt output the string PROMPT without a trailing newline before
attempting to read
  -s silent (without output to terminal, to enter passwords)

```

Example:

```

$ read -p "What's your name? " name
$ echo "Hello $name"

```

If we insert the input at the same line, the prompt does not appear:

ex_04.sh:

```

-----
$ read -p "What's your name? " name
$ echo "Hello $name"
-----
$ echo shmuel | bash ex04.sh
  Hello shmuel  # without prompt

```

3.49 gnome-terminal : open new linux terminal emulator

gnome-terminal is a terminal emulator application for accessing a UNIX shell environment which can be used to run programs available on your system.

```
$ gnome-terminal . & # may be . is redundant
```

3.50 arguments..of..process

```

$# - number of arguments
$0 - name of script file
$1 - the first argument
$2 - the second argument
$* is full list of arguments
$@ is full list of arguments

```

ex05.sh

```

-----
echo "this program was run with $0"  # $0 is name of script file

```

```

echo "received $# arguments."          # $# is number of arguments
echo "first arguemnt is [$1]"          # $1 is the first argument
echo "second argument is [$2]"        # S2 is the second argument
echo "third argument is [$3]"

echo "full list: $*"                  # $* is full list of arguments
echo "full list: $@"                  # $@ is full list of arguments

```

```

-----
$ bash ex05.sh aaaa bbbb ccccc
    this program was run with ex05.sh
    received 3 arguments.
    first arguemnt is [aaaa]
    second argument is [bbbb]
    third argument is [cccc]
    full list: aaaa bbbb ccccc

```

Note : \$# does not take into account the argument 0 (the command itself) !
it counts arguments without zero argument !!!

```

$ bash ex05.sh "aaaa bbbb ccccc" # now it treated as single argument !!!
    this program was run with ex05.sh
    received 1 arguments.
    first arguemnt is [aaaa bbbb ccccc]
    second argument is []
    third argument is []
    full list: aaaa bbbb ccccc
    full list: aaaa bbbb ccccc

```

3.51 clear :Equivalent to CTRL+L : clears the screen of the terminal.

Clears the screen of the terminal.

```

- Clear the screen (equivalent to pressing Control-L in Bash shell):
$ clear

```

3.52 process..output..to..stderr

Trick to print ouput to stderr from process:

```

-----
ex07.sh:
    echo "normal output"
    echo "error output" >&2
-----
$ bash ex07* >/dev/null
$ bash ex07* 2>/dev/null

```

3.53 exit : exit the process (script) , exit the shell

Example with exit the process:

```
-----  
echo "this program will fail"  
exit 1 # exit with status code other than 0 is: error  
# Note:  
#     exit  
# is the same as:  
#     exit 0  
-----  
$ bash ex06.sh  
    this program will fail  
$ echo $?  
1
```

Note : exit status is limited to 255

Example with exit the bash:

```
$ whoami  
amits  
$ su david  
password:  
$ whoami  
david  
$ exit # exit from david  
$ whoami  
amits
```

3.54 if : if command

```
if expression1; then # if expression1 return status of success (exit status 0)  
    # then statement1 will be executed.  
    statement1  
elif expression2 # elif (!), not elsif  
    then statement2  
else  
    statement3  
fi
```

Note : elif can appear several times
if and else can appear only once.

Examples (also see test command):

```
-----  
if ! grep $USER /etc/passwd  
    then echo "your user account is not managed locally";
```



```

fi

grep $USER /etc/passwd
if [ $? -ne 0 ] ; then
    echo "not a local account" ;
fi
-----
leap year example:

#! /usr/bin/bash

if [ -z "$1" ]; then
    echo "Error: no arguments" >&2
    exit 2
fi

if (( $# != 1 )); then
    echo "Error: must have exactly 1 argument for year" >&2
    exit 2
fi

if (( ($1%4)==0 )); then
    if (( ($1%100)==0 )); then
        if (( ($1%400)==0 )); then
            echo "$1 % 4 == 0, $1 % 100 == 0 and $1 % 400 == 0 "
            echo "leap year"
            exit 0
        else
            echo "$1%4==0 and $1%100==0, but not dividable by 400";
            echo "common year"
            exit 1
        fi
    else
        echo "$1%4==0 && $1%100 != 0";
        echo "it is leap year"
        exit 0
    fi
else
    echo "$1 % 4 != 0";
    echo "it is common year"
    exit 1
fi
-----
$ bash leap_year_check_upd.sh 2000
$ echo $?
0
$ bash leap_year_check_upd.sh 2002
$ echo $?
1
$ bash leap_year_check_upd.sh 2002 2000

```

Error: must have exactly 1 argument for year

```
$ if bash leap_year_check.sh 2020;then echo YES;else echo NO;fi
2020%4==0 && 2020%100 != 0
it is leap year
YES
```

3.55 test..[...] : command that can be used as test in if command

Evaluate conditional expression.

Exits with a status of 0 (true) or 1 (false) depending on the evaluation of EXPR. Expressions may be unary or binary. Unary expressions are often used to examine the status of a file. There are string operators and numeric comparison operators as well. The behavior of test depends on the number of arguments.

File operators:

-a FILE	True if file exists.
-b FILE	True if file is block special.
-c FILE	True if file is character special.
-d FILE	True if file exists and is a directory.
-e FILE	True if file exists.
-f FILE	True if file exists and is a regular file.
# Note: regular file: not dir and not soft link	
-g FILE	True if file is set-group-id.
-h FILE	True if file is a symbolic link.
-L FILE	True if file is a symbolic link.
-k FILE	True if file has its 'sticky' bit set.
-p FILE	True if file is a named pipe.
-r FILE	True if file is readable by you.
-s FILE	True if file exists and is not empty.
-S FILE	True if file is a socket.
-t FD	True if FD is opened on a terminal.
-u FILE	True if the file is set-user-id.
-w FILE	True if the file is writable by you.
-x FILE	True if the file is executable by you.
-O FILE	True if the file is effectively owned by you.
-G FILE	True if the file is effectively owned by your group.
-N FILE	True if the file has been modified since it was last read.

FILE1 -nt FILE2 True if file1 is newer than file2 (according to modification date).

FILE1 -ot FILE2 True if file1 is older than file2.

FILE1 -ef FILE2 True if file1 is a hard link to file2.

(same device and inode numbers)

String operators:

-z STRING	True if string is empty.
-n STRING	True if string is not empty.
STRING1 = STRING2	True if the strings are equal.
STRING1 != STRING2	True if the strings are not equal.
STRING1 < STRING2	True if STRING1 sorts before STRING2 lexicographically.
STRING1 > STRING2	True if STRING1 sorts after STRING2 lexicographically.

Other operators:

-o OPTION	True if the shell option OPTION is enabled.
-v VAR	True if the shell variable VAR is set.
-R VAR	True if the shell variable VAR is set and is a name reference.
! EXPR	True if expr is false.
EXPR1 -a EXPR2	True if both expr1 AND expr2 are true.
EXPR1 -o EXPR2	True if either expr1 OR expr2 is true.

arg1 OP arg2 Arithmetic tests. OP is one of -eq, -ne,
 -lt, -le, -gt, or -ge.

\$ which test
 /usr/bin/test

Options to perform test:

test EXPRESSION	
test	# always true
[EXPRESSION]	#] is argument
[]	# always true

Strings comparison :

NOTE : put"" on compared operands when using test command !!!

"="

"<"

">"

"<="

">="

"!="

-o : or

-a : and

! : not

```
$ [ "$password_guess" = 1234 -a "$username_guess" = grisha ]
$ ["$password_guess" = 1234 ] && ["$username_guess" = grisha ]
    # check that both commands succeeded !!!
```

Same options:

```
-----
if [! "$password_guess" != 1234 ] && ["$username_guess" = grisha ]; then
if ! ["$password_guess" != 1234 ] || ! ["$username_guess" = grisha ]; then
if ! ["$password_guess" != 1234 ] && ["$username_guess" = grisha ]; then
if ["$password_guess" = 1234 ] && ["$username_guess" = grisha ]; then
echo "Access granted"
else
echo "Access denied"
exit 1
fi
-----
```

```
$ [ 1 = 1 -o asd = 1234 ] # note backspaces
$ echo $?
0
```

```
$ test "AAA" = "AAA" # note spaces around "="
$1 $2 $3
```

```
$ test "AAA" = "AAA" # Note MUST enter spaces between arguments !!!
$ echo $?
0
$ test "AAA" = "BBB"
$ echo $?
1
$ test "AAA" != "AAA"
$ echo $?
1
$ test "AAA" "<" "BBB"
$ echo $?
0
$ test "AAA" ">" "BBB"
$ echo $?
1
```

Numbers comparison (integer comparison):

```
-lt (less than)
-eq (equal)
-ne (not equal)
```

```
-le (less or equal)
-gt (greater than)
-ge (greater or equal)
```

```
$ test 100 -lt 50 # less than
$ echo $?
1
$ test 100 -eq 50 # equal
$ echo $?
1
$ [ 100 -lt 200 ] && echo TRUE || echo FALSE
TRUE
$ [ 200 -lt 100 ] && echo TRUE || echo FALSE
FALSE
```

```
[                200  -lt  100 ]
$0(command)  $1      $2  $3  $4
```

Note : put " " around arguments within test/[]

Example1 (problematic, enter "1 = 1 -o asd" to):

```
# read password from the user
# the password is 1234
# if the password is correct print
# "Access granted"
# else print
# "Access denied" and exit with failure
read -p 'enter password:' password_guess
if [ $password_guess = 1234 ]; then
echo "Access granted"
else
echo "Access denied"
exit 1
fi
```

Example2 (now it can not be hacked):

```
# read password from the user
# the password is 1234
# if the password is correct print
# "Access granted"
# else print
# "Access denied" and exit with failure
read -p 'enter password:' password_guess
if [ "$password_guess" = 1234 ]; then # now password guess
```

```

# is one single word
# because of " "

echo "Access granted"
else
echo "Access denied"
exit 1
fi

Example 3 :
# read username and password from the user
# the password is 1234
# the username is grisha
# if the credentials are correct print
# "Access granted"
# else print
# "Access denied" and exit with failure
read -p 'enter username:' username_guess
read -p 'enter password:' password_guess

if [ "$password_guess" = 1234 \
    -a "$username_guess" = grisha ]; then
echo "Access granted"
else
echo "Access denied"
exit 1
fi

-----
$ [ ] # without any text always 1 !
$ echo $?
1
$ [ anyText ] # with any text always 0
$ echo $?
0

```

3.56 for : loop command

<https://www.cyberciti.biz/faq/bash-for-loop/>

For does not work as classic prog. languages.
but using (()) we can do the same
as in C:

Three-expression bash for loops syntax

This type of for loop share a common heritage with the C programming language. It is characterized by a three-parameter loop control expression; consisting of an initializer (EXP1), a loop-test or condition (EXP2), and a counting expression/step (EXP3).

```

for (( EXP1; EXP2; EXP3 ))
    ## The C-style Bash for loop ##
    for (( initializer; condition; step ))
    do
        shell_COMMANDS
    done

```

A representative three-expression example in bash as follows:

```

#!/bin/bash
# set counter 'c' to 1 and condition
# c is less than or equal to 5
for (( c=1; c<=5; c++ ))
do
    echo "Welcome $c times"
done

```

How do I use for as infinite loops? ↑

Infinite for loop can be created with empty expressions, such as:

```

#!/bin/bash
for (( ; ; ))
do
    echo "infinite loops [ hit CTRL+C to stop]"
done

```

Conditional exit with break

You can do early exit with break statement inside the for loop. You can exit from within a FOR, WHILE or UNTIL loop using break. General break statement inside the for loop:

```

for I in 1 2 3 4 5
do
    statements1      #Executed for all values of
                    #'I'', up to a disaster-condition
                    # if any.

    statements2
    if (disaster-condition)
    then
        break      #Abandon the loop.
    fi
    statements3      #While good and, no disaster-condition.
done

```

Early continuation with continue statement

To resume the next iteration of the enclosing FOR, WHILE or UNTIL loop use continue statement.

```

for I in 1 2 3 4 5
do
    statements1      #Executed for all values of
                    # ''I'', up to a disaster-condition if any.

```

```

statements2
if (condition)
    then
        continue    #Go to next iteration of I
                    #in the loop and skip statements3
    fi
statements3
done

```

```

-----
*           for loop syntax           *
-----

```

Numeric ranges for syntax is as follows:

```

    for VARIABLE in 1 2 3 4 5 .. N
    do
command1
command2
commandN
    done

```

OR

```

    for VARIABLE in file1 file2 file3
    do
command1 on $VARIABLE
command2
commandN
    done

```

OR

```

    for OUTPUT in $(Linux-Or-Unix-Command-Here)
    do
command1 on $OUTPUT
command2 on $OUTPUT
commandN
    done

```

```

-----
#!/bin/bash
for i in 1 2 3 4 5
do
    echo "Welcome $i times"
done

```

```

-----
#!/bin/bash
for i in {1..5}
do
    echo "Welcome $i times"
done

```



```
-----  
#!/bin/bash  
echo "Bash version ${BASH_VERSION}..."  
for i in {0..10..2}  
do  
    echo "Welcome $i times"  
done  
-----
```

Loop with strings (helps to divide string to separate words)

```
#!/bin/bash  
#Say we have a variable, and we need to loop through  
#a list of strings to install those packages:  
TRY_VAR="Mor Rotem Amit"  
for p in $TRY_VAR  
do  
    echo "$p is ready"  
done
```

Result:

```
$ bash try_for.sh  
    Mor is ready  
    Rotem is ready  
    Amit is ready
```

```
-----  
#!/bin/bash  
## $@ expands to the positional parameters, starting from one.  
for i in $@  
do  
    echo "Script arg is $i"  
done
```

Result:

```
$ bash try_for_for_args.sh well bad good  
    Script arg is well  
    Script arg is bad  
    Script arg is good
```

```
-----  
for var in $(command)  
do  
    print "$var"  
done
```

example

Show the name of every PDF file in the /nas directory:

```
for f in $(ls ./try_dir)  
do  
    printf "File is %s\n" "$f"  
done
```

```
Result:
$ bash for_with_ls_command.sh
File is 1.txt
File is 2.txt
File is my_hostname.txt
File is my_name.txt
```

3.57 case : case command

1. Example of case :

```
#!/bin/bash
if [ $# -lt 2 ];then
    echo "Usage : $0 Signalnumber PID"
    exit
fi

case "$1" in
    1) echo "Sending SIGHUP signal"
        kill -SIGHUP $2
        ;;
    2) echo "Sending SIGINT signal"
        kill -SIGINT $2
        ;;
    9) echo "Sending SIGKILL signal"
        kill -SIGKILL $2
        ;;
    *) echo "Signal number $1 is not processed"
        ;;
esac
```

2. Example of case :

```
#!/ env bash
while true; do
    echo 'Drinks Menu:
1) Water
2) Coffee
3) Cola
4) Tea
0) Exit
'
    read -p 'enter drink number:' drink_number

    case "$drink_number" in
        1|water)
            echo "you got water"
            ;;
        2|coffee|java)
            ;;
    esac
```

```

        echo "you got coffee"
        ;;
    3|cola)
        echo "you got cola"
        ;;
    4|tea)
        echo "you got tea"
        ;;
    # 0 or q or exit or quit or anything
    # that starts with "x"
    0|q|"exit"|quit|x*)
        echo "goodbye"
        break
    ;;
    *)
echo "ERROR: invalid drink number"
;;
    esac
done
# if--fi case-esac while-done for-done

```

3.58 while : while command

```

#!/usr/bin/bash
read -p "Enter base value: " base_value
read -p "Enter power value: " power_value
num_of_iterations=1
result=1;

while [ $power_value -ge $num_of_iterations ]; do
    echo "num_of_iterations = $num_of_iterations"
    num_of_iterations=$((num_of_iterations+1))
    result=$((result*base_value))
done

echo "result = $result"
exit

```

4 Processes/Jobs Control

4.1 process..input..output

How process gets information:

1. arguments
2. read file (side effect)

3. signals (KILL)
4. env. variables
5. stdin (0)

How process outputs information:

-
1. stdout(1)
 2. stderr(2)
 3. write file (side effect)
 4. exit code (0 - well, others - error)

4.2 arguments..of..process

\$# - number of arguments
 S0 - name of script file
 \$1 - the first argument
 \$2 - the second argument
 \$* is full list of arguments
 @\$ is full list of arguments

 ex05.sh

```
echo "this program was run with $0" # S0 is name of script file
echo "received $# arguments."      # $# is number of arguments
echo "first arguemnt is [$1]"      # $1 is the first argument
echo "second argument is [$2]"     # S2 is the second argument
echo "third argument is [$3]"

echo "full list: $*"               # $* is full list of arguments
echo "full list: @$"               # @$ is full list of arguments
```

```
$ bash ex05.sh aaaa bbbb ccccc
  this program was run with ex05.sh
  received 3 arguments.
  first arguemnt is [aaaa]
  second argument is [bbbb]
  third argument is [cccc]
  full list: aaaa bbbb ccccc
```

Note : \$# does not take into account the argument 0 (the command itself) !
 it counts arguments without zero argument !!!

```
$ bash ex05.sh "aaaa bbbb ccccc" # now it treated as single argument !!!
  this program was run with ex05.sh
  received 1 arguments.
  first arguemnt is [aaaa bbbb ccccc]
  second argument is []
```

```
third argument is []
full list: aaaa bbbb ccccc
full list: aaaa bbbb ccccc
```

4.3 Ctrl+Z : SIGTSTP - an inTeractive SToP signal

<https://medium.com/@prateeksrivastav598/understanding-ctrl-z-and-ctrl-c-in-the-linux-terminal-c28bd1e00dbe>

https://www.gnu.org/software/libc/manual/html_node/Job-Control-Signals.html

Unlike SIGSTOP, this signal can be handled and ignored. Your program should handle this signal if you have a special need to leave files or system tables in a secure state when a process is stopped. For example, programs that turn off echoing should handle SIGTSTP so they can turn echoing back on before stopping.

Ctrl + Z: Suspending a Process

Ctrl + Z is used to suspend a running process in the terminal. This means that you temporarily :

1. pause the execution of a program and
2. move it to the background, allowing you to continue using the terminal.

It's useful when you want to halt a process without terminating it. Here's how to use it:

Example 1: Pausing a Long-Running Process

Let's say you're compressing a large file using the gzip command, and it's taking longer than expected. You can press Ctrl + Z to pause the compression process and return to the terminal prompt. This doesn't terminate the gzip command; it simply stops it temporarily.

```
$ gzip -9 largefile.txt
```

```
[Press Ctrl + Z]
```

```
[1]+  Stopped          gzip -9 largefile.txt
```

We should note that even though the shell says the job is Stopped, it is, in fact, paused - not terminated - and we'll be able to resume it. To resume the operation, we can use the fg command. So, the gzip process is paused and can be resumed later using the fg (foreground) command:

```
$ fg
```

Example 2: Running a Process in the Background.

You can also use Ctrl + Z to start a command in the background.
For instance, you can run a command like this:

```
$ my_long_running_command &
```

However, if you forget to use the ampersand (&) to run it in the background, you can press Ctrl + Z to suspend it and then type bg (background) to resume its execution in the background.

4.4 Ctrl+C : SIGINT <Signal INTerrupt> interrupting a process

<https://medium.com/@prateeksrivastav598/understanding-ctrl-z-and-ctrl-c-in-the-linux-terminal-c28bd1e00dbe>

SIGINT(^C) - The gentle nudge

SIGINT(^C), or <Signal INTerrupt>, is perhaps the most commonly encountered signal for many users. This signal is typically associated with the CTRL+C command you often use in your terminal to stop a running process. The primary purpose of SIGINT is to notify a process that the user has requested an interruption.

Example of SIGINT:

```
$ kill -INT <process_id> # SIGINT (interrupt) signal
or
$ kill -2 <process_id>
```

Example :

run on first terminal window:

```
$ sleep 100
```

run on second terminal window:

```
$ ps aux | grep sleep
```

```
root  9416  0.0  0.0  3212  1792 ?        S   Feb26 0:00 sleep 3600
amits  816  0.0  0.0  8376  1920 pts/0  S+  00:16 0:00 sleep 100
amits 10822 0.0  0.0  9216  2432 pts/1  S+  00:16 0:00 grep --color=auto sleep
$ kill -2 10816 # this will kill the process
# 10816 on first terminal window.
```

Ctrl + C: Interrupting a Process

Ctrl + C is used to forcefully terminate a running process.

When you press Ctrl + C, the terminal sends an interrupt signal (SIGINT) to the process, causing it to stop immediately.

It's a quick way to exit a process or command.

Example: Stopping an Unresponsive Program

Imagine you're running a program that becomes unresponsive or is stuck in an infinite loop.

You can use Ctrl + C to interrupt the program and return to the terminal prompt.

```
$ ./my_program
[Press Ctrl + C]
^C
The ^C represents the interrupt signal.
```

4.5 Ctrl+D : End of File / exit shell

Note1: ^D is not signal, it is simply End Of File.

^D closes Bash

Sometimes when we try to exit the shell by pressing Ctrl+D or by using logout, we may receive an error message:

```
$ logout
```

There are stopped jobs. # message when there are paused jobs

When bash shows this message, it also prevents logout.

This is because bash doesn't allow us to exit the shell while there are paused jobs.

The current shell's process manages its jobs.

When we pause a job, it's left in an incomplete state. If we exit the shell with jobs paused, we might lose some critical data.

So, we need to take care of these paused jobs before we can exit the shell.

4.6 ps : "Process Status" - snapshot of the current processes

see https://sites.ualberta.ca/dept/chemeng/AIX-43/share/man/info/C/a_doc_lib/cmds/aixcmds4/ps.htm

The ps program (short for "process status") displays info on the currently-running processes. A related Unix utility named "top" provides a real-time view of the running processes.

This version of ps accepts 3 kinds of options:

- 1 UNIX options, which may be grouped and must be preceded by a dash.
- 2 BSD options, which may be grouped and must not be used with a dash.
- 3 GNU long options, which are preceded by two dashes.

Options of different types may be freely mixed, but conflicts can appear. There are some synonymous options, which are functionally identical, due to the many standards and ps implementations that this ps is compatible with.

To see only the processes owned by a specific user on Linux run:

```
$ ps -u {USERNAME}
```

```
$ ps -u | grep bash # present all processes of CURRENT USER !
```

```
amits      6425  0.0  0.0  12056  5760 pts/0    Ss   11:31   0:00 bash
amits      7263  0.0  0.0  11408  5120 pts/1    Ss+  12:27   0:00 /bin/bash
```

```

amits      13283  0.0  0.0  10108  3840 pts/0    T   14:49   0:00 bash ex04
amits      15526  0.0  0.0  10108  3840 pts/0    T   15:06   0:00 bash ex04

```

-a Select all processes except:

1. session leaders (see getsid(2))
2. processes not associated with a terminal.

-u userlist Select by effective user ID (EUID) or name.

This selects the processes whose effective user name or ID is in userlist.

-x is for tty :

Today in Linux, the tty is a legacy name used to refer to the user interface for text-based input and output, otherwise known as a "terminal".

In Linux systems, there can be multiple tty device "consoles", to support potentially dozens of serial ports or more.

tty0 is the current one in use, but Linux allows you to switch to another session by changing to a different tty, e.g., tty1. Linux (e.g., Ubuntu) supports up to 6 ttys by default, but this number is configurable

EXAMPLES

To see every process on the system using standard syntax:

```

$ ps -e      # Select all processes.
$ ps -ef     # Do full-format listing.
$ ps -eF     # Extra full format.
$ ps -ely    # -l : BSD Long format.
               -y : Do not show flags; show rss in place of addr.
               This option can only be used with -l.

```

By running `$ ps l`, you can see which processes are running, and which are sleeping. If a process is sleeping, the WCHAN column ("wait channel", the name of the wait queue) will tell you what kernel event the process is waiting for.

```

$ ps l # display BSD long format.
$ ps -l # Long format. The -y option is often useful with this

```

To see every process on the system using BSD syntax:

```

$ ps ax
$ ps aux

```

Example :

```

$ ps aux | grep bash
amits      5464  0.0  0.0  11408  5120 pts/0    Ss+  13:03   0:00 bash
amits      6966  0.0  0.0  11408  5248 pts/1    Ss   14:32   0:00 bash
amits      6974  0.0  0.0   9216  2432 pts/1    S+   14:32   0:00 grep
                                           --color=auto bash

```

To print a process tree:

```

ps -ejH
ps axjf

```


To get info about threads:

ps -eLf

ps axms

Column	Contents
%CPU	How much of the CPU the process is using
%MEM	How much memory the process is using
ADDR	Memory address of the process
C or CP	CPU usage and scheduling information
COMMAND*	Name of the process, including arguments, if any
NI	Nice value
F	Flags
PID	Process ID number
PPID	ID number of the process's Parent Process
PRI	Priority of the process
RSS	Resident set size (process memory occupied in RAM)
S or STAT	Process status code
START or STIME	Time when the process started
VSZ	Virtual memory usage
TIME	The amount of CPU time used by the process
TT or TTY	Terminal associated with the process
UID or USER	Username of the process's owner
WCHAN	Memory address of the event the process is waiting for

Note 1 : In computing, resident set size (RSS) is the portion of memory (measured in kilobytes) occupied by a process that is held in main memory (RAM). The rest of the occupied memory exists in the swap space or file system, either because some parts of the occupied memory were paged out, or because some parts of the executable were never loaded.

Note 2 : nice is a program found on Unix and Unix-like operating systems such as Linux. It directly maps to a kernel call of the same name. nice is used to invoke a utility or shell script with a particular CPU priority, thus giving the process more or less CPU time than other processes. A niceness of -20 is the lowest niceness, or highest priority.

Note 3 : The SZ field displays the size of the process in memory. The value of the field is the number of pages the process is occupying. On Linux machines, a page is 4,096 bytes.

PROCESS STATE CODES

<https://askubuntu.com/questions/1144036/process-status-of-s-s-s-sl>

<https://www.linusakesson.net/programming/tty/index.php>

Here are the different values that the s, stat and state output specifiers (header "STAT" or "S") will display to describe the state of a process:

D	uninterruptible sleep (waiting for some event)
R	running or runnable (on run queue)
S	interruptible sleep (waiting for some event or signal)
T	stopped by job control signal

t stopped by debugger during the tracing
 Z defunct "zombie" process, terminated but not yet
 reaped by
 its parent

A Linux process can be in one of the following states:

R----->Z
 D<-->R S----->Z
 R<--->S--->T-->Z
 R<----->T

For BSD formats and when the stat keyword is used, ADDITIONAL
 CHARACTERS may be displayed:

< high-priority (not nice to other users)
 N low-priority (nice to other users)
 L has pages locked into memory (for real-time and custom IO)
 s This process is a session leader
 l This process is multi-threaded (using CLONE_THREAD,
 like NPTL pthreads do)
 + This process is in the foreground process group

Job control is what happens when you press ^Z to suspend a program,
 or when you start a program in the background using &.

A job is the same as a process group.

Internal shell commands like jobs, fg and bg can be used to manipulate
 the existing jobs within a session.

Each session is managed by a session leader, the shell, which is
 cooperating tightly with the kernel using a complex protocol of signals
 and system calls.

 The default niceness for processes is inherited from its parent process
 and is usually 0.

Users can pipeline ps with other commands,
 such as less to view the process status output one
 page at a time:

```
$ ps -A | less
```

Users can also utilize the ps command in conjunction with
 the grep command (see the pgrep and pkill commands) to find
 information about a single process, such as its id:

```
$ # Trying to find the PID of 'firefox-bin' which is 2701
$ ps -A | grep firefox-bin
2701 ?          22:16:04 firefox-bin
```

Custom columns choose :

```
$ ps a -o user -o comm -o pid -o nice
```

USER	COMMAND	PID	NI
amits	gdm-wayland-ses	2342	0
amits	gnome-session-b	2345	0
root	agetty	3891	0
root	agetty	3931	0
amits	tex_file_data_e	14438	0
amits	htop	30232	0
root	sudo	30469	0
root	sudo	30471	0
root	geany	30472	-20
root	bash	30520	-20
root	dbus-launch	30529	-20

AIX FORMAT DESCRIPTORS

This ps supports AIX format descriptors, which work somewhat like the formatting codes of printf(1) and printf(3). For example, the normal default output can be produced with this: `ps -eo "%p %y %x %c"`. The NORMAL codes are described in the next section.

CODE	NORMAL	HEADER
%C	pcpu	%CPU
%G	group	GROUP
%P	ppid	PPID
%U	user	USER
%a	args	COMMAND
%c	comm	COMMAND
%g	rgroup	RGROUP
%n	nice	NI
%p	pid	PID
%r	pgid	PGID
%t	etime	ELAPSED
%u	ruser	RUSER
%x	time	TIME
%y	tty	TTY
%z	vsz	VSZ

Example:

```
$ ps a -o user -o comm -o pid -o nice
it is the same as :
$ ps a -o "%U %c %p %n"
```

The use of pgrep simplifies the syntax and avoids potential race conditions:

4.7 pstree : display a tree of processes

<https://www.geeksforgeeks.org/pstree-command-in-linux-with-examples/>

```
pstree [options] [pid or username]
```

A convenient tool to show running processes as a tree

The pstree command is a handy little utility that shows the processes currently running on your system and it show them in a tree diagram.

To display process tree:

```
$ pstree
```

To include command line arguments in output

```
$ pstree -a
```

To display PIDs

```
$ pstree -p
```

To force pstree to expand identical subtrees in output.

```
$ pstree -c
```

To sort processes

```
$ pstree -n
```

To see who is the owner/user of a process

```
$ pstree -u
```

To highlight the current process or any other process

```
$ pstree -h
```

To show process group IDs in output

```
$ pstree -g
```

To make pstree display process tree specific to a user.

```
$ pstree amits
```

4.8 pgrep : ps+grep

Find or signal processes by name

Search for a Linux process by name run:

```
$ pgrep -u {USERNAME} {processName}
```

Example :

```
$ pgrep -u amits sleep # only for "amits" user  
19267
```

```
$ pgrep sleep # all users
```

18588

19267

Return only PID number , pgrep = ps + grep :

```
$ pgrep ping = ps aux | grep -w [p]ing:
```

4.9 top : "Table Of Processes"

To show all processes for a specific user using top/htop

```
$ top -U amits
```

is a task manager or system monitor program,
found in many Unix-like operating systems,
that displays information about CPU and memory utilization

The program produces an ordered list of running processes
selected by user-specified criteria,
and updates it periodically.
Default ordering is by CPU usage,
and only the top CPU consumers are shown.
top shows how much processing power and memory are being used,
as well as other information about the running processes.
Some versions of top allow extensive customization of the display,
such as choice of columns or sorting method.
top is useful for system administrators, as it shows which
users and processes are consuming the most system
resources at any given time.

top view can be listed with help of arrows.

To get help :

```
$ top
```

```
~Z          # suspend and move the process to bg
```

```
$ man top
```

```
$ fg        # move the process to fg
```

4.10 htop : similar as top, but more user friendly

More user friendly than top,

The title is always on the top of the table.

Press F4 to insert filter word,

for example, "amits" word in filter will cause to present user "amits"
"bash" word in filter will present processes of bash.

Press F5 to see process tree !!!
Press F5 again to present process list.

NI: niceness in htop window.
The higher the NI value, the lower the priority-
the "nicer" the process is to other processes.

4.11 nice : run a program with a custom priority (niceness)

Niceness values range from -20 (highest priority - most favorable to the process) to 19 (lowest priority - least favorable to the process).

- The nice value determines the priority of a process, with lower values indicating higher priority.

NOTE: your shell may have its own version of nice, which usually supersedes the version described here.

Please refer to your shell's documentation for details about the options it supports.

```
$ type nice
nice is /usr/bin/nice # not built in
```

```
nice [OPTION] [COMMAND [ARG]...]
```

```
- Launch a program with altered priority:
$ nice -n {{niceness_value}} {{command}}
Example:
$ sudo nice -n -20 geany
[sudo] password for amits: ****
```

<https://www.scaler.com/topics/linux-nice/>

- The Nice command in linux allows users to prioritize process execution.
- The nice value determines the priority of a process, with lower values indicating higher priority.
- To check the nice value of a running process, use the `ps -l <process_id>` command.
- execute a process with given nice value :
`$ nice -n <nice_value> <command>` syntax.

```
$ sudo nice -n -20 geany # this caused to
[sudo] password for amits:
$ ps a -o user -o comm -o pid -o nice
USER      COMMAND                PID  NI
amits     gdm-wayland-ses        2342  0
```

amits	gnome-session-b	2345	0
root	agetty	3891	0
root	agetty	3931	0
root	agetty	3937	0
root	login	4195	0
amits	bash	4294	0
amits	bash	5964	0
amits	bash	6101	0
amits	tex_file_data_e	14438	0
amits	htop	30232	0
root	sudo	30469	0
root	sudo	30471	0
root	geany	30472	-20
root	bash	30520	-20
root	dbus-launch	30529	-20

4.12 renice : alters priority of already running process

The Renice command can be used to alter a process's priority while it is already running. Open a terminal and execute the following command:

```
$ renice <nice_value> -p <process_id>
```

Replace <nice_value> with the new priority level you want to assign

and <process_id> with the ID of the running process you wish to modify.

For example, to increase the priority of a process with ID 1234 to 5, you would run:

```
$ renice 5 -p 1234
```

Niceness values range from -20 (most favorable to the process) to 19 (least favorable to the process)

- Change priority of a running process:

```
$ renice -n {{niceness_value}} -p {{pid}}
```

- Change priority of all processes owned by a user:

```
$ renice -n {{niceness_value}} -u {{user}}
```

- Change priority of all processes that belong to a process group:

```
$ renice -n {{niceness_value}} --pgrp {{process_group}}
```

4.13 eval : combines arguments into a single string and runs

<https://www.geeksforgeeks.org/eval-command-in-linux-with-examples/>

<https://www.educative.io/answers/what-is-the-eval-command-in-linux>

eval runs string as a bash command

eval is a built-in Linux command which is used to execute arguments as a shell command.
It combines arguments into a single string and uses it as an input to the shell and execute the commands.

When you have Linux command saved in a variable and wish to run that command, the eval command is useful.

Example:
\$ COMM="ls"
\$ eval \$COMM

Example:
\$ eval echo hello

4.14 jobs : Execute arguments as a single command in shell

Display status of jobs in background in the current session

The jobs command marks the last paused job with the + sign and the immediate previous with the - sign. If we use fg and other job commands without a job number, the last is implied.

- Show status of all jobs:
jobs
- Show status of a particular job:
jobs %{{job_id}}
- Show status and process IDs of all jobs:
jobs -l
- Show process IDs only of all jobs:
jobs -p

Example :

```
$ man ed
^Z # ctrl+Z by user
[1]+  Stopped                  man ed
$ man cp
^Z # ctrl+Z by user
[2]+  Stopped                  man cp
$ nano 2.txt
^T^Z # ctrl T + ctrl+Z
[3]+  Stopped                  nano 2.txt
$ man pwd
^Z
[4]+  Stopped                  man pwd
$ jobs
[1]   Stopped                  man ed      # "stopped" means paused !!!
```



```

[2]   Stopped                  man cp
[3]-  Stopped                  nano 2.txt  # job before the last
[4]+  Stopped                  man pwd    # last job
$ jobs -l    # including job number
[1]   6065 Stopped              man ed
[2]   6084 Stopped              man cp
[3]-  6120 Stopped (signal)     nano 2.txt
[4]+  6123 Stopped              man pwd
$ fg %1
man ed

```

4.15 kill : send a signal to a process

NAME

kill - send a signal to a process

SYNOPSIS

```
kill [options] <pid> [...]
```

DESCRIPTION

The default signal for kill is TERM.

Use -l or -L to list available signals.

Particularly useful signals include HUP, INT, KILL, STOP, CONT, and 0.

Alternate signals may be specified in three ways: -9, -SIGKILL or -KILL.

Negative PID values may be used to choose whole process groups; see the PGID column in ps command output.

A PID of -1 is special;

it indicates all processes except the kill process itself and init.

kill sends a signal to a process, usually related to stopping the process.

All signals except for SIGKILL/KILL and SIGSTOP/STOP can be intercepted by the process to perform a clean exit.

To present signals list (to be used without the SIG prefix, first 15 signals are mandatory):

```
$ kill -l
```

1) SIGHUP	2) SIGINT (1, ^C)	3) SIGQUIT (^\\)	4) SIGILL	5) SIGTRAP

6) SIGABRT	7) SIGBUS	8) SIGFPE	9) SIGKILL(3)	10) SIGUSR1

11) SIGSEGV	12) SIGUSR2	13) SIGPIPE	14) SIGALRM	15) SIGTERM (2)

16) SIGSTKFLT	17) SIGCHLD	18) SIGCONT	19) SIGSTOP	20) SIGTSTP(^Z)

Note1: ^D is not signal, it is simply End Of File.

^D closes Bash

Note2: To respond on kill signal the process need to be run,
if the process receive the ^C signal while it is suspended with ^Z ,
it will finish its work only after resume from suspend with fg command.

SIGINT(^C) - The gentle nudge

SIGINT(^C), or <Signal INTerrupt>, is perhaps the most commonly encountered signal for many users. This signal is typically associated with the CTRL+C command you often use in your terminal to stop a running process. The primary purpose of SIGINT is to notify a process that the user has requested an interruption.

Example of SIGINT:

```
$ kill -INT <process_id> # SIGINT (interrupt) signal
or
$ kill -2 <process_id>
```

Example :

run on first terminal window:

```
$ sleep 100
```

run on second terminal window:

```
$ ps aux | grep sleep
```

```
root  9416  0.0  0.0  3212  1792 ?        S   Feb26 0:00 sleep 3600
amits  816  0.0  0.0  8376  1920 pts/0 S+  00:16 0:00 sleep 100
amits 10822 0.0  0.0  9216  2432 pts/1 S+  00:16 0:00 grep --color=auto sleep
$ kill -2 10816 # this will kill the process
# 10816 on first terminal window.
```

SIGTERM - The polite request

see: <https://linuxhandbook.com/sigterm-vs-sigkill/>

The SIGTERM (SIGnal and TERMinate) signal is used for terminating a program. SIGTERM is more forceful than SIGINT but still gives a process the opportunity to perform clean-up tasks before it ends. It allows the process to catch the signal and manage its termination elegantly - saving data or finishing essential tasks. The SIGTERM can also be referred as a soft kill because the process that receives the SIGTERM signal may choose to ignore it. SIGTERM doesn't kill the child processes, With SIGTERM, a process gets the time to send the information to its parent and child processes. Its child processes are handled by init.

To send SIGTERM:

```
$ kill <process_id> # default SIGTERM (terminate)
or
$ kill -s TERM <process id>
```

SIGKILL - The last resort

see: <https://linuxhandbook.com/sigterm-vs-sigkill/>

Now, what if a process does not respond to the SIGTERM signal, or is stuck in an endless loop and not releasing the resources? This is where SIGKILL comes in. SIGKILL, as the name suggests, kills the process immediately. The system does not give the process any chance to clean up or free up resources

SIGKILL (the last option) is used for immediate termination of a process. This signal cannot be ignored or blocked !

The process will be terminated along with its threads (if any). It is a brutal way of killing a process, and it should only be used as a last resort.

Suppose you have an unresponsive process that you want to close.

The SIGKILL can be used in such a case.

SIGKILL kills the child processes as well.

Use of SIGKILL may lead to the creation of a zombie process because the killed process doesn't get the chance to tell its parent process that it has received a kill signal.

```
$ kill -9 <process id>
```

or

```
$ kill -KILL <process id>
```

Terminate a background job:

```
$ kill %{{job_id}}
```

see https://www.gnu.org/software/libc/manual/html_node/Standard-Signals.html
https://www.gnu.org/software/libc/manual/html_node/Termination-Signals.html
https://www.gnu.org/software/libc/manual/html_node/Job-Control-Signals.html

INT is for ^C

QUIT is for ^/ , it is similar to ^C but performs the core dump.

TERM is a generic signal used to program termination. Unlike

KILL, this signal can be blocked, handled, and ignored.

KILL is for immediate process termination. It cannot be handled or ignored, and is therefore always fatal. It is also not possible to block this signal.

TSTP is ^Z signal The SIGTSTP signal is an interactive stop signal.

Unlike STOP, this signal can be handled and ignored

STOP is The STOP signal stops the process. It cannot be handled, ignored, or blocked.

4.16 pkill : Signal process by name

Signal processes by process's name

see https://www.gnu.org/software/libc/manual/html_node/Standard-Signals.html
https://www.gnu.org/software/libc/manual/html_node/Termination-Signals.html
https://www.gnu.org/software/libc/manual/html_node/Job-Control-Signals.html

INT is for ^C
 QUIT is for ^/ , it is similar to ^C but performs the core dump.
 TERM is a generic signal used to program termination. Unlike
 KILL, this signal can be blocked, handled, and ignored.
 KILL is for immediate process termination. It cannot be
 handled or ignored, and is therefore always fatal.
 It is also not possible to block this signal.
 TSTP is ^Z signal The SIGTSTP signal is an interactive stop signal.
 Unlike STOP, this signal can be handled and ignored
 STOP is The STOP signal stops the process. It cannot be handled,
 ignored, or blocked.

First option:

```
$ pkill -QUIT ping
$ pkill -INT ping    # ^C
```

Second option:

```
$ pkill -TERM ping
```

The Last option:

```
$ pkill -KILL ping
```

To stop process:

```
^Z: $ pkill -TSTP xed
```

- Force kill matching processes (can't be blocked):

```
pkill -9 "{{process_name}}"
```

- Send KILL signal to processes which match:

```
pkill -KILL "{{process_name}}"
```

Example :

on first terminal :

```
$ sleep 500
```

on second terminal

```
$ ps aux | grep sleep
```

```
root      14276  0.0  0.0   3212  1664 ?        S    01:41   0:00 sleep 3600
amits     18341  0.0  0.0   8376  1920 pts/0    S+   02:36   0:00 sleep 500
```

```
$ pkill -KILL sleep
```

```
$ pkill -KILL sleep
```

```
pkill: killing pid 14276 failed: Operation not permitted # because of root !
```

4.17 xkill : Kill a window interactively in a graphical session

- Display a cursor to kill a window when pressing
the left mouse button (press any other mouse button to cancel):
\$ xkill

4.18 bg : resume suspended process to run it in the background.

<https://www.scaler.com/topics/how-to-run-process-in-background-linux/>

- Resume the most recently suspended job and run it in the background:

```
$ bg
```

- Resume a specific job (use jobs -l to get its ID) and run it in the background:

```
$ bg %{{job_id}}
```

How To Run a Process in Background in Linux?

Running processes in the background is a common requirement in Linux systems.

Running a process in the background allows you :

1. to continue using the terminal
2. execute other commands while the process runs independently.

This can be particularly useful for :

1. long-running tasks or
2. when you want to execute multiple commands simultaneously.

What is a Background Processes

Let's first understand what a background process is.

In Linux, a background process refers to a process that runs independently of the terminal.

When you execute a command, it typically runs in the foreground, meaning it occupies the terminal until it completes.

On the other hand, running a process in the background allows you to execute other commands while the process continues to run silently.

How to Use the bg Command in Linux

One way to run a process in the background is by using the bg command.

The bg command is built-in to most Linux distributions and allows you to send a running process to the background. Following steps are involved to run the process in background:

1. Start a process in the foreground by executing a command.

For example, let's say we want to compress a large file using the gzip command:

```
$ gzip largefile.txt
```

2. While the process is running, press Ctrl+Z to pause the process and bring it to the background.

3. Use the bg command to resume the process in the background:

```
$ bg
```

4. The process will now continue running in the background, and you will see its job ID displayed on the terminal.

You can execute other commands while it progresses silently.

However, keep in mind that the process may still produce output, which can appear on the terminal.

4.19 & symbol : start process in the background immediately

& Symbol

Another method to run a process in the background is by appending the ampersand symbol & to the command you execute.

This symbol tells the shell to start the process in the background immediately.

To just start the command in the background at the beginning:
all you need to do is put an ampersand at the end of any Linux command.
This will allow to use terminal along with running the command:

Example:

```
$ gzip largefile.txt &
```

In this example, the gzip command starts running in the background as soon as you execute it.
The process ID (PID) will be displayed on the terminal, allowing you to continue using the shell while the process completes silently.

It's important to note that when using the & symbol, the process may still produce output, which can interfere with the prompt on the terminal.

Note:

To avoid this, it's recommended to redirect the output to a file or to /dev/null, which discards the output. You can do this by appending > filename or > /dev/null to the command, respectively.

4.20 disown : Signal process by process's name

Allow sub-processes to live beyond the shell that they are attached to.

- Disown the current job:

```
$ disown
```

- Disown a specific job:

```
$ disown %{{job_number}}
```

- Disown all jobs:

```
$ disown -a
```

If you close current terminal, the jobs that it run will be also closed, because the jobs are terminal's children.
Terminal itself is the child of graphical system (that presents windows).
If you need to close your terminal, and don't want these commands to stop, then you need to "disown" the job(s).

Disown command will cause the job to be child of graphical system, in place of child of terminal.

If you only have one job in the background, the following command will work:

```
$ disown
```

If you have multiple, then you'll need to specify the job ID.

```
$ disown %1
```

You'll no longer see the job in your jobs table when you execute the jobs command.

Now it's safe to close the terminal and your command will continue running.

```
$ jobs -l
```

You can still keep an eye on your running command by using the ps command.

```
$ ps aux | grep sleep
```

```
linuxco+ 1650 0.0 0.0 8084 524 pts/0 S 12:27 0:00 sleep 10000
```

And if you want to stop the command from running, you can use the kill command and specify the process ID.

```
$ kill 1650
```

Examples :

```
$ geany | disown
```

```
$ geany & disown # send to bg and disown
```

4.21 time : Run the command and print the time measurements

Users of the bash shell need to use an explicit path in order to run the external time command `/bin/time` and not the shell builtin variant `"time"` !!!

```
$ which -a time
/usr/bin/time
/bin/time
$ time sleep 2
real 0m2.003s
user 0m0.002s
sys 0m0.000s
$ /bin/time --format="%E real, %U user, %S sys" sleep 2
0:02.00 real, 0.00 user, 0.00 sys
```

On system where

time is installed in `/usr/bin`, the first example would become `/usr/bin/time wc /etc/hosts`

`"time"` runs the program `"COMMAND"` with any given arguments `"ARG"....`

When `COMMAND` finishes, `time` displays information about resources used by `COMMAND` (on the standard error output, by default).

If `COMMAND` exits with non-zero status, `time` displays a warning message and the exit status.

Run the command and print the time measurements to stdout:

```
$ time <command> <args>
```

4.22 fg : resume a specific suspended job by its job number.

The `fg` command allows us to resume a specific suspended job by its job number. The job's name is output when it starts:

```
Resume last job:
$ fg
Resume job number 1:
$ fg %1
```

4.23 \$? : result of last command execution

```
$ ls -la
total 772
drwxrwxr-x 2 amits amits 4096 Feb 28 12:12 .
drwxrwxr-x 8 amits amits 4096 Feb 28 11:42 ..
-rw-rw-r-- 1 amits amits 256176 Feb 28 12:11 try.pdf
-rw-rw-r-- 1 amits amits 148037 Feb 28 11:41 try.tex
$ echo $?
0
https://www.baeldung.com/linux/check-if-command-executed-successfully#:~:text=We%20can%20use%20a%20binary%20comparison%20operator%20eq,the%20pwd%20command%20displays%20the%20present%20working%20directory.
```

We can use a binary comparison operator `eq` to check the outcome of the previous command, stored in the `$?` variable:

```
#!/bin/bash
pwd
if [ $? -eq 0 ]; then
    echo "Command Executed Successfully"
else
    echo "Command Failed"
fi{verbatim}
```

5 Commands envistigation

5.1 type : Display the type of command the shell will execute.

The 'type' is a built-in shell command used to identify the kind of command. For example it checks if it is shell built in or external command.

```
type without any key :
    if NAME is an alias,
    shell reserved word,
    shell function,
    shell builtin,
    disk file
    or not found, respectively
```


-t output a single word which is one of
 'alias',
 'keyword',
 'function',
 'builtin',
 'file'

-p Returns the disk file that would be executed
-a Returns all locations containing an executable.

For example:

```
$ type echo
echo is a shell builtin
$ type cat
cat is /usr/bin/cat
$ type code
code is /usr/bin/code
$ type -a time
time is a shell keyword
time is /usr/bin/time
time is /bin/time
```

5.2 which : Locate a program in the user's path.

Locate a program in the user's path.

- Search the PATH environment variable and display the location of any matching executables:

```
$ which {{executable}}
```

- If there are multiple executables which match, display all:

```
$ which -a {{executable}}
```

Example :

```
$ which -a time
/usr/bin/time
/bin/time
```

5.3 whereis : locate the source, and manual page for a command.

```
$ type whereis
whereis is /usr/bin/whereis
```

5.4 command : locate the source, and manual page for a command.

5.5 history : command-line history

```
history    : Display the history list with line numbers.
history -c : delete all of the entries
history N  : Only list the last N commands.
history 10 : list last 10 entries

-----
!!          : to execute the last command you typed.
!n          : to execute command number n.
!-n         : execute the command n times before
!!==!-1    : equivalent commands
!"string"   : execute the last command starting with
              that "string" (last command with given name)
-----

CTRL-R      : to perform a "reverse-i-search". For
              example, if you wanted to use the command
              you used the last time you used snort,
              you would type: CTRL-R then type "snort".
```

An argument of N lists only the last N entries.

5.6 man : display manual pages

The table below shows the section numbers of the manual followed by the types of pages they contain.

- 1 General commands
- 2 System calls
- 3 Library functions, covering in particular
the C standard library
- 4 Special files (usually devices, those found in /dev) and drivers
- 5 File formats and conventions
- 6 Games and screensavers
- 7 Miscellaneous
- 8 System administration commands and daemons

When you invoke the man command, you can optionally specify the section of the manual to read :

```
$ man 3 printf
```

To see all the man pages available in Ubuntu use

```
$ man -k .
```

-f : to find page that contains the command :

```
$ man -f signal
    signal (7)          - overview of signals
    signal (2)          - ANSI C signal handling
$ man -f print
    print (1)           - execute programs via entries
                        in the mailcap file
```

```
-k : to find command and its page that contains the word:
$ man -k pgrep
    pgrep (1)          - look up, signal, or wait for processes
                        based on name a...
$ zipgrep (1)         - search files in a ZIP archive for lines
                        matching a pat...
```

man is an interface to the system reference manuals

```
$ man -k : the same as apropos
$ man -f : the same as whatis
$ -h - help on interactive commands
```

find words in man manual:

```
to find "name" option in man manual choose :
    / "name"
to find next search result press :
    press "/" and press "enter"
```

5.7 tldr : "Too Long; Didn't Read" - simplified man pages

simplified man pages

To update :

```
$ tldr -update
```

To install:

```
$ sudo apt install tldr
```

Update in Ubuntu (there is a problem to update tldr,
since tldr -u did not work):

```
$ tldr --version
0.6.4 # this is the old version
$ apt remove tldr
$ pip install tldr
$ tldr --version
tldr 3.2.0 (Client Specification 1.5)
```

see: <https://forums.linuxmint.com/viewtopic.php?t=407665>

5.8 whatis : brief command description

whatis is a command for obtaining the brief description
of a specific command whose exact name is already known

5.9 apropos : command to search the man page files

Often a wrapper for the man -k command.

Used to search the "name" sections of all manual pages for the specified string or strings (called keywords).

The output is a list of all manual pages containing the search term (case insensitive) in their name or description.

This is often useful if one knows the action that is desired, but does not remember the exact command or page name.

6 Text and text files manipulations

6.1 pandoc : Convert documents between various formats

Convert documents between various formats:

- List all supported input formats:

```
$ pandoc --list-input-formats
```

- List all supported output formats:

```
$ pandoc --list-output-formats
```

- Convert file to PDF (the output format is determined by file extension):

```
$ pandoc {{input.md}} -o {{output.pdf}}
```

Examples :

```
$ pandoc latex-document-prototype.tex -o try.html
```

```
$ pandoc latex-document-prototype.tex -o try.pdf
```

```
$ pandoc latex-document-prototype.tex -o try.docx
```

6.2 tab : autocomplete

Tab : automatic completion

Tab Tab : all options for complete

6.3 | : pipe - redirect stdout of one cmd to stdin of another cmd.

- “Pipe” redirects standard output of one command to standard input of another command
- t1, t2, tL are “temporary files” in this explanation.
- “|” acts as temporary file.
- Pipe redirects only REGULAR input/output, not error

```
keyboard --> command1 ---> | ---> command2 ---> |--> command3---> display
command1 > t1
command2 <t1 >t2
command3 > tL
```

- Example 1:
Redirect output of (print of all files *.log)

to (grep -i (insensitive) that search lines with “error”)
and sort all this lines:

```
$ cat *log|grep -i error|sort
```

- Example 2:

Redirect (result of recursive insensitive search ,
starting with current directory) to (search of lines that not
include word “ignored”), the (sort the results with “unic”)
and send the result to file serious_errors.log:

```
$ grep -ri error .|grep -v “ignored”|sort u> serious_errors.log
```

- Example 3 (YES command usage):

```
$ cat dir1_rm_inputs.txt
```

```
y  
y  
n  
n  
y  
y  
y
```

```
$ cat dir1_rm_inputs.txt | rm ./dir1/*.* -i
```

```
$ rm -i dir2/*
```

```
rm: remove regular empty file ‘dir2/6.txt’? n  
rm: remove regular empty file ‘dir2/7.txt’? n  
rm: remove regular file ‘dir2/errors2.log’? n  
rm: remove regular file ‘dir2/errors3.log’? n  
rm: remove regular file ‘dir2/errors.log’? n
```

```
$ yes | rm -i dir2/*
```

```
rm: remove regular empty file ‘dir2/6.txt’? rm: remove regular  
empty file ‘dir2/7.txt’? rm: remove regular file ‘dir2/  
errors2.log’? rm: remove regular file ‘dir2/errors3.log’? rm:  
remove regular file ‘dir2/errors.log’?
```

- Example: automation to enter “yes” when install:

```
$ yes | sudo apt install some_programs
```

- apt has -flag to allow install without interrupts.

6.4 Redirections of I/O

Process:

We can control input and output direction of process:

```
0 ---> stdin
1 ---> stdout
2 ---> err
```

- Standard output can be redirected to file using `>`
(remove previous file and creates from the start of file).
- Standard output can be appended to file using `>>`
(appends to the end of existing file).

Redirect input to file:

```
$ cat operations.txt
2+3
4*5
x=7
x*100
```

```
$ bc < operations.txt
5
20
700
```

Redirect output to file :

Redirect result of `ls` command to file `ls_comm_result.txt`:

```
$ ls -l 1.txt 7.txt > ls_comm_result.txt
ls: cannot access '7.txt': No such file or directory
```

Redirect regular output of `ls` command and also ERR output to the same file `ls_comm_result.txt` as regular output:

```
$ ls -l 1.txt 7.txt > ls_comm_result.txt 2>&1
ls: cannot access '7.txt': No such file or directory
```

or:

```
$ ls -l 1.txt 7.txt 1>ls_comm_result.txt 2>&1
ls: cannot access '7.txt': No such file or directory
```

Redirect ERR output to `ls_comm_result.txt` and regular output to the same file as error output:

```
$ ls -l 1.txt 7.txt 2>ls_comm_result.txt 1>&2
# or:
$ ls -l 1.txt 7.txt 2>ls_comm_result.txt >&2
```

In This case regular output is redirected to errors output:

```
$ echo "error" >&2
```

Send `grep` command output to file :

```
$ grep error errors.log > grep_result.txt
```

Append `grep` command output to the end of the file:

```
$ grep error errors.log >> grep_result.txt
```

Append cat command output to file:

```
$ cat 1.txt
this is very interesting.
what ?
this.
$ cat 2.txt
also this is very nice and interesting
exactly
$ cat 2.txt >> 1.txt
$ cat 1.txt
this is very interesting.
what ?
this.
also this is very nice and interesting
exactly
```

Print cat command output to file:

```
$ cat 1.txt > 2.txt
$ cat 1.txt
this is very interesting.
what ?
this.
also this is very nice and interesting
exactly
$ cat 2.txt
this is very interesting.
what ?
this.
also this is very nice and interesting
exactly
```

Print ls command output to file:

```
$ ls
1.txt  bc    dir2    fruits.txt    numbers.txt  sizes.txt
2.txt  dir1  errors.log  grep_result.txt  out1
$ ls > out
$ cat out
1.txt
2.txt
bc
dir1
dir2
errors.log
fruits.txt
grep_result.txt
numbers.txt
out
out1
sizes.txt
```

Use echo command output to prepare file in place of nano:

```
$ echo 'this
> is a file
> that try to use echo command
> in place of nano command'> my_file.txt
$ cat my_file.txt
this
is a file
that try to use echo command
in place of nano command
```

Append line to file with echo command output:

```
$ echo "ups forgot this line" >> my_file.txt
$ cat my_file.txt
this
is a file
that try to use echo command
in place of cat command
ups forgot this line
```

Redirect errors output to additional file errors.txt:

```
$ ls 5.txt z.txt
ls: cannot access 'z.txt': No such file or directory # err output
5.txt # regular output
$ ls 5.txt z.txt >out.txt 2>error.txt # same as 1>out.txt
$ cat out.txt
5.txt
$ cat error.txt
ls: cannot access 'z.txt': No such file or directory
```

6.5 echo : display line of text

Display line of text/string that are passed as an argument.

Mostly used in shell scripts and batch files

to output status text to the screen or a file

Helps to see a command after expansion

```
$ echo hello
hello
```

```
$ echo hello world # 2 different arguments : Hello and World
hello world
```

```
$ echo "hello world" # 1 argument "Hello world"
hello world
```



```
$ echo ~  
/home/amit
```

```
$ echo "~"  
~
```

```
echo ls *.txt # to see the command after expansion
```

```
? replaces exactly one character:
```

```
$ echo ?.txt # ? replace exactly one character  
1.txt 3.txt 5.txt 7.txt
```

```
The list of files with extension of 4 characters :
```

```
$ echo *.????  
2.html 5.html
```

```
$ echo ???.txt  
11.txt 12.txt
```

```
$ echo ????.txt  
111.txt
```

```
$ echo 1?.txt  
11.txt 12.txt
```

```
$ echo 1???.txt  
111.txt
```

```
$ echo [0-9][0-9][0-9].txt  
111.txt
```

```
^ is for "not" :
```

```
$ echo [^0-9].txt # not(numbers 0 to 9)  
a.txt b.txt g.txt i.txt
```

```
$ echo [a-z].txt  
a.txt b.txt g.txt i.txt
```

```
$ echo [1a2x3456bc].txt  
1.txt 3.txt 5.txt a.txt b.txt
```

```
$ echo {1..9}  
1 2 3 4 5 6 7 8 9
```

```
$ echo {1..100..5} # from 1 to 100 with step 5  
1 6 11 16 21 26 31 36 41 46 51 56 61 66 71 76 81 86 91 96
```

```
$ echo a{,x,y,z}4 # {,x,y,x} means : nothing,x,y,z  
a4 ax4 ay4 az4
```

```
$ echo a{“,x,y,z}4
a4 ax4 ay4 az4
```

```
$ echo {,.*}.txt # with or without "."
*.txt .hidden.txt .secret.txt
```

```
$ echo *.txt *.txt
*.txt .hidden.txt .secret.txt
```

```
$ echo */*.txt
docs/111.txt docs/11.txt docs/12.txt docs/1.txt
docs/3.txt docs/5.txt docs/7.txt docs/a.txt
docs/b.txt docs/g.txt docs/i.txt
```

```
$ echo "a{b,c,d}e"
a{b,c,d}e
```

```
$ echo a{b,c,d}e
abe ace ade
```

to enable ** recursive glob :

```
$ shopt -s globstar // set recursive glob on
$ shopt globstar
```

```
globstar on
```

```
$ echo **/*.txt # recursive glob:
```

```
# find in the current dir (!)
```

```
# and all sub dirs under it
```

```
1.txt 2.txt 3.txt 4.txt 5.txt # files from current dir also
```

```
temp1/a.txt temp1/b.txt temp1/c.txt temp1/d.txt
```

```
temp2/10.txt temp2/1.txt temp2/2.txt temp2/3.txt
```

```
temp2/4.txt temp2/5.txt temp2/6.txt temp2/7.txt
```

```
temp2/8.txt temp2/9.txt
```

```
$ shopt -u globstar
```

```
$ shopt globstar
```

```
globstar off
```

```
$ echo **/*. * # now it will search only in sub dirs
```

```
temp1/a.txt temp1/b.txt temp1/c.txt temp1/d.txt
```

```
temp2/10.txt temp2/1.txt temp2/2.txt temp2/3.txt
```

```
temp2/4.txt temp2/5.txt temp2/6.txt temp2/7.txt
```

```
temp2/8.txt temp2/9.txt
```

search in subdirs only :

```
$ echo **/*.txt
```

```
temp1/a.txt temp1/b.txt temp1/c.txt temp1/d.txt
```

```
temp2/10.txt temp2/1.txt temp2/2.txt temp2/3.txt
```

```
temp2/4.txt temp2/5.txt temp2/6.txt temp2/7.txt
```

```
temp2/8.txt temp2/9.txt
```

split text in several strings:

```
$ echo 'this
```

```
> is afile
```

```
> that is vary long'
this
is afile
that is vary long
```

Trick to print ouput to stderr from process:

```
-----
ex07.sh:
  echo "normal output"
  echo "error output" >&2
-----
$ bash ex07* >/dev/null
$ bash ex07* 2>/dev/null
```

6.6 printf : display line of text

Format and print text.

Similar to echo command, but does not output new line, if not defined particularly.

```
$ printf "\rA" - return to start of the same line and
                print A.
$ printf "A\n" - print A and add enter.
```

```
$ printf "Hello World\n"
Hello World
$ printf "Hello World"
Hello World $
```

This will output A, wait 3 sec., clear line and output B:

```
$ printf "A"; sleep 3; printf "\rB\n"
```

6.7 grep : "Global Regular Expression search and Print"

<https://en.wikipedia.org/wiki/Grep>

print lines that match patterns

grep allows to enter regular expression for the search.

```
returns status $? = 0 if string is found,
returns status $? = 1 if string is not found
returns status $? = 2 if error occured
```

grep is a command-line utility for searching

plain-text data sets for lines that match a regular expression
i.e. to search files for certain patterns
Filter only lines with desired contents

- Search for a pattern within a file:
 `grep "{{search_pattern}}" {{path/to/file}}`

```
$ grep "is" *.txt
1.txt:this is an addittional try
try1.txt:This is very convinient editor,
try1.txt:Its name is tilde.
```

```
$ grep is *.txt
1.txt:this is an addittional try
try1.txt:This is very convinient editor,
try1.txt:Its name is tilde.
```

Options:

- L : list of FILES that don't include the word
 - l : list of FILES that include the result
 - i : Insensitive to upper case
 - w : Select only lines with whole word.
 - o : present ONLY WORD we search
 - r : recursive search
 - v : (inVerterd) list of rows that does not include the word
 - c : Count number of lines with word
 - q : quiet, exit immediately with 0 status if any match is found.
-

- Search for an exact string (disables regular expressions):
 `grep --fixed-strings "{{exact_string}}" {{path/to/file}}`
- Search for a pattern in all files recursively in a directory, showing line numbers of matches, ignoring binary files:
 `grep --recursive --line-number --binary-files={{without-match}}
 "{{search_pattern}}" {{path/to/directory}}`

Global Regular Expression "error" search:

```
$ grep error errors.log
error: aeasasadf
dfdfdfdf error sfsadgasdg
ldksldjks no errors
```

- i : Insensitive to upper case
 `$ grep error errors.log -i`
 error: aeasasadf
 dfdfdfdf error sfsadgasdg
 ERROR dssdsdsdsd
 ldksldjks no errors

```

-w : exact word
$ grep errors errors.log -i -w
ldksldjks no errors

-o : present only word we search, without text
$ grep error errors.log -o -i
error
error
ERROR
error

-r : recursive search in all subdirectories :
$ grep error -r
dir2/errors2.log:error: aeasasadf
dir2/errors2.log:dfdfdfdf error sfsadgasdg
errors.log:dfdfdfdf error sfsadgasdg
errors.log:ldksldjks no errors

-l : list of files that include the result :
$ grep error -r -l
dir2/errors2.log
dir2/errors.log
dir2/errors3.log
errors.log

-L : list of files that don't include the word :
$ grep error -r -L
.secret.txt
.hidden.txt
dir1/5.txt

-v : (inVerterd) list of rows that does not include the word "error":
$ grep error errors.log -v
aaaaa
bbbb

dfdf

ERROR dssdsdsdsd
sddsd
dsdsds

-v -i : (inVerterd) list of rows that does not include the word "error"
insensitive :
$ grep error errors.log -v -i
aaaaa
bbbb

dfdf

```

```
sddsd
dsdsds
```

List all files, that contain at least one row without word error:

```
$ grep error -v -r -l
dir1/b.txt
dir1/dudu.txt
dir2/errors2.log
```

List all files that does not contain any word "error":

```
$ grep error -r -L
.secret.txt
.hidden.txt
dir1/5.txt
dir1/b.txt
```

Count number of lines with word "error":

```
$ grep error errors.log -c
3
$ grep error errors.log -i -c
4
```

6.8 cat : "ConcATenite" : concatenate and print.

The cat utility serves a dual purpose: concatenating and printing.

With a single argument, it is often used to print a file to the user's terminal emulator.

With more than one argument, it concatenates several files

- Print the contents of a file to stdout:

```
cat {{path/to/file}}
```

- Concatenate several files into an output file:

```
cat {{path/to/file1 path/to/file2 ...}} > {{path/to/output_file}}
```

- Append several files to an output file:

```
cat {{path/to/file1 path/to/file2 ...}} >> {{path/to/output_file}}
```

6.9 more : present text file in parts.

Same as cat , but presents the contents
in several parts with more option

```
$ cat /usr/include/stdio.h
$ more /usr/include/stdio.h
```

6.10 less : present text file in parts and allows search

Same as cat , presents the contents
in option to move with arrows
allows search

```
$ less /usr/include/stdio.h
```

allows to find with
/ prin....
/+''enter''

6.11 head : Display n rows at the head of file

Present n rows at the head of fil

Present 3 rows at the head:

```
$ head /usr/include/stdio.h -n 3
```

6.12 tail : Display n rows at the end of file

Present n rows at the end of file

- Show last 'count' lines in file:

```
tail --lines {{count}} {{path/to/file}}
```

```
$ tail --lines 3 /usr/include/stdio.h
```

6.13 sort : Display lines in sorted order

prints the lines of its input or concatenation
of all files listed in its argument list in sorted order
regular sort:

```
$ sort fruits.txt
apple
apple
banana
banana
```

```
pineapple
strawberry
tomato
watermelon
```

-u: unic sort remove the same words:

```
$ sort fruits.txt -u
apple
banana
pineapple
strawberry
tomato
watermelon
```

-r : reverse sort :

```
$ sort fruits.txt -u -r
watermelon
tomato
strawberry
pineapple
banana
apple
```

-h: sort file sizes in human order :

```
$ sort files sizes.txt -h
10
30K
200K
500K
5M
1G
2G
```

- sort numbers:

```
$ sort numbers.txt
1
1
1212
19898990
2
33
45
```

6.14 wc : "Word Count" - standard input statistics

Count lines, words, and bytes.

The program reads either standard input or a list of computer files and generates one or more of the following statistics:

1. newline count,
2. word count,

3. byte count.

4. characters count

If a list of files is provided, both individual file and total statistics follow.

- Count all lines in a file:
 `wc --lines {{path/to/file}}`
- Count all words in a file:
 `wc --words {{path/to/file}}`
- Count all bytes in a file:
 `wc --bytes {{path/to/file}}`
- Count all characters in a file (taking multi-byte characters into account):
 `wc --chars {{path/to/file}}`
- Count all lines, words and bytes from stdin:
 `{{find .}} | wc`
- Count the length of the longest line in number of characters:
 `wc --max-line-length {{path/to/file}}`

Example:

```
$ wc ex1.c
  8  20 138 ex1.c
```

```
#8   lines
#20  words,
#138 characters
```

7 Compressing and archiving

7.1 gzip/gunzip : shrink huge files and save space

GNU zip compression utility.

Creates .gz files (original file is deleted)

Ordinary performance (similar to Zip).

Very popular in Linux, almost every Linux has gzip.

Relative fast and well decompression

Has no password

NOTE: Does not work with directories !

```
$ gzip <file>      # to compress
$ gunzip <file>    # to decompress
```

Example :

Suppose in current directory there are 3 files:

1.txt , 2.txt , 3.txt

```
$ gzip 1.txt 2.txt 3.txt
```

Normally this will replace the original file by compressed file :

```
1.txt.gz in place of 1.txt
2.txt.gz in place of 2.txt
3.txt.gz in place of 3.txt
```

Note: if the file has a hard link, the compression will not be done by gzip:

```
$ gzip 1.txt 2.txt 3.txt
zip: 1.txt has 1 other link -- unchanged
$ file 2.txt
$ file 2.txt.gz
 2.txt.gz: gzip compressed data, was "2.txt",
last modified: Thu Feb 15 11:40:21 2024, from Unix, truncated
$ gunzip 2.txt # produce 2.txt
$ gunzip 3.txt # produce 3.txt

-k : keep input files during compression (don't replace)
$ gzip -k 2.txt # now there are 2 files : 2.txt and 2.txt.gz
```

7.2 bzip2/bunzip2 : better compression than gzip

Similar usage as gzip.

Compression level is higher than in in gzip

Decompression sometimes problematic (may be).

Does not work with directories

```
$ bzip2 <file>      # to compress
$ bunzip2 <file>    # to decompress
```

Example:

```
$ bzip2 3.txt          # replace 3.txt with 3.txt.bz2
```

```
$ bunzip2 3.txt.bz2   # replace 3.txt.bz2 with 3.txt
```

```
$ bzip2 -k 3.txt      # generates 3.txt.bz2
                      # and dont change 3.txt
```

7.3 tar : Tape ARchive - back up set of files

Archiving utility (without compressing !!!).

Often combined with a compression method, such as gzip or bzip2

So for extraction of information within *.tar.gz we perform following steps:

```
*.tar.gz ---> gzip decompress ---> *.tar ----> tar extract
```

Note : jar in Java has the same flags as tar in linux

Useful to backup or release a set of files into a single file
In the past used to move files from Linux to tape.
tar stores also structure of directories, inods , etc. and send
it to tape archive.

Note : tar stores the file system along with inode data
so it stores the original permissions of the files !!!

Creating an archive:

```
-----  
$ tar -c -f <archive_file_name> <files or directories>
```

Extracting archive:

```
-----  
-x
```

Listing archive:

```
-----  
-t
```

-c: create. The opposite action is extract (-x)

-x: extract

-t: test (listing) check that archive is OK

In the past tar used >> to sent to tape

now we use -f to tell tar to use file:

-f: file. Archive created in/extracted from file

(tape used otherwise if no -f).

Note, that if we don't add -f flag

tar will read from stdin and write the result to stdout

```
-----  
Also:
```

-v: verbose. Useful to follow archiving progress.

```
-----  
Creating an archive:
```

```
$ tar -cvf <archive> <files or directories>
```

Extracting an archive:

```
$ tar -xvf <archive> <files or directories>
```

Viewing the contents of an archive or integrity check:

```
$ tar -tvf <archive>
```

Combination of compressing and archiving on the fly.

Useful to avoid creating huge intermediate files

Much simpler than with tar and bzip2\gzip!

-j --bz2 option: [un]compresses on the fly with bzip2

-z --gzip option: [un]compresses on the fly with gzip

-J --xz option: [un]compresses on the fly with xz

create tar.gz file (- czf):

```
$ tar -czfv files.tar.gz 1.txt 2.txt dir1 dir2 # create z file
```

unzip tar.gz file:

```
$ tar -xzf files.tar.gz # eXtract z file
```

list tar.gz file :

```
$ tar -tzf files.tar.gz # list z file
```

```
-----  
tar -czfv files.tar.gz 1.txt
```

```
tar -cfv 1.txt | gzip > files.tar.gz # the same operation in  
                                         # another form
```

```
-----  
Files or directories are given with paths relative to  
the archive's directory.
```

Use -C DIR or --directory=DIR, to specify the
extraction directory location.

Suppose we have following directories tree:

Desktop

```
|  
|--- linux_2312  
      |  
      |---- playground  
              |  
              |--- 1.txt  
              |--- 2.txt
```

And we want like to compress playground directory,
including name playground:

```
amits:Desktop$ tar -czf play.tar.gz -C linux_2312 -v playground/  
playground/  
playground/2.txt  
playground/1.txt
```

```
amits:Desktop$ ls  
Documents      LinuxAdminLearn.sh      play.tar.gz  
amits:Desktop$ tar -tzf play.tar.gz  
playground/  
playground/2.txt  
playground/1.txt
```

```
amits:Desktop$ mkdir try_dir  
amits:Desktop$ mv play.tar.gz ./try_dir  
amits:Desktop$ cd try_dir  
amits:Desktop$ tar -xzf play.tar.gz  
amits:Desktop$ ls  
playground  play.tar.gz  
amits:try_dir$ cd playground/  
amits:playground$ ls  
1.txt 2.txt
```

```
-----  
Example :
```

```

create archive "text_files.tar":
$ tar -c -f text_files.tar 1.txt 2.txt ./dir1 ./dir2

to check what is inside the archive text_files.tar :
$ tar -t -f text_files.tar
or
$ tar -t <text_files.tar

Now if we want to extract only 1.txt from tar file
text_files.tar :
$ tar -x -f text_files.tar 1.txt
-----
tar      -x      -f text_files.tar      1.txt
command flag  flag with its argument  argument
-----
Now we run gzip on file text_files.tar:
$ gzip text_files.tar # in place of text_files.tar
                        # it place text_files.tar.gz
$ gunzip text_files.tar
-----
$ mkdir try_dir
$ mv text_files.tar try_dir
$ gzip text_files.tar # run gzip on tar
$ ls
    text_files.tar.gz    # this file stores inode and compressed
$ gunzip text_files.tar.gz
$ ls
    text_files.tar
$ tar -xf text_files.tar
$ ls
    1.txt  2.txt  dir1  dir2  text_files.tar
-----
Example with xz :
$ tar --xz -cf data.xz 1.txt
$ tar --xz -xf data.xz
-----

```

7.4 xz/ : relative new, better compression

New application with huigh compression level.

7.5 7zip/ : better compression than bzip2

7zip supports strong AES256 encryption.
 No need to encrypt in a separate pass.
 At last a solution available for Unix and Windows!
 The tool supports most other compression formats:
 zip, cab, arj, gzip, bzip2, tar, cpio, rpm , deb.

Option 1 to install :

<https://www.linuxcapable.com/how-to-install-7-zip-on-ubuntu-linux/>
Download 7-Zip using the wget command below:

```
$ wget https://www.7-zip.org/a/7z2301-linux-x64.tar.xz
$ mkdir 7zip
$ tar -xJf 7z2301-linux-x64.tar.xz -C 7zip
$ cd 7zip
$ ./7zz
```

Option 2 to install (preferred):

<https://www.geeksforgeeks.org/how-to-install-7zip-in-ubuntu/>

```
$ sudo apt-get update
$ sudo apt-get install p7zip-full
```

Much better compression ratio than bzip2 (up to 10 to 20%).
Actually its compression is the best of all options.
Free of charge, but was developed by company
\$ man 7z # 7zip manual

Add files to 7z :

```
$ 7z a data.7z 1.txt # add files to archive
```

```
7-Zip [64] 16.02 : Copyright (c) 1999-2016 Igor Pavlov : 2016-05-21
p7zip Version 16.02 (locale=en_IL,Utf16=on,HugeFiles=on,64 bits,12
CPUs 12th Gen Intel(R) Core(TM) i7-1255U (906A4),ASM,AES-NI)
Scanning the drive:
1 file, 474 bytes (1 KiB)
Creating archive: data.7z
Items to compress: 1
Files read from disk: 1
Archive size: 409 bytes (1 KiB)
Everything is Ok
```

List data in archive:

```
$ 7z x py2403_day03.zip # to see contents of zip file
$ 7z l data.7z
```

Date	Time	Attr	Size	Compressed	Name
2024-02-26	15:18:03	A	474	295	1.txt
2024-02-26	15:18:03		474	295	1 files

Decompress files:

```
$ 7z x data.7z
```

Unzip zip file:

```
$ 7z x py2403_day03.zip
```

8 File comparison and file integrity

8.1 file..integrity..check

We can look at file data as single large number.

We can define function that translates this large number to another number called "checksum"

Can another file generate the same checksum ?

crc32 number present 4.294.967.296 different numbers
there are more than 4e9 different files

According to shovah ha yonim there are two different files with the same crc number.

8.2 crc32 : calculate crc32

install:

```
$ sudo apt install libarchive-zip-perl
```

crc32 - compute CRC-32 checksums for the given files

```
crc32 filename [ filename ... ]
```

For example:

```
$ cat 1.txt
```

```
    this is David's file
```

```
$ crc32 1.txt
```

```
    7b6accd4
```

9 Find files and check directory size

9.1 du : "Disk Usage" - file and dir usage space

<https://phoenixnap.com/kb/show-linux-directory-size>

Display the current directory size by running du without any arguments:

The system lists the current directory content and subdirectories with a

number representing the object size in kilobytes.

The current directory size is the last line.

```
$ du
```

Print the Current Directory Size in a Human-Friendly Format

Add the -h option to the du command to make the output easily readable:

```
$ du -h
```

Find the Size of a Specific Directory

```
$ du -h ./example1
```

Show Only the Total Disk Usage of a Directory

To print an extra total line at the end of the output and summarize the disk space usage for all files and directories, use the -c argument:

```
$ du -c ./example1
```

Scan up to a Chosen Subdirectory Level

Limit the scan to a certain subdirectory level with the --max-depth option.

For example, to scan only the top-level directory (example1),

use --max-depth=0:

```
$ du -h --max-depth=0 ./example1
```

To list the top directory (example1) and the first layer of subdirectories (example2 and example3), use --max-depth=1:

```
du -h --max-depth=1 ./example1
```

Disk usage: estimate and summarize file and directory space usage

- List the sizes of a directory and any subdirectories, in human-readable form (i.e. auto-selecting the appropriate unit for each size):

```
$ du -h {{path/to/directory}}
```

- List the human-readable sizes of a directory and of all the files and directories within it:

```
$ du -ah {{path/to/directory}}
```

Summary of disk usage for two directories:

```
$ du -h
```

```
8.0K ./dir2/2024
```

```
12K ./dir2
```

```
8.0K ./dir1/2024
```

```
20K ./dir1
```

```
76K .
```

```
$ du -h | sort -h
```

```
8.0K ./dir1/2024
```

```
8.0K ./dir2/2024
```

```
12K ./dir2
```

```
20K ./dir1
```

```
76K .
```

```
$ du -h -s ./dir1 ./dir2 # -s is for summary of size
```



```
20K ./dir1
12K ./dir2
```

Apparent size is the actual amount of data that can be read from the file. Block-oriented devices can only store in terms of blocks, not bytes. As a result, the disk usage is always rounded up to the next highest block

```
$ du -h -s --apparent-size ./dir1 ./dir2
```

9.2 locate : command

Uses database to find files.

Note: the database is recomputed periodically (usually weekly or daily).

Manual : the database is located in /var/lib/mlocate/mlocate.db

If we add file to disk or delete file from disk we need to update the database. To update database:

```
$ sudo updatedb
- locate a file by its exact filename
  $ locate stdio.h | less
- locate all text files in current directory:
  $ locate "$PWD/*.txt"
- locate file *.txt starting with root directory:
  $ locate "/*1.txt" | head
```

9.3 find : command

Find filenames quickly.

Find files or directories under the given directory tree, recursively

```
Suppose we need to find file stdio.h
$ cd /
$ find . -name "stdio.h" 2>/dev/null
```

```
Suppose we need to find all *.txt files starting from ~
$ cd ~
$ find . -name "*.txt"
$ cd ~/Desktop
```

There are many flags in find command ,

the flags are divided in categories :
Actions (-print by default)
Operators (-o logical OR, -a logical AND)
Tests (+n(>n) -n(<n) n (=n)) check size, time , etc.
For example:
-mtime +5 : if file modified in 5 last day

List all files that modified in 14 last days :
\$ find -mtime -14

-mtime n
File's data was last modified less than, more than or exactly
n*24 hours ago.

List all files that size of them is larger than 2G :
\$ find -size +2G 2>/dev/null

List all files from ~ that are at least 100Mb :
\$ find ~ -size +100M -name "*cl*" 2>/dev/null

9.4 tree : show curr. dir as tree.

Show current directory as tree ,
with sizes in human style and permissions :

Combine tree with du :

\$ tree --du -h -p

-L Level
-s Print the size of each file in bytes along with the name.
-h Print the size of each file but in a more human readable way,
e.g. appending a size letter for kilobytes (K), megabytes (M),
gigabytes (G), terabytes (T), petabytes (P) and exabytes (E).

--du For each directory report its size as the accumulation of sizes
of all its files and sub-directories

--inodes
Prints the inode number of the file or directory

-p Print the file type and permissions for each file (as per ls
-l).

-u Print the username, or UID # if no username is available, of the

- file.
- g Print the group name, or GID # if no group name is available, of the file.
- Print files and directories up to 'num' levels of depth (where 1 means the current directory):
 \$ tree -L {{num}}
- Print directories only:
 \$ tree -d
- Print hidden files too with colorization on:
 \$ tree -a -C

10 Dirs and files manipulation

10.1 directory.permissions.in.Linux

see <https://unix.stackexchange.com/questions/21251/execute-vs-read-bit-how-do-directory-permissions-in-linux-work>

When applying permissions to directories on Linux, the permission bits have different meanings than on regular files.

The read bit (r) allows the affected user to list the files within the directory

The write bit (w) allows the affected user to create, rename, or delete files within the directory, and modify the directory's attributes

The execute bit (x) allows the affected user to enter the directory, and access files and directories inside

The sticky bit (T, or t if the execute bit is set for others) states that files and directories within that directory may only be deleted or renamed by their owner (or root)

10.2 basename : store the name of dir based on full path

Remove leading directory portions from a path.

- Show only the file name from a path:
 \$ basename {{path/to/file}}
- Show only the rightmost directory name from a path:

```
$ basename {{path/to/directory/}}
```

10.3 nautilus : file explorer in Ubuntu

nautilus : native file explorer in Ubuntu

10.4 chmod : CHange file MODe (permissions and spec. modes)

- At the begin of -rwxrwxrwx is the type of file
(soft link (l), dir(d), regular file(-))

r - read rights (ex: present the contents with cat)

w - write rights (ex: write with nano)

x - execute rights (ex: run executable)

u - user (file's owner)

g - group

o - others

Process knows who is its owner and what are groups that run it.

When process wants TO USE FILE it can check the rights to use it.

inode of file contains info of file's owner and what is the file's group. Sometimes operational system prevents from use the files, and sometimes process checks the match between process's owner and group again file's inode information.

If there is no match - the process stops its execution.

In case of files: all the above info relates only to the CONTENTS of the file, not INODE and not FILE's NAME.

In case of file of type "directory", the content of file contains the list of files in the folder that can be read with "ls" command.

Read the file of type "directory" is to run ls command.

File of type "directory" contains the list of files and directories within the directory.

FILE PROTECTION must be done at two (!) levels :

1. Permission of the file itself.
2. Permissions of the folder that contains the file (to protect the info about the file).

Otherwise the PROTECTED file can be DELETED from the directory's list of files (by delete the directory) !!!

Because "rm" command only deletes the registration of the file within the directory.

Delete of the write protected file related to directory permissions.

If directory of the file has write permissions and at the same time the file has no write permissions - it will be deleted !!!:

```

$ ls -l 1.txt
-rw-rw-r-- 1 amit amit 146 Sep 22 12:54 1.txt
$ chmod u-w 1.txt
$ ls -l 1.txt
-r--rw-r-- 1 amit amit 146 Sep 22 12:54 1.txt
$ rm 1.txt
rm: remove write-protected regular file '1.txt'? Y
$ ls -l 1.txt
ls: cannot access '1.txt': No such file or directory

```

Only write protection to file's directory really protects the files in this directory from being deleted. But it does not protect the files from being edited !!! So both protection is needed : the file protect and the directory protect.

So, protection of the file from being deleted must be done in two levels simulataneously:

1. Protection of file's folder (write option - off).
2. Protection of the file by setting the file's own protection option (write option off).

CMOD command:

```
$ chmod [symbolic permissions/octal premissions] filename
```

Symbolic permissions :

Who:	Action:	Permission type:
u - user	+ (turn on)	r(read)
g - group	- (turn off)	w(write)
o - others	= (set)	x(execute)
a - all		

Take combination of u,g,o,a ; one of +/-/= ; combination of r,w,x

- u+r : turn on "r" of user
- additional permissions separated by " , " :
 u+r,g-wx : set on r to user and set off wx to group
- ug+x : set on x for u and g
- unset all r : ugo-r ; a-r
- u= #u equals nothing
- unset rxw for u : u-rxw
- use of = : u=r

Octal permissions:

```

$ chmod 777 1.txt
  U   G   O
  rwX rwX rwX
  7   7   7

```

```

$ chmod 746 1.txt
  U   G   O
  rwX rw- r--
  7   6   4

```

```

-----
-R : Apply chmod recursively.
Note : uppercase X will be set only for folders or executable files.
      always if you want to permit execution to folders and
      files - use X not x !!!
-R : change files and directories recursively
    $ chmod -R o-rwx ./ # Unset rwx permissions for o(thers) recursevely
    $ chmod -R o+rwX ./ # Set rwx permissons recuresevely
                        # Uppercase X : x will be set only for folders or
                        # executable files !
    $ tree -p          # present permissions also

```

To make directory dir1 and everything in it available to everyone:

```

$ chmod -R a+rX dir1
$ ls -liR dir1

```

```

-----
See https://linuxhandbook.com/suid-sgid-sticky-bit/
In addition of 9 options {u(r,w,x), g(r,w,x), o(r,w,x)} there are 3
additional options

```

- Set uid (SUID)
- Set gid (SGID)
- Sticky bit

```

-----
SUID : -rwsr-xr-x # s here is SUID permission
When the SUID bit is set on an executable file, this means
that the file will be executed with the same permissions as the
<owner of this file> (for example - root).
Practical example: If you look at the binary executable file
of the passwd command, it has the SUID bit set:

```

```

$ ls -lai /usr/bin/passwd
2885961 -rwsr-xr-x 1 root root 59976 Feb  6 14:54 /usr/bin/passwd

```

This means that any user running the passwd command will be running it with the same permission as root. The passwd command needs to edit files like /etc/passwd, /etc/shadow to change the password. These files are owned by root and can only be modified by root. But thanks to the setuid flag (SUID bit), a regular user will also be able to modify these files (that are owned by root) and change his/her password. This is the reason why you can use the passwd command to change your own password despite of the fact that the files this command modifies are owned by root.

```

$ stat /usr/bin/passwd
File: /usr/bin/passwd
Size: 59976      Blocks: 120      IO Block: 4096   regular file
Device: 10303h/66307d Inode: 2885961    Links: 1
Access: (4755/-rwsr-xr-x)  Uid: (    0/    root)  Gid: (    0/    root)

```

How to set SUID bit?

I find the symbolic way easier while setting SUID bit.
You can use the chmod command in this way:

```
$ chmod u+s file_name
```

Difference between small s and capital S as SUID bit
The S as SUID flag means there is an error that you should look into.
You want the file to be executed with the same permission as the owner
but there is no executable permission on the file.
Which means that not even the owner is allowed to execute
the file and if file cannot be executed, you won't get
the permission as the owner.

SGID # example is "s" within : drwx rws r-x
is similar to SUID. With the SGID bit is set,
any user executing the file will have same permissions
as the group owner of the file.
It's benefit is in handling the directory.
When SGID permission is applied to a directory, all sub directories
and files created inside this directory will get the same group
ownership as main directory (not the group ownership of the user that
created the files and directories).
How to set SGID?
You can set the SGID bit in symbolic mode like this:

```
$ chmod g+s directory_name
```

The Sticky Bit # Its example is "t" within drwx rwx rwt
Sticky bit:

- for the files: in the past used for file mark
- for the directory: the sticky bit sets shared directory:
every user can create in this directory file, but the other user
cannot delete this files that are not of the user. The user
can update the files of other users but cannot delete them (because
the deletion is related to directory permissions).

Sticky Bit works mostly on the directory.

With sticky bit set on a directory, all the files
in the directory can only be DELETED or RENAMED by the
file owners ONLY or the root.

This is typically used in the /tmp directory that works
as the trash can of temporary files. This means that a user (except root)
cannot delete the temporary files created by other users in the /tmp director

```
$ ls -ldi /tmp
5242881 drwxrwxrwt 25 root root 20480 Feb 20 12:45 /tmp
$ stat /tmp
File: /tmp
```

```

        Size: 20480      Blocks: 48      IO Block: 4096    directory
        Device: 10303h/66307d Inode: 5242881    Links: 24
        Access: (1777/drwxrwxrwt)  Uid: (    0/    root)  Gid: (    0/    root)
How to set the sticky bit?
As always, you can use both symbolic and numeric mode to set the sticky
bit in Linux.
    $ chmod +t my_dir

```

10.5 ln : Creates links to files and directories

Creates links to files and directories.

Two types of links in Linux:

- Symbolic links
 - Hard links
- Create a symbolic link to a file or directory:


```
ln -s {{/path/to/file_or_directory}} {{path/to/symlink}}
```
 - Create a hard link to a file:


```
ln {{/path/to/file}} {{path/to/hardlink}}
```
 - Overwrite an existing symbolic link to point to a different file:


```
ln -sf {{/path/to/new_file}} {{path/to/symlink}}
```

To create symbolic link (-s option):

```
$ ln -s 1.txt 1_soft.txt
```

To create hard link:

```
$ ln 1.txt 1_hard.txt
```

Linux file system EXT and hard link

How files are stored in disk:

Linux file system name is EXT (Extended).

All files are stored with 3 linked parts:

```
file name-->inode(information node)-->data of the file
```

-
- Hard link functions as original file ,
 Hard link is simply new <file name> that linked
 to the inode of the original file.
 Hard link is only additional name to the same file (inode + data) !!!
 Hard link is difficult to recognize. It can be recognized only
 by inode number. If inode number of the file is similar to inode
 number of another file - it is hard link.
 Sector on disk is analogues to address.
 So hard link must be located on the same partition,
 Hard link can't link to another partition or the network address of

folders!!! For example, we cannot create hard link on USB disk with link to file on the hard disk.

User cannot create hard link for folders, because recursive command on folders tree with hard link would cause to infinite loop.

But he can create soft link to folders.

Operational system Linux creates hard link to directories:

"." is a hard link to current directory.

".." is the hard link with same inode in subdirs as "." in top dir.

Every folder contain at list one hard link named ".".

Every subfolder contains hard link "..".

Hard link counter of folder less 1 shows how many subfolders has the folder.

```
$ mkdir dir31
```

```
$ mkdir dir31/dir32
```

```
$ mkdir dir31/dir32/dir33
```

```
$ ls -li
```

```
1502894 -rw-rw-r-- 1 amit amit 86 Sep 22 05:42 2_txt
```

```
1502893 drwxrwxr-x 3 amit amit 4096 Sep 22 07:53 dir31 # 2 subfolders
```

rm of hard link does not delete the original file !

rm of orogonal file does not delete the hard link and file contents !!!

In the inode there is inode counter. If we delete the hard link

(additional name of the same file), inode counter is diminished by 1.

IF inode counter reaches zero - the operational system knows that the space in the disk in empty and it can be used for other purposes.

```
$ ls -li # here inode counter is 2
```

```
7753157 -rw-rw-r-- 2 amits amits 45 Feb 19 19:26 1.txt
```

```
7753157 -rw-rw-r-- 2 amits amits 45 Feb 19 19:26 hard.1.txt
```

```
$ ln 1.txt 1.hard
```

File name	inode	data
1.txt	---> 123	---> How do you do ?
	Shemuel	
	^	
1.hard	-----	

Example :

```
$ ln 1.txt hard.1.txt
```

```
$ stat hard.1.txt
```

```
File: hard.1.txt
```

```
Size: 45          Blocks: 8          IO Block: 4096   regular file
```

```
Device: 10303h/66307d Inode: 7764243    Links: 2
```

```
Access: (0664/-rw-rw-r--)  Uid: ( 1001/   amits)   Gid: ( 1001/   amits)
```

```
Access: 2024-02-19 15:46:06.573765125 +0200
```

```

        Modify: 2024-02-15 16:31:55.646124081 +0200
        Change: 2024-02-19 15:45:28.276807762 +0200
        Birth: 2024-02-15 16:31:55.646124081 +0200
$ stat 1.txt
  File: 1.txt
  Size: 45          Blocks: 8          IO Block: 4096   regular file
Device: 10303h/66307d Inode: 7764243    Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1001/   amits)   Gid: ( 1001/   amits)
Access: 2024-02-19 15:46:06.573765125 +0200
Modify: 2024-02-15 16:31:55.646124081 +0200
Change: 2024-02-19 15:45:28.276807762 +0200
Birth: 2024-02-15 16:31:55.646124081 +0200

$ ls -li # returns inode numbers of files !!!
$ ls -li
    7764243  1.txt
    7764243  hard.1.txt

```

-
- Soft link has <its own inode> and the data that includes path to linked file.
Soft link is separate file with data that contain address of linked file.
Soft link is signed with "l" at the head of list of permissions:
Link counter of soft link is always 1.

```

$ ls -la
lrwxrwxrwx 1 amits amits    5 Feb 19 21:15 soft1.txt -> a.txt

File name      inode      data
1.txt         ---> 123      ---> How do you do ?
              Shemuel

1.soft        ---> 456      ---> ./1.txt
              L

```

Sector on disk is analogues to address.
So hard link must be located on the same partition,
Hard link can't link to another partition or the network address of folders!!! For example, we cannot create hard link on USB disk with link to file on the hard disk.

10.6 find : Search for a file

The find command in UNIX is a command line utility for walking a file hierarchy.
It can be used to find files and directories and perform subsequent operations on them.
It supports searching by file, folder, name, creation date,

modification date, owner and permissions.

By using the '-exec'

other UNIX commands can be executed on files or folders that found.

Syntax:

```
$ find [where to start searching from]
      [expression determines what to find]
      [-options] [what to find]
```

1. Search for a file with a specific name.

```
$ find ./GFG -name sample.txt
```

2. Search for a file with pattern.

```
$ find ./GFG -name *.txt
```

3. Find and delete a file with confirmation.

```
$ find ./GFG -name sample.txt -exec rm -i {} \;
```

4. Find empty files and directories.

```
$ find ./GFG -empty
```

5. Search for file with given permissions.

```
$ find ./GFG -perm 66
```

6. Displays repositories and sub-repositories

```
$ find . -type d
```

7. Search text 'Geek' within multiple text files.

```
$ find ./ -type f -name "*.txt" -exec grep 'Geek' {} \;
```

- Find files by extension:

```
find {{root_path}} -name '{{*.ext}}'
```

- Find files matching multiple path/name patterns:

```
find {{root_path}} -path '{{**/path/**/*.*ext}}'
-or -name '{{*pattern*}}'
```

- Find directories matching a given name,
in case-insensitive mode:

```
find {{root_path}} -type d -iname '{{*lib*}}'
```

- Find files matching a given pattern, excluding
specific paths:

```
find {{root_path}} -name '{{*.py}}' -not
-path '{{*/site-packages/*}}'
```

- Find files matching a given size range, limiting
the recursive depth to "1":

```
find {{root_path}} -maxdepth 1 -size
```

```
{{+500k}} -size {{-10M}}
```

- Run a command for each file (use {} within the command to access the filename):

```
find {{root_path}} -name '{{*.ext}}'  
-exec {{wc -l {} }}\;
```
- Find files modified in the last 7 days:

```
find {{root_path}} -daystart -mtime -{{7}}
```
- Find empty (0 byte) files and delete them:

```
find {{root_path}} -type {{f}} -empty -delete
```

Examples :

Find all files *.c starting from current directory:

```
$ find . -name "*.c"
```

Find all files *.jpg starting with root directory:

```
$ find / -name "*.jpg" -print
```

Find all files *.jpg starting from home directory:

```
$ find ~ -name "*.jpg" -print
```

Insert "" to make globing at find command level,
no at bash level, before expansion for find command:

```
$ find -name "*.txt"  
./amit.txt  
./docs/11.txt  
./docs/.secret.txt  
./docs/5.txt  
./docs/7.txt
```

10.7 touch : Create files or update access times

Update the access and modification times if the file exists

A FILE argument that does not exist is created empty, unless -c or -h is supplied.

1. In its default usage, it is the equivalent of creating or opening a file and saving it without any change to the file contents. touch avoids opening, saving, and closing the file. Instead it simply updates the dates associated with the file or directory. An updated access or modification date can be important for a variety of other programs such as backup utilities.
2. The touch command can also be useful for quickly creating files for programs or scripts that require a file with a

specific name to exist for successful operation of the program, but do not require the file to have any specific content.

Example:

```
create files 1.txt , 2.txt, ... , 5.txt
touch {1..5}.txt
```

10.8 stat : display file (inode) and file system info

stat() is a Unix system call that returns file attributes about an inode.

The semantics of stat() vary between operating systems. As an example, Unix command ls uses this system call to retrieve information on files that includes:

```
atime: time of last access (ls -lu)
mtime: time of last modification (ls -l)
ctime: time of last status change (ls -lc)
```

The stat() and lstat() functions take a filename argument. If the file is a symbolic link, stat() returns attributes of the eventual target of the link, while lstat() returns attributes of the link itself.

File system information:

```
$ stat -f try.py
  File: "try.py"
  ID: 5a2e6f11fdd5900a Namelen: 255      Type: ext2/ext3
Block size: 4096      Fundamental block size: 4096
Blocks: Total: 120454536 Free: 91353786 Available: 85216660
Inodes: Total: 30670848 Free: 29126190
```

10.9 ls : "LiSt" files and dirs

Lists the contents of files and directories. Without options and arguments displays a list of the names of all files in the current working directory in alphabetical order

-i : return inode numbers of files !!

-l : displays detailed information about files and directories in a long format

-a : includes hidden files and directories in the listing

-t : sorts files and directories by their last modification time,
most recently modified ones first

-S : sorts files and directories by their sizes,
listing the largest ones first

-r : reverse order (most recent at the end)

-R : lists files and directories recursively, including subdirectories

-h : prints file sizes in a human-readable format (e.g., 1K, 234M, 2G)

-d : info for directories only.

ls command examples:

```
ls          # files list (alef bet order)
ls /        # root dir
ls .        # current dir
ls          # current dir
ls ~        # current dir
ls -l       # display detailed info: long format
ls -a       # include hidden files
ls -lh      # size of files in human format
ls -lah     # detailed info long format, include hidden files
             size of files in human format
ls [0-9].txt #
ls -R       # include subdirs
ls -ld docs # dir info only
ls -t       # most recent first
ls -r       # oldest first
ls -S       # sort by size (largest first)
ll          # alias of ls -l
```

file types:

- : regular file

d : directory (file listing a set of files)

l : soft link file (Files referring to the name of another file)

s : socket file (inter process communication)

p : pipe file (Used to cascade programs)

c : char file

b : block file driver (disk) communication files

search hidden files:

```
ls *.txt *.txt
```

```
$ ls [0-9].txt
```

```
1.txt 3.txt 5.txt 7.txt
```

```
amits:temp1$ ls -l [a-c].txt
-rw-rw-r-- 1 amits amits 0 Feb 15 13:55 a.txt
-rw-rw-r-- 1 amits amits 0 Feb 15 13:55 b.txt
-rw-rw-r-- 1 amits amits 0 Feb 15 13:55 c.txt
amits:temp1$ ls -l {a..c}.txt # same result
-rw-rw-r-- 1 amits amits 0 Feb 15 13:55 a.txt
-rw-rw-r-- 1 amits amits 0 Feb 15 13:55 b.txt
-rw-rw-r-- 1 amits amits 0 Feb 15 13:55 c.txt
$ ls -li # returns inode numbers of files !!!
7764243 1.txt
7764243 hard.1.txt
```

10.10 du : "Disk Usage" - file and dir usage space

<https://phoenixnap.com/kb/show-linux-directory-size>

Display the current directory size by running du without any arguments:
The system lists the current directory content and subdirectories with a number representing the object size in kilobytes.

The current directory size is the last line.

```
$ du
```

Print the Current Directory Size in a Human-Friendly Format

Add the -h option to the du command to make the output easily readable:

```
$ du -h
```

Find the Size of a Specific Directory

```
$ du -h ./example1
```

To print an extra total line at the end of the output and summarize the disk space usage for all files and directories, use the -c argument:

```
$ du -c ./example1
```

Disk usage: estimate and summarize file and directory space usage

- List the sizes of a directory and any subdirectories, in human-readable form (i.e. auto-selecting the appropriate unit for each size):

```
$ du -h {{path/to/directory}}
```

- List the human-readable sizes of a directory and of all the files and directories within it:

```
$ du -ah {{path/to/directory}}
```

Summary of disk usage for two directories:

```
$ du -h
```

```

8.0K ./dir2/2024
12K ./dir2
8.0K ./dir1/2024
20K ./dir1
76K .
$ du -h | sort -h
8.0K ./dir1/2024
8.0K ./dir2/2024
12K ./dir2
20K ./dir1
76K .
$ du -h -s ./dir1 ./dir2 # -s is for summary of size
20K ./dir1
12K ./dir2

```

Apparent size is the actual amount of data that can be read from the file. Block-oriented devices can only store in terms of blocks, not bytes. As a result, the disk usage is always rounded up to the next highest block

```
$ du -h -s --apparent-size ./dir1 ./dir2
```

10.11 ncdu : "Norton Commander and Disk Usage" - file and dir usage space

- Analyze the current working directory:
\$ ncdu
- Colorize output:
\$ ncdu --color {{dark|off}} # dark is color , off is regular
- Analyze a given directory:
\$ ncdu {{path/to/directory}}
- Save results to a file:
\$ ncdu -o {{path/to/file}}

10.12 fdisk : partitions in hard disk

- List partitions:
\$ sudo fdisk -l

10.13 lsblk : Lists information about block(storage) devices

- List all storage devices in a tree-like format:
\$ lsblk

- Also list empty devices:
\$ lsblk -a
- Print the SIZE column in bytes rather than in a human-readable format:
\$ lsblk -b
- Output info about filesystems:
\$ lsblk -f

10.14 df : overview of the filesystem disk space usage

- Display all filesystems and their disk usage in human-readable form:
\$ df -h
- Display all filesystems and their disk usage:
\$ df

10.15 cd : "Change Dir"

```

go to parent directory : cd ..
go to home directory   : cd ~
go to root directory   : cd /
go to prev. dir.       : cd -

$ cd -
/home/amit
$ cd -
/home/amit/Documents

```

10.16 pwd : "Print Working Directory"

10.17 mkdir : "MaKe DIRectory"

The mkdir command in Linux allows the user to create directories. This command can create multiple directories at once as well as set the permissions for the directories.

This command can create multiple directories at once as well as set the permissions for the directories.

- Create specific directories:
mkdir {{path/to/directory1 path/to/directory2 ...}}
- Create specific directories and their [p]arents if needed:

```

    mkdir -p {{path/to/directory1 path/to/directory2 ...}}
-p to create parent folders :
    $ mkdir -p aa/bb/cc

- Create directories with specific permissions:
    mkdir -m {{rwxrw-r--}} {{path/to/directory1 path/to/directory2 ...}}

```

```

$ mkdir aa/bb/cc
mkdir: cannot create directory 'aa/bb/cc': No such file
    or directory
$ mkdir -p aa/bb/cc # now it is performed

```

10.18 cp : "CoPy" file to file, file to dir, dir to dir

Copy files and directories

By default, the cp command copies files without preserving their attributes such as permissions, timestamps, and ownership.

Recursively copy a directory's contents :

```

cp -R {source_directory} {target_directory}
cp -r {source_directory} {target_directory}

```

When the program has two arguments of path names to <files>, the program copies the contents of the first <file> to the second <file>, creating the second <file> if necessary.

When the program has one or more arguments of path names of <files> and following those an argument of a path to a <directory>, then the program copies <each source file> to <the destination directory>, creating any files not already existing (!).

When the program's arguments are the path names to two <directories>, cp copies all files in the source <directory> to the destination <directory>, creating any files or directories needed. This mode of operation requires an additional option flag, typically -r, to indicate the recursive copying of directories. If the destination directory already exists, the source is copied into the destination, while a new directory is created if the destination does not exist.

Example:

```

mkdir temp{1..5}

```

Renaming a file by copying and deleting it

Linux users copy a file by using the "cp" command.

When you copy a file, you give the source files and rename the files.

```

$ cp old_file new_file

```

As an example, if we were to rename “august.png” to “september.png”:

```
$ cp august.png september.png
```

This will create a copy of the file with a new name.

But now you’ll have two copies.

You’ll resolve this by deleting the old file.

You delete with “rm.”

```
$ rm august.png
```

10.19 mv : "MoVe" files or dirs from one place to another.

If both filenames are on the same filesystem, this results in a simple file rename; otherwise the file content is copied to the new location and the old file is removed.

Using mv requires the user to have write permission for the directories the file will move between. This is because mv changes the content of both directories (i.e., the source and the target) involved in the move. When using the mv command on files located on the same filesystem, the file’s timestamp is not updated.

rm of hard link does not delete the original file !

rm of original file does not delete the hard link and file contents that is linked

Example : move interactive (-i) :

```
$ mv 1.txt b.txt -i
```

```
cp: overwrite ‘b.txt’? y
```

```
$ mv dir1 dir2      # move dir1 under dir2
```

```
$ mv dir2/dir1 ./   # move dir1 under current dir
```

You can rename a file by “moving it” to a new file name.

```
$ mv august.png september.png
```

This actually does what you would think of as a “rename” command and in a single st

But it should be noted that it’s also the general “move” command.

So, you are moving the file at the same time.

If you move it to a different directory, it will be moved as well as renamed.

10.20 rm : "ReMove" removes references(!) to objects from the filesystem

The rm command removes references to objects from the filesystem using the unlink system call, where those objects might have had multiple references (for example, a file with two different names),

and the objects themselves are discarded only when all references have been removed and no programs still have open handles to the objects.

The command generally does not destroy file data, since its purpose is really merely to unlink references, and the filesystem space freed may still contain leftover data from the removed file. This can be a security concern in some cases, and hardened versions sometimes provide for wiping out the data as the last link is being cut, and programs such as <shred> and <srm> are available which specifically provide data wiping capability.

`rm` : Remove specific files

`rm -i` : Remove specific files [i]nteractively
prompting before each removal.

`rm -r` : (r)ecursively delete a directory and all its contents (!)

Note : normally `rm` will not delete directories, while `rmdir` will only delete empty directories). But `-r` allows to remove directories:

```
$ rm dir3
rm: cannot remove 'dir3' Is a directory
$ rm dir3 -r # remove dir3 and all child directories
$           # Done directory remove

$ rm dir1/{2..4}.txt
$ rm dir2/{1,4,5}.txt
```

Remove all files with answer "yes" to all prompts :
\$ yes | rm -r Documents/

10.21 `rmdir` : "ReMove DIR" remove empty(!) dir from filesystem.

Note : removes only empty directories !!!

```
$ rmdir dir1
rmdir: failed to remove 'dir1': Directory not empty
```

`rmdir` : remove specific empty directory
`rmdir -p` : remove specific directory and all its ancestors - "AVOT"

10.22 `tree` : show curr. dir as tree.

Show current directory as tree ,
with sizes in human style and permissions :

```
$ tree --du -h -p
```

- L Level
- s Print the size of each file in bytes along with the name.
- h Print the size of each file but in a more human readable way, e.g. appending a size letter for kilobytes (K), megabytes (M), gigabytes (G), terabytes (T), petabytes (P) and exabytes (E).

- du For each directory report its size as the accumulation of sizes of all its files and sub-directories

- inodes
 Prints the inode number of the file or directory

- p Print the file type and permissions for each file (as per ls -l).

- u Print the username, or UID # if no username is available, of the file.
- g Print the group name, or GID # if no group name is available, of the file.

- Print files and directories up to 'num' levels of depth (where 1 means the current directory):
 \$ tree -L {{num}}

- Print directories only:
 \$ tree -d

- Print hidden files too with colorization on:
 \$ tree -a -C

10.23 file : determine file type.

- Give a description of the type of the specified file. Works fine for files with no file extension:
 file {{path/to/file}}
- b, -brief : This is used to display just file type in brief mode

Examples:

```
$ type -a cat
```

```

$ file [a-z]*
$ file /bin/cat
$ file tex_file_data_extract

$ file $(which test) # $( ) means substitute command
/usr/bin/test: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=68ccce12c6a2a96d3a40fb5868f8b8c45e6015a5, for GNU/Linux 3.2.0,
stripped
$ file 'which test' # ' ' means substitute command
/usr/bin/test: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=68ccce12c6a2a96d3a40fb5868f8b8c45e6015a5, for GNU/Linux 3.2.0,
stripped

```

11 User Groups

11.1 CreateNewUser - steps to create new user

1. Create Home : `$ sudo useradd tal -m` # new user with home directory
2. Skeleton files : copy files from `/etc/skel` to `/home/tal`
Note : copy without `"."` and `".."`
3. set shell type : `$ sudo usermod -s /bin/bash tal`
4. set pw : `sudo passwd tal`
5. add user to sudo group
: `$ sudo usermod -aG sudo david`
Debian : `"sudo"`
RedHat : `"wheel"`
or...
`visudo` (edit sudoer file)

11.2 visudo - edit sudoer file

Safely edit the sudoers file.

- Edit the sudoers file:
`sudo visudo`
- Check the sudoers file for errors:
`sudo visudo -c`
- Edit the sudoers file using a specific editor:
`sudo EDITOR={{editor}} visudo`
- Display version information:
`visudo --version`

11.3 passwd : Change a user's PASSWoRD.

Change user password

- Change the password of the current user interactively:
passwd
- Change the password of a specific user:
passwd {{username}}
- Get the current status of the user:
passwd -S
- Make the password of the account blank
(it will set the named account passwordless):
passwd -d

11.4 useradd : add user to the system

It is not simple, more complicated
relative to adduser (simple perl script that uses useradd)

When we use useradd, it creates the account without password :

```
$ sudo useradd tal
password for amit:
$
```

But we cannot enter the account with su - tal
because it has not user:

```
$ su - tal
Password:
scdsds
ssu: Authentication failure
```

To give to user tal password, run following command (with sudo):

```
$ passwd tal
passwd: You may not view or modify password information for tal.
$ sudo passwd tal
New password:
Retype new password:
passwd: password updated successfully
```

Create user with home directory as his name use \$ useradd -m

```
$ sudo useradd tal -m
$ su - tal
Password:
$ sudo passwd tal
New password:
Retype new password:
```

```
passwd: password updated successfully
$ su - tal
Password:
$
$ pwd
/home/tal
$ echo $SHELL # when we create user with useradd , shell will be
/bin/sh
```

By default the user tal uses shell "dash", to define shell to user we need define option -s /bin/bash when we define new user with useradd command

But now, when the user already exists, we can change the shell type with usermod command.

```
$ sudo usermod -s /bin/bash tal
$ su - tal
Password:
$ ll
total 36
drwxr-x--- 4 tal  tal  4096 Sep 27 16:02 ./
drwxr-xr-x 5 root root 4096 Sep 27 15:58 ../
-rw-r--r-- 1 tal  tal   220 Jan  6  2022 .bash_logout
-rw-r--r-- 1 tal  tal  3771 Jan  6  2022 .bashrc
drwx----- 2 tal  tal  4096 Sep 27 16:02 .cache/
drwxr-xr-x 5 tal  tal  4096 Jul 11 19:21 .config/
-rw-r--r-- 1 tal  tal    22 Sep  8  2011 .gtkrc-2.0
-rw-r--r-- 1 tal  tal   516 Dec 17  2013 .gtkrc-xfce
-rw-r--r-- 1 tal  tal   807 Jan  6  2022 .profile
```

Above skeleton files are within /etc/skel directory, and can be copied from it manually. But when we copy the file from one user to another user, it will have the user name of previous user !!!

There are two ways to solve this:

- Allow the user to copy files.
- Change the user of the files.

Note: don't run command `cp -r /etc/skel/*` because it will copy also "." And ".." that is not desirable.

So it is preferred to copy files by exact list of its names.

Things to concern when we create new user:

- User's home
- User's skeleton files
- User's shell
- User's password

Note: for useradd don't use -P option !!!

-P, --prefix PREFIX_DIR # Gibrish
Apply changes in the PREFIX_DIR directory and use the configuration files from the PREFIX_DIR directory.
This option does not chroot and is intended for preparing a cross-compilation target. Some limitations: NIS and LDAP users/groups are not verified. PAM authentication is using the host files. No SELINUX support.

11.5 \$SHELL : env. variable (check current shell type)

<https://unix.stackexchange.com/questions/43499/difference-between-echo-shell-and-which-bash>

```
$ echo $SHELL
/bin/zsh
$ which bash
/bin/bash
```

The first command tells me which shell will be executed by login when you log in. In my case, /bin/zsh.
The second command tells me the first occurrence in my \$PATH the bash command can be found.

11.6 adduser : perl script, that uses useradd

simple perl script (that uses useradd command internally):

```
$ sudo adduser david
[sudo] password for amit:
Adding user 'david' ...
Adding new group 'david' (1001) ...
Adding new user 'david' (1001) with group 'david'; ...
Creating home directory '/home/david' ...
Copying files from '/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for david
Enter the new value, or press ENTER for the default
Full Name []: David
Room Number []: 1
Work Phone []: 046778723
Home Phone []: 0584266865
Other []: no
Is the information correct? [Y/n] Y

$ cat /etc/passwd | grep bash # check users of bash
root:x:0:0:root:/root:/bin/bash
```

```
amits:x:1001:1001:AmitS,,,:/home/amits:/bin/bash
david:x:1000:1000:David,1,046778723,0584266865,no:/home/david:/bin/bash
```

11.7 userdel : delete user from the system

it is not simple, more complicated
relative to deluser (simple perl script that uses userdel)

Remove the user:

```
$ userdel -r moshe
```

Files in the user's home directory will be removed
along with the home directory itself and the user's
mail spool. Files located in other file systems
will have to be searched for and deleted manually.

11.8 deluser : perl script, that uses userdel

It is simple Perl script (that uses userdel command)

Delete a user account and related files.
The userdel command modifies the system account files,
deleting all entries that refer to the user name LOGIN.
The named user must exist.

Try to delete user tal:

```
$ sudo deluser --remove-all-files tal
....
/usr/sbin/deluser: Cannot handle special file /proc/3614/fd/1
/usr/sbin/deluser: Cannot handle special file /proc/3614/fd/2
/usr/sbin/deluser: Cannot handle special file /etc/systemd/
system/sudo.service
Removing user 'tal' ...
Warning: group 'tal' has no more members.
Done.
```

11.9 /etc/group - file with list of groups

```
$ head /etc/group
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
adm:x:4:syslog,amits
tty:x:5:syslog
disk:x:6:
lp:x:7:
```

```
mail:x:8:
news:x:9:
```

```
$ cat /etc/group | grep amits
adm:x:4:syslog,amits
cdrom:x:24:amits
sudo:x:27:amits
dip:x:30:amits
plugdev:x:46:amits
lpadmin:x:120:amits
smbashare:x:131:amits
amits:x:1001:
```

check who has permission to use sambashare (program to share files):

```
$ cat /etc/group | grep sambashare
smbashare:x:136:amit
```

11.10 /etc/passwd : contains info what users are in the system

```
$ cat /etc/passwd | grep amit
amits:x:1001:1001:AmitS,,,:/home/amits:/bin/bash
# /bin/bash : what shell the user uses
```

11.11 groups : present all groups of the user

groups - print the groups a user is in

```
$ groups david
david : david sudo
$ groups amits
amits : amits adm cdrom sudo dip plugdev lpadmin sambashare
$ groups
amits adm cdrom sudo dip plugdev lpadmin sambashare
```

11.12 chown : CHange OWNEr (user/group) of files and dirs

Change user and group ownership of files and directories.

Unprivileged (regular) users who wish to change the group membership of a file that they own may use chgrp.

- Change the owner user of a file/directory:
chown {{user}} {{path/to/file_or_directory}}
- Change the owner user and group of a file/directory:
chown {{user}}:{{group}} {{path/to/file_or_directory}}

- Recursively change the owner of a directory and its contents:
chown -R {{user}} {{path/to/directory}}
- Change the owner of a symbolic link:
chown -h {{user}} {{path/to/symlink}}
- Change the owner of a file/directory to match a reference file:
chown --reference={{path/to/reference_file}} {{path/to/file_or_directory}}

```
$ mkdir project_dir
$ sudo chown david:family project_dir # david : owner , family : group
```

11.13 members : present all users of the group

```
$ groups
amits adm cdrom sudo dip plugdev lpadmin sambashare
$ members lpadmin
cups-pk-helper amits
$ members sudo
amits david
```

11.14 groupadd : Add user groups to the system

Add user groups to the system. See also: groups, groupdel, groupmod

- Create a new group:
\$ sudo groupadd {{group_name}}
 - Create a new system group:
\$ sudo groupadd --system {{group_name}}
 - Create a new group with the specific groupid:
\$ sudo groupadd --gid {{id}} {{group_name}}
- ```
$ sudo groupadd family
$ sudo usermod -aG family amits
$ groups amits
amits : amits adm cdrom sudo dip plugdev lpadmin sambashare family
$ groups
amits adm cdrom sudo dip plugdev lpadmin sambashare
```

## 11.15 passwd : change password

Change password for current user :  
passwd

## 11.16 whoami : currently logged-in user

allows users to see the currently logged-in user.

## 11.17 su : "Substitute User"

The 'su' command, short for 'substitute user', is a simple yet powerful command in Linux. It allows you to switch to another user account on your system, right from the command line. This can be particularly useful for system administrators who need to perform tasks on behalf of other users.

To enter David's user account including David's environment variables:

```
$ su - david
```

enter and exit from user david :

```
$ whoami
amits
$ su david
password:
$ whoami
david
$ exit
$ whoami
amits
```

To connect as root user:

```
$ sudo su - root # or sudo su -.
```

It allows to work without sudo for every command.

Don't forget to run exit.

```
$ sudo su - root
[sudo] password for amit:
#
#
whoami
root
#
```

Note: "#" signs that the user is root.

Note : sudo is not user, sudo is group

## 11.18 exit : exit the shell

```
$ whoami
amits
$ su david
password:
$ whoami
david
$ exit # exit from david
$ whoami
amits
```

## 11.19 usermod : MODifies a USER account/add user to group.

- Change a username:  
    sudo usermod --login {{new\_username}} {{username}}
- Change a user id:  
    sudo usermod --uid {{id}} {{username}}
- Change a user shell:  
    sudo usermod --shell {{path/to/shell}} {{username}}
- Add a user to supplementary groups (mind the lack of whitespace):  
    sudo usermod --append --groups {{group1,group2,...}} {{username}}
- a, --append # the same as --append --groups  
    Add the user to the supplementary group(s).  
    Use only with the -G option (!) : -aG
- G, --groups GROUP1[,GROUP2,...[,GROUPN]]  
    A list of supplementary groups which the user is also a member of.  
    Each group is separated from the next by a comma, with no intervening  
    whitespace. The groups are subject to the same restrictions as  
    the group given with the -g option.
- Change a user home directory:  
    sudo usermod --move-home --home {{path/to/new\_home}} {{username}}

```
$ echo $SHELL # check the shell type
/bin/sh
```

Change the shell type for user "david" with usermod command:

```
$ sudo usermod -s /bin/bash david
```

Add the user "david" to group "sudo":

```
amits $ su - david
Password:
david $ sudo echo hello
```

```

[sudo] password for david:
david is not in the sudoers file. This incident will be reported.
david $ exit
logout
amits $ sudo usermod -aG sudo david
[sudo] password for amits:
amits $ su - david
Password:
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
david $ sudo echo hello # to check that david is sudoer
[sudo] password for david:
hello

$ sudo usermod -aG family david # add user david to group family
$ groups david # check in what groups is david ?
david : david sudo family

```

## 11.20 usermod : add Tal to sudo group and remove tal from sudo group.

```

to add user to sudo group :
amits$ sudo usermod -a -G sudo tal

to remove tal from sudo group:
amits$ sudo usermod -r -G sudo tal

```

```

\subsection{sudo : run command as administrator.}
\begin{verbatim}

```

```

To run a command as administrator (user
"root"), use "sudo <command>".
See "man sudo_root" for details.
To be "sudoer" the user must be added to
the group "sudo".

$ su - david
Password:
$ sudo echo hello
[sudo] password for david:
david is not in the sudoers file. This incident will be reported.
$ exit
logout
amits:Temp_Amit$ sudo usermod -aG sudo david
[sudo] password for amits:

```

```
$ su - david
Password:
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
```

```
david@amits-vostro-3520:~$ sudo echo hello # to check that david is sudoer
[sudo] password for david:
hello
```

## 12 Special Device files

### 12.1 Device..files with special behaviour

These files are related to drivers, placed in /dev directory and are of two types:

- char file
- block file

Read or write to char or block file causes to some system action. These both files are related to writing data and reading data from one place to other place.

But the difference between them is how they read/write data.

Block file:

-----

A block file is a hardware file which read/write data in blocks instead of character by character. This type of files are very much useful when we want to write/read data in bulk fashion. All our disks such are HDD, USB and CDRoms are block devices. This is the reason when we are formatting we consider block size. The write of data is done in asynchronous fashion and it is CPU intensive activity. These devices files are used to store data on real hardware and can be mounted so that we can access the data we written. Have a look at our other post on getting block size of a device.

Block file examples:/dev/loop0, /dev/ram0, /dev/sda1, /dev/sr0.

Character file:

-----

A char file is a hardware file which reads/write data in character by character fashion. Some classic examples are keyboard, mouse, serial printer. If a user use a char file for writing data no other user can use same char file to write data which blocks access to other user.

Character files uses synchronise technic to write data.

Of you observe char files are used for communication purpose and they can not be mounted.

Example character files:

/dev/autofs,



```

/dev/console,
/dev/crash,
/dev/lp0,
/dev/null,
/dev/ppp,
/dev/random,
/dev/tty ,etc
ls -l output for a char file:
$ ls -l /dev/null
crw-rw-rw- 1 root root 1, 3 Sep 21 06:54 /dev/null
$ ls -l /dev/zero
crw-rw-rw- 1 root root 1, 5 Sep 21 06:54 /dev/zero
$ ls -l /dev/random
crw-rw-rw- 1 root root 1, 8 Sep 21 06:54 /dev/random
$ ls -l /dev/urandom
crw-rw-rw- 1 root root 1, 9 Sep 21 06:54 /dev/urandom

```

Note: stdout, stdin and stderr are also placed in /dev directory:  
(fd/0,1,2 - file descriptor 0,1,2; stdout,stdin,stderr are soft links)

```

$ ls -li /dev
482 lrwxrwxrwx 1 root root 15 Feb 20 12:41 stderr -> /proc/self/fd/2
480 lrwxrwxrwx 1 root root 15 Feb 20 12:41 stdin -> /proc/self/fd/0
481 lrwxrwxrwx 1 root root 15 Feb 20 12:41 stdout -> /proc/self/fd/1

```

## 12.2 /dev/null and /dev/zero : data sink files

Data written to the /dev/null and /dev/zero special files is discarded

Reads from /dev/null file always return end of file ^D,  
whereas reads from /dev/zero file always return bytes  
containing zero ('\0' characters with 0x0 ASCII code).

Read 20 zeros in decimal format from /dev/zero file :

```

$ od -tu1 -N20 /dev/zero
00000000 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0000020 0 0 0 0
0000024

```

```

$ od -tu1 -An -N20 /dev/zero # without present byte count
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0

```

See: \$ man null

If you want to sent the output to nowhere, send it to /dev/null:

```
$ echo "this will not be output to any place" > /dev/null
```

If you don't want to see error messages, you can send errors to /dev/null:

```
$ ls -l dir1 dir2
ls: cannot access 'dir2': No such file or directory
dir1:
total 4
-rw-r--r-- 1 amit amit 13 Sep 23 22:51 a.txt
$ ls -l dir1 dir2 2>/dev/null # redirect errors to dev/null
dir1:
total 4
-rw-r--r-- 1 amit amit 13 Sep 23 22:51 a.txt
```

If you want to see only error messages, you can send regular output to /dev/null:

```
$ ls -l dir1 dir2 1>/dev/null
ls: cannot access 'dir2': No such file or directory
```

If we don't want to see messages of any type (error and regular) we can send both outputs to /dev/null:

```
$ ls -l dir1 dir2 1>/dev/null 2>&1
```

## 12.3 /dev/random file - random numbers generator file

Read 5 random numbers from /dev/random file:

Random is quality generator !

```
$ od -tu1 -An -N5 /dev/random
253 85 48 224 101
$ od -tu1 -An -N5 /dev/random
166 2 125 190 224
$ od -tu1 -An -N5 /dev/random
215 185 237 29 144
```

4 byte numbers :

```
$ od -tu4 -An -N4 /dev/random # -tu4 is 4 byte decimal number
1255275280
$ od -tu4 -An -N4 /dev/random # another random number
1409517909
$ od -tu4 -An -N4 /dev/random
1099118913
```

Randomization of /dev/random is of less quality

Randomization of /dev/urandom is of high quality

## 12.4 /dev/full - file for device full simulation

If we try to write to /dev/full the system returns that device is full.

```
$ echo 'hello' > /dev/full
bash: echo: write error: No space left on device
```

## 12.5 /etc/group - file with list of groups

```
$ head /etc/group
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
adm:x:4:syslog,amits
tty:x:5:syslog
disk:x:6:
lp:x:7:
mail:x:8:
news:x:9:
```

```
$ cat /etc/group | grep amits
adm:x:4:syslog,amits
cdrom:x:24:amits
sudo:x:27:amits
dip:x:30:amits
plugdev:x:46:amits
lpadmin:x:120:amits
smbashare:x:131:amits
amits:x:1001:
```

check who has permission to use smbashare (program to share files):

```
$ cat /etc/group | grep smbashare
smbashare:x:136:amit
```

## 12.6 /etc/passwd - info what users are in the system

```
$ head /etc/group
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
adm:x:4:syslog,amits
tty:x:5:syslog
disk:x:6:
lp:x:7:
mail:x:8:
news:x:9:
```

```
$ cat /etc/group | grep amits
adm:x:4:syslog,amits
cdrom:x:24:amits
sudo:x:27:amits
dip:x:30:amits
plugdev:x:46:amits
lpadmin:x:120:amits
smbashare:x:131:amits
```

```
amits:x:1001:
```

check who has permission to use sambashare (program to share files):

```
$ cat /etc/group | grep sambashare
sambashare:x:136:amit
```

Users that are using BASH:

```
$ cat /etc/passwd | grep bash
root:x:0:0:root:/root:/bin/bash
amit:x:1000:1000:amit,,,:/home/amit:/bin/bash
```

## 12.7 /etc/skel - skeleton files for new user

Skeleton files (.bashrc , .profile) for new user are within /etc/skel directory, and can be copied from it manually. But when we copy the file from one user to another user, it will have the user name of previous user !!!

There are two ways to solve this:

- Allow the user to copy files.
- Change the user of the files.

Note: don't run command `cp -r /etc/skel/*` because it will copy also "." And ".." that is not desirable.

So it is preferred to copy files by exact list of its names.

```
$ ls -lai /etc/skel
total 28
9699438 drwxr-xr-x 2 root root 4096 Nov 15 10:14 .
9699329 drwxr-xr-x 154 root root 12288 Feb 20 23:18 ..
9701502 -rw-r--r-- 1 root root 220 Feb 25 2020 .bash_logout
9701503 -rw-r--r-- 1 root root 3771 Feb 25 2020 .bashrc
9701504 -rw-r--r-- 1 root root 807 Feb 25 2020 .profile
```

## 13 Binary files treatment

### 13.1 binary files compare

1. Convert files in hex format and compare as unicode files:

```
$ hexdump file1.bin > file1.hex
$ hexdump file2.bin > file2.hex
$ diff file1.hex file2.hex
```

2. A visual binary diff tool that allows you to compare binary files interactively.

```
$ vbindiff file1.bin file2.bin
```

## 13.2 nm : "Name Mangling" dump symbol table from object files.

[https://en.wikipedia.org/wiki/Nm\\_\(Unix\)](https://en.wikipedia.org/wiki/Nm_(Unix))

- List global (extern) functions in a file (prefixed with T):  
`nm -g {{path/to/file.o}}`
- List only undefined symbols in a file:  
`nm -u {{path/to/file.o}}`
- List all symbols, even debugging symbols:  
`nm -a {{path/to/file.o}}`
- Demangle C++ symbols (make them readable):  
`nm --demangle {{path/to/file.o}}`

Example:

```
amit@amit-vm-mint21:~/C_programs$ nm ex1
000000000000038c r __abi_tag
0000000000004010 B __bss_start
0000000000004010 b completed.0
 w __cxa_finalize@GLIBC_2.2.5
0000000000004000 D __data_start
...
```

Symbol Type Description

|   |                                   |
|---|-----------------------------------|
| A | Global absolute symbol.           |
| a | Local absolute symbol.            |
| B | Global bss symbol.                |
| b | Local bss symbol.                 |
| D | Global data symbol.               |
| d | Local data symbol.                |
| f | Source file name symbol.          |
| L | Global thread-local symbol (TLS). |
| l | Static thread-local symbol (TLS). |
| R | Global read only symbol.          |
| r | Local read only symbol.           |
| T | Global text symbol.               |
| t | Local text symbol.                |
| U | Undefined symbol.                 |

### 13.3 readelf : ELF file info.

display information about ELF files ONLY  
including their metadata

```
readelf [-a|--all]
 [-h|--file-header]
 [-l|--program-headers|--segments]
 [-S|--section-headers|--sections]
 [-g|--section-groups]
 [-t|--section-details]
 [-e|--headers]
 [-s|--syms|--symbols]
```

### 13.4 od : "Octal Damp" - display binary file.

od is a command on various operating systems  
for displaying ("dumping") data in various human-readable  
output formats. The name is an acronym for "octal dump"  
since it defaults to printing in the octal data format.

Display file contents in octal, decimal or hexadecimal  
format. Optionally display the byte offsets and/or  
printable representation for each line.

Default settings: Octal format, 8 bytes per line,

-----

byte offsets in octal, and duplicate lines replaced with \*:

```
$ od tex_file_data_extract
```

```
-tu1 - decimal format
-tx1 - hexadecimal format
-N - limit dump to N bytes:
-v - do not use * to mark line duplicates
```

Decimal:

```
$ od -tu1 a.txt
```

Hexadecimal:

```
- Display file in hexadecimal format (1-byte units),
 and 4 bytes per line:
$ od --format={{x1}} --width={{4}} -v {{path/to/file}}
$ od -tx1 tex_file_data_extract -N 1024
```

Read text file with od to see ASCII codes :

```
$ echo "ABCD" > a.txt
$ cat a.txt
```

```

ABCD
$ od -tu1 a.txt # decimal ASCII values
00000000 65 66 67 68 10
00000005
$ od -tx1 a.txt # 1 Byte (8 bit) hexadecimal ASCII values
00000000 41 42 43 44 0a
00000005
Note : 0x0a is for line feed character
$ od -tx4 a.txt # 4 BYTE (32 BIT) hexadecimal ASCII values
00000000 44434241 0000000a
00000005

```

## 14 Editors, text operations

### 14.1 nano : run text editor.

```

Run text editor
nano ex1.c

```

### 14.2 vim :v improved editor.

```

vim (v improved) , to run :
vim 1.txt

```

2 modes of work :

- Edit (:i to enter insert mode),  
i.e. type ":" and after it "i"
- Commands (esc to enter command mode)
  - > Shift +:e [file] - Opens a [file] that you want to open. Here [file] is the filename that you want to open
  - > Esc, :w - Save the file but do not exit.
  - > Esc, :q! - To quit the file without first saving what you were working on.
  - > Esc, :wq - To save the file and exit from Vim.
  - > Esc, :help - To get help

### 14.3 tilde : intuitive text editor.

```

tilde - an intuitive text editor
for the console/terminal

```

## 15 Install applications

### 15.1 dpkg : Debian PaKaGe install

Command to install debian package (adds it to apt)

- `-i` OR `--install`: Install a package using the `dpkg` command. The command will extract all control files for the specified package, remove any previously installed older instance of the package, and install the new package on our system.
- `-r` OR `--remove`: Remove an installed package from our system. It removes every file belonging to the specific package except the configuration files. This can be seen as the uninstallation option.
- `-P` OR `--purge`: An alternative way to remove an installed package from our system. It completely removes every file belonging to the specific package, including the configuration files. This can be seen as the 'complete uninstallation' option.
- `-l` OR `--list`: Lists all installed packages on your system.
- `-L` OR `--listfiles`: List installed package's file locations.
- `-s` OR `--status`: Shows the status of an installed package.
- `-S` OR `--search`: Search for a pattern in installed packages.

Example of VS Code install:

```
$ sudo dpkg -i code_1.81.1-1691620686_amd64.deb
```

DPKG - s command : shows the status of an installed package:

```
$ dpkg -s code
```

```
sudo dpkg -i *.deb
```

```
sudo dpkg -install *.deb
```

see APT to find differences between APT and DPKG

### 15.2 apt : "Advanced Package Tool"

[https://en.wikipedia.org/wiki/APT\\_\(software\)](https://en.wikipedia.org/wiki/APT_(software))

APT provides a high-level commandline interface for the package management system.

- Update the list of available packages and versions (it's recommended to run this before other apt commands):  

```
sudo apt update
```
- Search for a given package:



- ```
apt search {{package}}
```
- Example : apt search code # for Visual Studio
- Show information for a package:

```
apt show {{package}}
```

Example : apt show code # for Visual Studio
 - Install a package, or update it to the latest available version:

```
sudo apt install {{package}}
```
 - Remove a package (using purge instead also removes its configuration files):

```
sudo apt remove {{package}}
```
 - Upgrade all installed packages to their newest available versions:

```
sudo apt upgrade
```
 - List all packages:

```
apt list
```
 - List installed packages:

```
apt list --installed
```

*****Apt vs. Dpkg*****

<https://www.geekbits.io/what-is-the-difference-between-apt-and-dpkg/>

Apt (Advanced Package Tool) is a newer package manager that offers more features than dpkg. It can automatically handle dependencies, so you don't have to worry about manually installing them. Apt also has a wider range of repositories from which you can install packages.

Dpkg (Debian Package Management System) is the older package manager. It doesn't have as many features as apt, but it's still a powerful tool. Dpkg can't automatically handle dependencies, so you'll need to install them manually. Dpkg also has a limited range of repositories, but you can still find the most common packages.

Apt is a higher-level tool that provides a more straightforward interface to work with than dpkg. While dpkg can only install, remove, or query individual packages, apt offers additional functionality, such as automatically handling dependencies, retrieving and installing packages from remote locations, and upgrading all installed packages to their latest versions.

Additionally, apt offers several useful commands

for managing repositories and upgrading the system.
For example, the apt-get command can install or remove packages,
while the apt-cache command can search for available packages.

Another difference is that dpkg is primarily a command-line tool,
while apt also provides a graphical user interface (GUI)
in the form of the aptitude tool.

Which Package manager should you use?

So, which package manager should you use? It depends on your needs.
If you want a more powerful tool with automatic dependency management,
then apt is the way to go.

On the other hand, if you want a more straightforward package manager
or if you need to install packages from a specific repository,
then dpkg is the better choice.

Overall, apt (Recommended tool) is a more user-friendly and robust
option than dpkg, making it the preferred package manager
for most users. It provides a simpler and more automated way to
manage packages on your system.

However, dpkg can still be helpful in certain situations,
such as when working with old or custom packages
that are unavailable through apt.

16 GCC and binary files operations

17 Ethernet

17.1 Wireshark install

<https://askubuntu.com/questions/700712/how-to-install-wireshark>

1.Add the stable official PPA. To do this, go to terminal by pressing Ctrl+Alt+T
and run:

```
sudo add-apt-repository ppa:wireshark-dev/stable
```

2.Update the repository:

```
sudo apt-get update
```

3.Install wireshark 2.0:

```
sudo apt-get install wireshark
```

4.Run wireshark:

```
sudo wireshark
```

5.If you get an error

"couldn't run /usr/bin/dumpcap in child process: Permission Denied", go to the terminal again and run:

```
sudo dpkg-reconfigure wireshark-common
```

Say YES to the message box. This adds a wireshark group. Then add user to the group by typing

```
sudo adduser $USER wireshark
```

Then restart your machine and open Wireshark. It works. Good Luck.

17.2 packet.names.at.different.levels.of.TCP/IP

packet names at different levels of TCP/IP

application		message
transport		segment
network		datagram
link		frame

17.3 TCP.vs.UDP

TCP is Transmission Control Protocol

UDP is User Datagram Protocol

TCP use processes to check that the packets are correctly sent to the receiver.

UDP send packets without any checks but with the advantage of being quicker than TCP.

17.4 sysctl : change kernel Networking params

All TCP/IP tuning parameters are located under /proc/sys/net/

important tuning parameters:

/proc/sys/net/core/rmem_max - Maximum TCP Receive Window

/proc/sys/net/core/wmem_max - Maximum TCP Send Window

/proc/sys/net/ipv4/tcp_rmem - memory reserved for TCP receive buffers

/proc/sys/net/ipv4/tcp_wmem - memory reserved for TCP send buffers

we dont change the above parameters directly,

sysctl command is used to modify kernel parameters at runtime.

The parameters available are those listed under `/proc/sys/`.

Example for modifying kernel receiver buffer to 16Mbyte:

```
$ sysctl -q -w net.core.rmem_max = 16777216
$ sysctl -q -w net.core.rmem_default=16777216

-q : quiet
-w : write
```

17.5 IP..netmask..explanation

Four basic elements are combined to make an Ethernet system

1. The FRAME, which is a standardized set of bits used to carry data over the system.
2. The MEDIA ACCESS CONTROL protocol, which defines how multiple computers access the shared Ethernet channel in a fair manner.
3. The SIGNALING COMPONENTS, which consists of standardized electronic devices that send and receive signals over an Ethernet channel.
4. The PHYSICAL MEDIUM, which consists of the cables and other hardware used to carry the digital Ethernet signals between computers attached to the network.

The Ethernet frame (MAC Frame)

```
-----
7 bytes  Preamble (for synchronizing)
1 byte   SFD      (Start Frame Delimeter)
6 bytes  DESTINATION ADDRESS (MAC address)
6 bytes  SOURCE ADDRESS
2 bytes  LENGTH/TYPE (frame type or frame length)
```

Data field:

```
46 TO 1500 or 1504  DSAP and SSAP (Destination and Source
or 1982 bytes       Service Access Point)+CTRL (1 byte each)
```

```
4 bytes FRAME CHECK SEQUENCE (CRC 32)
```

The Media Access Control protocol

```
-----
MAC protocol defines how the physical medium is shared among
stations connected.
```

Defines how multiple computers access the shared Ethernet channel in a fair manner.

Ethernet-equipped computer operates independently of all other stations on the network; there is no central controller

Ethernet uses a broadcast delivery mechanism, in which each frame that is transmitted is heard by every station.

Every frame sent has an Destination Address and Source Address

MAC address (true HW address, used for switching):

MAC refers to a unique hardware address assigned to network devices for identification at the data link layer.

Every Ethernet Station has a unique 48 bit MAC Address (6 bytes).

IP address (can be changed, used for routing):

Two computers with different MAC addresses can have the same IP address . Also for example, in case when we change HW adapter in the same computer

This use of m.a.c. address rise some problems :

1. there are network hw adapters that allow to put different m.a.c. address.
2. In the case when in the network one of network adapters is damaged and replaced by new one (for ex. fix the adaptor problem).

That is why defined IP (Internet Protocol), and IP address that was defined by Internet Protocol. There are IP addresses ver. 4 and ver. 6 (4 is more common).

Internet protocol ---> IP ----> IP address

IP address consists of two parts : (subnet number)&(Host number)

10.0.0.100 --- 10.0.0.101----			-----10.0.1.100 ----
	10.0.0.1--	10.0.1.1	
	GW	GW	
10.0.0.102 ----10.0.0.103----			-----10.0.1.101-----
Subnet mask : 255.255.255.0			255.255.255.0
10.0.0.*			10.0.1.*

GW : Gate Way (Router)

IP address ver. 4 consists of 4 bytes:

0-255		0-255		0-255		0-255
192	.	168	.	0	.	1

Subnet mask number

netmask/subnet number helps to computer to understand if the package will be send inside the subnet or directly to GW router to pass the message outside of subnet.

We would like to identify between local networks. For this purpose network mask (netmask) is definid.

We tell that some bits of the IP number present the network number - netmask number is used for this purpose.

The bits in netmask number that are equal to 1 define area field of network number.

The bits in netmask number that are equal to 0 define area field of computer number within local network.

If the computer 10.0.0.100 wants to send message to computer 10.0.0.103 - it knows that it must send the message in local network 10.0.0 and there is no need to pass the message out of the local network 10.0.0. Enough that 10.0.0.100 will send broadcast to all the computers in its local network 10.0.0...until it will get the answer from 10.0.0.103.

But if the computer 10.0.0.100 will try to send the message to 10.0.1.101...Because it knows that network 10.0.1 is not its own network, it will send the request to Gate Way computer 10.0.0.1. Gate Way computer 10.0.0.1 sends the message to Gate Way computer 10.0.1.1 and the Gate Way of 10.0.1 will send broadcast on its local network 10.0.1.

After number of hopes the message from 1.0.0.100 will arrive to computer 1.0.1.101 At home network we connected by WiFi to router that si the GW of local home newtwork. The router is connected to Internet provider... and so on , until the message arrives its target.

In the following example the network number can be 24 bit and the computer number can be 8 bits.

10	.	0	.	0	.	100	IP address
255	.	255	.	255	.	0	Subnet mask
Network number Computer Number							255.255.255.0 <--> /24
(subnet mask) (Host number)							

| Num. of computers that can be connected:
| $2^8 = 256$
| 255 is for broadcast
| 1 is for GW

10	.	0	.	0	.	100	IP address
255	.	255	.	0	.	0	Subnet mask
Network number Computer Number							255.255.0.0 <--> /16

```

(subnet mask) | (Host number)
-----
| Num. of computers that can be connected:
|           2^16 = 65000
| 255 is for broadcast
| 1   is for GW
|
10 . 0 . 0 . 100          IP address
255 . 240 . 0 . 0        Subnet mask
Network number | Computer Number    255.240.0.0 <--> /12
-----
| Num. of computers: 2^20 = 1048576
| 255 is for broadcast
| 1   is for GW
240d=11110000b

```

Loopback channel

```

-----
Intended for inter process communication - IPC:
In some computers 127.0.0.1 is the same computer (for loopback
perform), in other computers it is address 0.0.0.0 (for loopback
perform). We can take 2 programs in the same computer, without common
memory and send messages from one program to another (inter process
communication - IPC).

```

IF one of computers has the same IP than another computer in the local network, we can change of IP and netmask of one of computers with ifconfig command with following options :

```

$ ifconfig eth0 192.168.0.102          # only IP number chagne
$ ifconfig eth0 192.168.0.102 255.255.255.0 # IP + Mask
$ ifconfig eth0 192.168.0.102 /24 # define how many bits takes
                                   # Network address

```

An additional example:

```

$ ifconfig wifi0 10.0.0.100 255.255.0.0
$ ifconfig wifi0 10.0.0.100 /16  # define how many bits takes
                                   # Network address

```

17.6 DHCP : Dynamic Host Configuration Protocol

https://en.wikipedia.org/wiki/Dynamic_Host_Configuration_Protocol

DHCP server runs within router (Gate Way).

DHCP client runs within computer in the same subnet as router.

DHCP server manage internal table with all computers in subnet:

Leas time helps to manage time given to computer to have some IP.

MAC addr.	IP addr	Time
a1-a1-a1-b1-c2-d4	10.0.0.100	~~~~~
b2-c4-d2-12-28-aa	10.0.0.101	~~~~~

We can use DHCP (Dynamic Host Configuration Protocol) to define IP numbers automatically.

DHCP stands for Dynamic Host Configuration Protocol.

It is a protocol that allows devices on a network (DHCP clients) to get an IP address and other network settings automatically from a DHCP server. This makes it easier to connect multiple devices to a network without manually configuring each one.

The DHCP protocol also provides a mechanism whereby a client can learn important details about the network to which it is attached, such as the location of a default router (GW), the location of a name server, and so on.

Some of the benefits of using DHCP are:

- It reduces the chance of IP address conflicts, where two devices have the same IP address.
- It simplifies network management, as the DHCP server can assign and reclaim IP addresses centrally.
- It supports mobility, as devices can move from one network to another and get a new IP address automatically.

Some of the drawbacks of using DHCP are:

- It depends on the availability and reliability of the DHCP server. If the server is down or unreachable, devices cannot get an IP address or renew their lease.
- It may introduce security risks, as malicious users can spoof DHCP messages or set up rogue DHCP servers to intercept or redirect network traffic.
- It may not be suitable for some devices that need a fixed or static IP address, such as servers or printers.

Router usually is used as DHCP server.

He is also Gate Way.

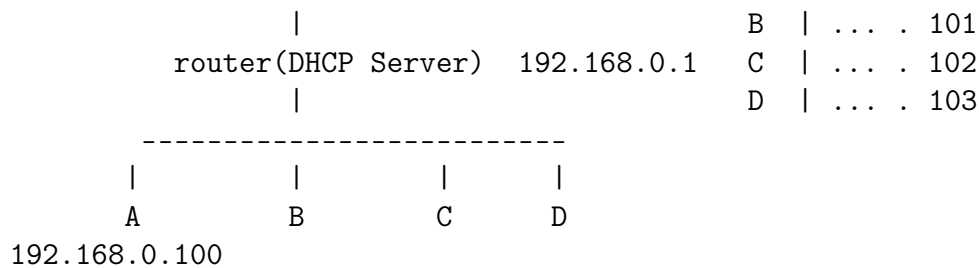
It's address is usually constant : 192.168.0.1.

Operation systems are familiar with such addresses.

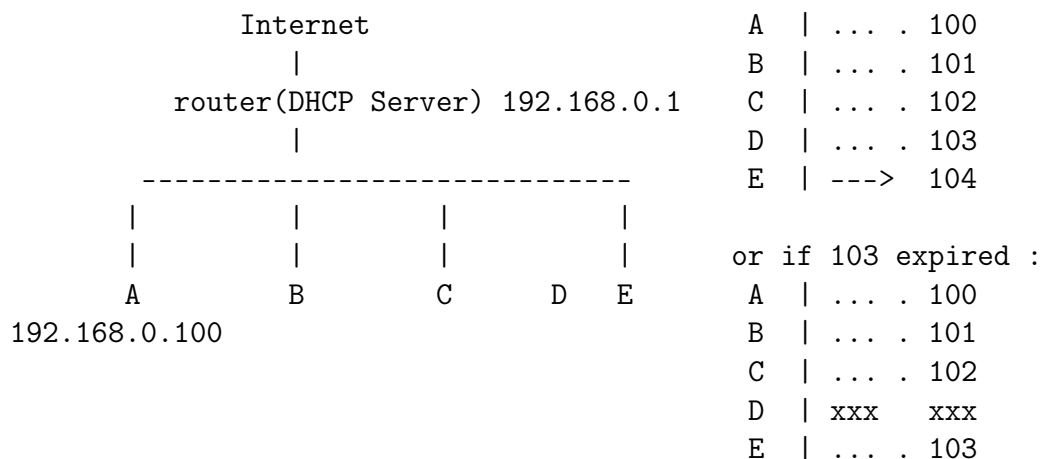
Computer that is turned on for the first time in the network, send messages to check which computer is GateWay.

After it finds it , the computer ask the G.W. router to get IP address (this operation is calles IP lease).

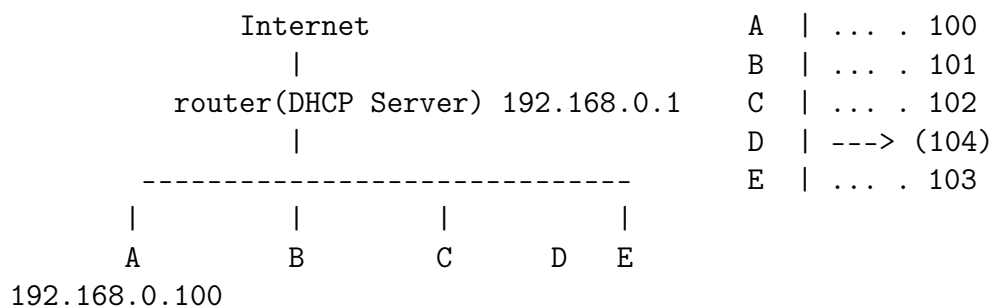
The router manages internally the table of mac address, IP , expiration date of IP , etc.



Now computer D is disconnected and in place of D connected new computer E. If address of D is not expired, router will set address of E ...104, but if address of D is expired, then the router can set the address of E to be ...103.



But now the computer D connected again and D asks from DHCP server to refresh its IP address. Now DHCP server will set its address to ...104:



If the address ...103 in any case is important for D, it can ask E computer to release its address, to allow ifconfig, in order not to make collision with existing IP. E is calling the router and ask him to release the IP address. In this case D can also make release to get the address. Additional option is to define IP address per m.a.c. address within router (i.e. edit routing table within the router).

DHCP protocol

CLIENT SERVER

discover
-----> (1)

offer
<----- (2)

request
-----> (3)

acknowledge
<----- (4)

DHCP operations fall into four phases:

- server discovery,
- IP lease offer,
- IP lease request,
- IP lease acknowledgement.

These stages are often abbreviated as DORA for
Discovery, Offer, Request, and Acknowledgement.

Discovery

The DHCP client broadcasts a DHCPDISCOVER message on the network subnet using the destination address 255.255.255.255 (limited broadcast) or the specific subnet broadcast address (directed broadcast). A DHCP client may also request an IP address in the DHCPDISCOVER, which the server may take into account when selecting an address to offer.

Offer

When a DHCP server receives a DHCPDISCOVER message from a client, which is an IP address lease request, the DHCP server reserves an IP address for the client and makes a lease offer by sending a DHCPOFFER message to the client. This message contains the client's client id (traditionally a MAC address), the IP address that the server is offering, the subnet mask, the lease duration, and the IP address of the DHCP server making the offer.

Request

In response to the DHCP offer, the client replies with a DHCPREQUEST message, broadcast to the server, requesting the offered address. A client can receive DHCP offers from multiple servers, but it will accept only one DHCP offer.

Acknowledgement

When the DHCP server receives the DHCPREQUEST message from the client, the configuration process enters its final phase. The acknowledgement phase involves sending a DHCPACK packet to the client. This packet includes the lease duration and any other configuration information that the client might have requested. At this point, the IP

configuration process is completed.

17.7 dhclient : obtain/release the IP address from DHCP server

DHCP server runs within router (Gate Way).

DHCP client runs within computer in the same subnet as router.

- Get an IP address for the eth0 interface:

```
$ sudo dhclient {{eth0}}
```

- Release an IP address for the eth0 interface:

```
$ sudo dhclient -r {{eth0}}
```

To obtain fresh IP address from DHCP server (router (Gate Way)):

```
$ sudo dhclient enp0s3
```

To release IP address within DHCP server:

```
$ sudo dhclient -r enp0s3
```

Once current lease has been released, the client exits.

Example:

```
$ ip address
```

```
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000 link/ether 08:00:27:78:7a:16 brd ff:ff:ff:ff:ff:ff inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic
```

```
$ sudo dhclient -r enp0s3
```

```
$ ip address
```

```
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000 link/ether 08:00:27:78:7a:16 brd ff:ff:ff:ff:ff:ff inet6 fe80::e013:b6a6:6af3:9b07/64 scope link noprefixroute
```

```
$ sudo dhclient enp0s3
```

```
$ ip address
```

```
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000 link/ether 08:00:27:78:7a:16 brd ff:ff:ff:ff:ff:ff inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic
```

17.8 SSH : OpenSSH remote login client

ssh (SSH client) is a program for logging into a remote machine and for executing commands on a remote machine. It is intended to provide secure encrypted communications between two

untrusted hosts over an insecure network.

Used to access a remote computer on most Linux/Unix systems (write shell commands just as if you were sitting at the workstations) just like telnet.

SSH is Secure SHell (connection to remote computer),
SCP is Secure CoPy (copy to remote computer).
SSH allows connect another Linux computer in safety manner.
SCP allows copy files from one computer to another computer in safety manner.

1. check if ssh exists on the computer:
\$ ssh -V
OpenSSH_8.9p1 Ubuntu-3ubuntu0.6, OpenSSL 3.0.2 15 Mar 2022
2. If not exists , install SSH server (daemon):
\$ sudo apt update
\$ sudo apt install openssh-server
3. run SSH server on both computers:
\$ sudo systemctl start ssh #start ssh for this session
or
\$ sudo systemctl enable ssh #start ssh from boot to computer
4. connect to computer :
\$ ssh user_name@computer_name
or
\$ ssh user_name@computer_ip_address

Example :

```
$ ssh amit@192.168.1.12
The authenticity of host '192.168.1.12 (192.168.1.12)'
can't be established.
ED25519 key fingerprint is SHA256:P5+OF1k8R/kSq4BSPgypkbda/
o4XhXsdgHjVtvhuz5c.
This key is not known by any other names
Are you sure you want to continue connecting
(yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.12' (ED25519)
to the list of known hosts.
amit@192.168.1.12's password:
Last login: Sun Oct 29 06:17:17 2023 from 192.168.1.18
```

```
amit@amit-vm-mint21:~$
amit@amit-vm-mint21:~$
amit@amit-vm-mint21:~$
```

Note1 : \$USER NAME is amit,
\$HOSTNAME is amit-vm-mint21

Note2 : SSH uses computer fingerprint to recognize the remote computer.

There are two commands to operate SSH : systemctl and service

SSH operate commands:

```
$ sudo systemctl status ssh #check the status
$ sudo systemctl stop ssh   #stop ssh for this session
$ sudo systemctl start ssh  #start ssh for this session
$ sudo systemctl disable ssh #stop ssh from boot to computer
$ sudo systemctl enable ssh  #start ssh from boot to computer
```

```
$ sudo systemctl status ssh
[sudo] password for amits:
o ssh.service - OpenBSD Secure Shell server
Loaded: loaded (/lib/systemd/system/ssh.service; disabled;
       vendor preset: enabled)
Active: inactive (dead)
Docs: man:sshd(8)
      man:sshd_config(5)
```

Virtual box settings to access external computer network :

To connect virtual machine to external network we must choose
bridge adapter (it connects virtual machine to home network).
Choose:

```
Settings,
Network,
Attached to : Bridged adapter
Name : MedaTek Wifi 6 M7921 Wireless LAN Card
```

Note: you must be connected to Internet in outer computer !

Check if network connection exists:

```
$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 192.168.1.12 netmask 255.255.255.0 broadcast 192.168.1.255
```

Note: NAT network in virtual box is internal network between
different machines inside the virtual box. If we define NAT network for
two machines, we can run ssh and scp commands on both virtual computers.
May there is needed to run

```
$ dhclient enp0s3 # in order to get IP address
or
```

```
$ ip address add
```

17.9 fail2ban : security intrusion prevention software framework

<https://en.wikipedia.org/wiki/Fail2ban>

Fail2ban is an intrusion prevention software framework.
Written in the Python programming language, it is designed to prevent brute-force attacks.

17.10 who : display who is logged in

- Display the username, line, and time of all currently logged-in sessions:

```
$ who
```

Example:

```
$ who
amit      tty7          2024-03-10 07:17 (:0)
amit      pts/1         2024-03-10 17:37 (192.168.1.18)
```

17.11 SCP : OpenSSH secure file copy

scp - remote file copy program, copies files between hosts on a network

SCP is Secure CoPy (copy to remote computer).
SCP allows copy files from one computer to another computer in safety manner.

uses ssh for data transfer authentication and provides the same security as SSH does.

Copies between two remote hosts are permitted also !

1. check if ssh exists on the computer:

```
$ ssh -V
```

```
OpenSSH_8.9p1 Ubuntu-3ubuntu0.6, OpenSSL 3.0.2 15 Mar 2022
```

2. If not exists , install SSH server (daemon):

```
$ sudo apt update
```

```
$ sudo apt install openssh-server
```

3. run SSH server on both computers:

```
$ sudo systemctl start ssh #start ssh for this session
```

or

```
$ sudo systemctl enable ssh #start ssh from boot to computer
```

4. Copy a local file to a remote host:

```
scp {{path/to/local_file}} {{remote_host}}:{{path/to/remote_file}}
```

Copy a file from a remote host to a local directory:

```
scp {{remote_host}}:{{path/to/remote_file}} {{path/to/
                                local_directory}}
    Recursively copy the contents of a directory from a remote
    host to a local directory:
scp -r {{remote_host}}:{{path/to/remote_directory}} {{path/to/
                                local_directory}}
```

Example:

```
$ scp p_data.txt amit@192.168.1.12:/home/amit/Documents
    amit@192.168.1.12's password:
    p_data.txt                100% 648   126.7KB/s   00:00
```

Recursive copy of folders:

```
$ scp -r /home/amits/Documents/RTC
                                amit@192.168.1.12:/home/amit/Documents/RTC
```

Windows :

```
$ pscp.exe bennyc@192.168.42.38:/home/minicom.log c:
```

17.12 pscp : transfer files between Windows and linux

The open source PSCP utility makes it easy to transfer files and folders between Windows and Linux computers

Using PSCP

PSCP (PuTTY Secure Copy Protocol) is a command-line tool for transferring files and folders from a Windows computer to a Linux computer.

Download pscp.exe from its website.

<https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html>

Move pscp.exe to a folder in your PATH (for example, Desktop\App if you followed the PATH tutorial here on Opensource.com). If you haven't set a PATH variable for yourself, you can alternately move pscp.exe to the folder holding the files you're going to transfer.

Open Powershell on your Windows computer using the search bar in the Windows taskbar (type 'powershell' into the search bar.)

Type pscp -version to confirm that your computer can find the command.

Using PSCP

PSCP (PuTTY Secure Copy Protocol) is a command-line tool for transferring files and folders from a Windows computer to a Linux computer.

Download pscp.exe from its website.

Move pscp.exe to a folder in your PATH (for example, Desktop\App if you followed the PATH tutorial here on Opensource.com). If you haven't set a PATH variable for yourself, you can alternately move pscp.exe to the folder holding the files you're going to transfer.

Open Powershell on your Windows computer using the search bar in the Windows taskbar (type 'powershell' into the search bar.)

Type pscp -version to confirm that your computer can find the command.

17.13 ping : check connection

This is convenient way to check that the opposite side is connected and supports ping protocol.

It sends packets to given machine and get acknowledgment packets in return.

If you can ping GW (router) - your network interface is OK
If you is able to ping an external IP address, your network settings are correct.

ping also evaluates time and hops that takes to the command to arrive the opposite side.

Ask ping to check google dns site 3 times only :

```
$ ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
 64 bytes from 8.8.8.8: icmp_seq=1 ttl=119 time=12.2 ms
 64 bytes from 8.8.8.8: icmp_seq=2 ttl=119 time=13.5 ms
 64 bytes from 8.8.8.8: icmp_seq=3 ttl=119 time=13.8 ms
```

--- 8.8.8.8 ping statistics ---

3 packets transmitted, 3 received, 0% packet loss, time 2003ms

rtt min/avg/max/mdev = 12.215/13.174/13.791/0.687 ms

--- 8.8.8.8 ping statistics ---

8 packets transmitted, 8 received, 0% packet loss, time 7008ms

rtt min/avg/max/mdev = 12.861/14.049/15.676/0.881 ms

TTL : how many hops it can made maximum !!!

TTL : time to leave

<https://www.cloudflare.com/en-gb/learning/cdn/glossary/time-to-live-ttl/>

Time to live (TTL) refers to the amount of time or ‘hops’ that a packet is SET to exist inside a network BEFORE being DISCARDED BY A ROUTER.

Time To Leave:

How many jumps(hops) the packet maximum will pass until the next router will throw it.

to set TTL:

```
$ ping rt-ed.co.il -t 50 # set TTL=50
```

In following picture : rectangular is computer and circle is router. We want to send message from one computer to another. Firstly the message arrives to 1-st router, it must to calculate the best rout from it to second computer.

In the well case the data arrives the destination :

```
[]---> 0 ----> 0 ----> 0 ----> []
```

In case of fault the packet can enter the infinite loop between routers. It can cause traffic jam. In order to prevent such situation Internet uses TTL - time to live, how time the packet can hop maximum between different routers.

```
[]---> 0 <---- 0 xxx 0 xxx []  
      |      ^  
      |      |  
      v      |  
      0-----> 0
```

When some packet is in closed loop it can cause to conjesction it the net, when other packets arrive.

To prevent conjesction , defined TTL.

17.14 iperf : measure quality and bandwidth of link

iperf is a tool to measure the bandwidth and the quality of a network link.

The network link is delimited by two hosts running iperf.

The quality of a link can be tested as follows:

- Latency (response time or RTT): can be measured with the Ping command.
- Jitter (latency variation): can be measured with an iperf UDP test.
- Datagram loss: can be measured with an iperf UDP test.

On the first computer (with IP 192.168.1.18) we run the server :

```
$ iperf -s
```

```
-----  
Server listening on TCP port 5001
```

```
TCP window size: 128 KByte (default)  
-----
```

```
[ 1] local 192.168.1.18 port 5001 connected with 192.168.1.12 port 38950  
[ ID] Interval      Transfer      Bandwidth  
[ 1] 0.0000-10.2928 sec 55.5 MBytes 45.2 Mbits/sec
```

On the second computer we run the client , enter the IP address of the first computer :

```
$ iperf -c 192.168.1.18
```

```
-----  
Client connected to 192.168.1.18, TCP port 5001
```

```
TCP window size: 85 KByte (default)  
-----
```

```
[ 1] local 192.168.1.12 port 38950 connected with 192.168.1.18 port 5001  
[ ID] Interval      Transfer      Bandwidth  
[ 1] 0.0000-10.2928 sec 55.5 MBytes 45.2 Mbits/sec
```

Send ctrl+C signal to stop the iperf server.

To run the server in UDP mode :

```
$ iperf -s -u
```

Change Network parameters and run iperf server :

```
$ sysctl -q -w net.core.rmem_max=16777216
```

```
$ sysctl -q -w net.core.rmem_default=16777216
```

```
$ iperf -s -u
```

Run client in UDP aith defined size :

```
iperf -c 2.2.2.11 -u -b 2G
```

17.15 TTL : Time To Leave

In case of fault the packet can enter the infinite loop between routers. It can cause traffic jam. In order to prevent such situation Internet uses TTL - time to live, how time the packet can hop maximum between different routers.

```

[]--> 0 <---- 0 xxx 0 xxx []
      |       ^
      |       |
      v       |
      0-----> 0

```

When some packet is in closed loop it can cause to congestion in the net, when other packets arrive.
To prevent congestion, defined TTL.

Time To Leave:

How many jumps(hops) the packet maximum will pass until the next router will throw it.

to set TTL:

```
$ ping rt-ed.co.il -t 50 # set TTL=50
```

<https://www.cloudflare.com/en-gb/learning/cdn/glossary/time-to-live-ttl/>

Time to live (TTL) refers to the amount of time or "hops" that a packet is SET to exist inside a network BEFORE being DISCARDED BY A ROUTER.

check Google dns server :

```
$ ping 8.8.8.8
```

```

PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
 64 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=16.4 ms
 64 bytes from 8.8.8.8: icmp_seq=2 ttl=118 time=15.5 ms
 64 bytes from 8.8.8.8: icmp_seq=3 ttl=118 time=14.4 ms
 64 bytes from 8.8.8.8: icmp_seq=4 ttl=118 time=14.4 ms

```

17.16 Gate Way : Time To Leave

GW is router/computer that is used to connect us out of our local network.

17.17 traceroute : shows hops(jumps) between devices

Print the route of packets trace to network host

```
$ traceroute 192.168.1.12
```

```
traceroute to 192.168.1.12 (192.168.1.12), 30 hops max, 60 byte packets
```

```
1 192.168.1.12 (192.168.1.12) 11.090 ms 11.044 ms 11.284 ms
```

```
$ traceroute rt-ed.co.il
```

```
traceroute to rt-ed.co.il (212.199.114.183), 30 hops max, 60 byte packets
```

```
1 InnboxG84 (192.168.1.1) 6.118 ms 6.123 ms 6.462 ms
```

```
2 100.64.32.1 (100.64.32.1) 9.977 ms 11.990 ms 12.110 ms
```

```
3 45.80.88.197 (45.80.88.197) 12.803 ms 12.791 ms 12.779 ms
```

```
4 82.102.142.225 (82.102.142.225) 36.986 ms 37.342 ms 37.331 ms
```

```
5 212.116.177.49.static.012.net.il (212.116.177.49) 25.972 ms 25.961 ms  
25.950 ms
```

```
6 212.116.177.50.static.012.net.il (212.116.177.50) 20.347 ms 9.816 ms  
9.730 ms
```

```
7 * * *
```

```
8 * * *
```

```
9 * * *
```

```
10 * * *
```

```
$ traceroute google.com
```

```
traceroute to google.com (216.239.38.120), 30 hops max, 60 byte packets
```

```
1 InnboxG84 (192.168.1.1) 3.762 ms 3.687 ms 3.808 ms
```

```
2 100.64.32.1 (100.64.32.1) 10.539 ms 14.998 ms 15.167 ms
```

```
3 45.80.88.197 (45.80.88.197) 18.678 ms 18.653 ms 18.617 ms
```

```
4 72.14.221.92 (72.14.221.92) 18.128 ms 18.109 ms 17.911 ms
```

```
5 * * *
```

```
6 any-in-2678.1e100.net (216.239.38.120) 17.687 ms 11.035 ms 10.995 ms
```

17.18 route : set/display the routing table

Use route cmd to set the route table

This command allows us to define the default Gate Way IP .

- Display the information of route table:

```
$ route -n
```

or

```
$ route
```

- Add route rule:

```
$ sudo route add -net {{ip_addr}} netmask {{netmask_addr}} gw {{gw_addr}}
```

- Delete route rule:

```
$ sudo route del -net {{ip_addr}} netmask {{netmask_addr}} dev {{gw_addr}}
```

```
$ route
Kernel IP routing table
Destination      Gateway          Genmask          Flags Metric Ref    Use Iface
default          InnboxG84       0.0.0.0          UG      600    0      0 wlp0s20f3
link-local       0.0.0.0         255.255.0.0      U       1000   0      0 wlp0s20f3
192.168.1.0      0.0.0.0         255.255.255.0    U       600    0      0 wlp0s20f3
```

```
$ route -n
Kernel IP routing table
Destination      Gateway          Genmask          Flags Metric Ref    Use Iface
0.0.0.0          192.168.1.1     0.0.0.0          UG      600    0      0 wlp0s20f3
169.254.0.0      0.0.0.0         255.255.0.0      U       1000   0      0 wlp0s20f3
192.168.1.0      0.0.0.0         255.255.255.0    U       600    0      0 wlp0s20f3
```

To define default GW :

```
$ sudo route add default gw 192.168.1.0
SIOCADDRT: Network is unreachable
```

17.19 ip/ip route : ip command

ip - show / manipulate routing, network devices,
interfaces and tunnels

```
ip [ OPTIONS ] OBJECT { COMMAND | help }
```

```
OBJECT := { link | address | addrlabel | route | rule | neigh |
           table | tunnel | tuntap | maddress | mroute
           | mrule | monitor | xfrm | netns | l2tp | tcp_metrics |
           token | macsec | vrf | mptcp | ioam }
```

-Similar but newer than ifconfig command.
display network status :

```
$ ifconfig
or
$ ip address show #
$ ip addr show   # short form
```

- Make an interface up/down:
\$ ip link set enp0s3 up # enable enp0s3 interface
\$ ip link set enp0s3 down # disable enp0s3 interface

```
or
$ ifconfig enp0s3 up    # enable enp0s3 interface
$ ifconfig enp0s3 down # disable enp0s3 interface
```

- Set IP address of eth0:

```
$ ifconfig eth0 10.0.0.100
```

or

```
$ ip address add 10.0.0.100 dev eth0
```

- Change IP of eth0 to 10.0.0.100
and netmask to 255.255.255.0

```
$ ifconfig eth0 10.0.0.100 netmask 255.255.255.0
```

or

```
$ ifconfig eth0 10.0.0.100/24
```

or

```
$ ip address add 10.0.0.100/24 dev eth0
```

- Change default gateway to 10.0.0.1 for eth0 :

```
$ route add default gw 10.0.0.1
```

```
$ ip route add default via 10.0.0.1 dev eth0
```

Show/manipulate routing, devices, policy routing and tunnels

- List interfaces with detailed info:

```
$ ip address
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
   group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp1s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc
   fq_codel state DOWN group default qlen 1000
   link/ether 04:bf:1b:72:b9:5d brd ff:ff:ff:ff:ff:ff
3: wlp0s20f3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc
   noqueue state UP group default qlen 1000
   link/ether f4:6d:3f:8e:03:dc brd ff:ff:ff:ff:ff:ff
   inet 192.168.1.18/24 brd 192.168.1.255 scope global dynamic
       noprefixroute wlp0s20f3
       valid_lft 74418sec preferred_lft 74418sec
   inet6 fe80::dad7:1350:f3b3:451d/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

- Display the routing table:

```
$ ip route
```

- Show neighbors (ARP table):

```
$ ip neighbour
```

This command can be used to display or modify the existing IP routing table. We can add, delete, or modify specific static routes to specific hosts or networks using ip route command

- Display the routing table:

```
$ ip route {{show|list}}
```

- Add a default route using gateway forwarding:

```
$ sudo ip route add default via {{gateway_ip}}
```

- Add a default route using eth0:

```
$ sudo ip route add default dev {{eth0}}
```

- Add a static route:

```
$ sudo ip route add {{destination_ip}} via {{gateway_ip}} dev {{eth0}}
```

Example :

```
$ sudo ip route add default via 192.168.7.3 dev enp1s0
```

- Delete a static route:

```
$ sudo ip route del {{destination_ip}} dev {{eth0}}
```

- Change or replace a static route:

```
$ sudo ip route {{change|replace}} {{destination_ip}} via {{gateway_ip}} dev {
```

```
$ route -n
```

Kernel IP routing table

Destination	Gateway	Genmask	Flags	Metric	Ref
Use Iface					
0.0.0.0	192.168.1.1	0.0.0.0	UG	600	0
0 wlp0s20f3					
169.254.0.0	0.0.0.0	255.255.0.0	U	1000	0
0 wlp0s20f3					
192.168.1.0	0.0.0.0	255.255.255.0	U	600	0
0 wlp0s20f3					

```
$ ip route
```

```
default via 192.168.1.1 dev wlp0s20f3 proto dhcp metric 600
```

```
169.254.0.0/16 dev wlp0s20f3 scope link metric 1000
```

```
192.168.1.0/24 dev wlp0s20f3 proto kernel scope link src 192.168.1.18  
metric 600
```

17.20 DNS..server

<https://www.cloudflare.com/en-gb/learning/dns/what-is-a-dns-server/>

Your programs need to know what IP address corresponds to a given host name (ex.)

Domain Name Servers (DNS) take care of this.

You just have to specify the IP address of 1 or more DNS servers in your `/etc/resolv.conf` file:

```
nameserver 217.19.192.132
```

```
nameserver 212.27.32.177
```

The changes takes effect immediately

DNS (Domain Name Server) server is used to translate internet site names (host name) to IP address.

IP addresses of DNS servers are placed in `/etc/resolv.conf` file.

To add Google DNS server see following :

<https://easyengine.io/tutorials/linux/google-public-dns/>

The Domain Name System (DNS) is the phonebook of the Internet.

When users type domain names such as 'google.com' or 'nytimes.com' into web browsers, DNS is responsible for finding the correct IP address for those sites.

Browsers then use those IP addresses to communicate with origin servers or CDN edge servers to access website information. This all happens thanks to DNS servers: machines dedicated to answering DNS queries.

- Configure your network settings to use the IP addresses 8.8.8.8 and 8.8.4.4 as your DNS servers.

check Google dns server :

```
$ ping 8.8.8.8
```

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
```

```
64 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=16.4 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=2 ttl=118 time=15.5 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=3 ttl=118 time=14.4 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=4 ttl=118 time=14.4 ms
```

17.21 arp : manage Address Resolution Protocol

The `arp` command in Linux is a network utility tool used to display, add, and remove entries in the Address Resolution Protocol (ARP) cache. This command is crucial for managing network communications in a Linux system

- Show the current ARP table (mac addresses of every machine that with current m
arp -a

display the ARP cache.

The output shows the IP address (192.168.1.1),
the corresponding MAC address (ab:cd:ef:gh:ij:kl),
the network interface (eth0) on which the ARP entry is located.

- Delete a specific entry:

arp -d {{address}}

- Create an entry in the ARP table:

arp -s {{address}} {{mac_address}}

-a Show all entries in the ARP cache.	arp -a
-d Delete an entry in the ARP cache.	arp -d 192.168.1.1
-s Add a new entry to the ARP cache.	arp -s 192.168.1.1 ab:cd:ef:gh:ij:kl
-v Enable verbose mode.	arp -v -a
-n Do not resolve names.	arp -n -a
-i Specify the network interface.	arp -i eth0 -a
-D Read hardware address from given device.	arp -D eth0 -s 192.168.1.1
-e Display in default (Linux) style.	arp -e -a
-H Specify the hardware type.	arp -H ether -a
-p Publish the entry.	arp -s 192.168.1.1 ab:cd:ef:gh:ij:kl -p
-r Read new entries from file or standard input.	arp -r < arp_entries.txt
-t Specify the hardware type.	arp -t ether -a

17.22 ifconfig : InerFace CONFIGurator

<https://www.baeldung.com/linux/ifconfig-command>

ifconfig can be installed with command :
\$ sudo apt install net-tools

Connection names in Linux:

lo ---> loopback

(w)lps20 wifi0 ---> Wireless connection

(e)nps20 eth0 ---> Ethernet cable connection

(e)nps21 eth1

Prints details about all the network interfaces available
on your system.

It is used to view and change the configuration of the network interfaces on your system.

Display information about all network interfaces currently in operation:

```
$ ifconfig
```

To ENABLE an inactive interface, provide ifconfig with the interface name followed by the keyword up:

```
$ sudo ifconfig eth1 up
```

You can DISABLE an active network interface using the down keyword.

```
$ sudo ifconfig eth1 down
```

Configuring an interface

ifconfig can be used at the command line to configure (or re-configure) a network interface. This is often unnecessary since this configuration is often handled by a script when you boot the system. If you'd like to do so manually, you need superuser privileges, so we'll use sudo again when running these commands.

To assign a static IP address to an interface, specify the interface name and the IP address.

For example, to assign the IP address 192.168.7.100 to the interface enp1s0, use the command:

```
$ sudo ifconfig enp1s0 192.168.7.100
```

To assign a network mask to an interface, use the keyword "netmask" and the netmask address. For instance, to configure the interface enp1s0 to use a network mask of 255.255.255.0, the command would be:

```
$ sudo ifconfig enp1s0 netmask 255.255.255.0
$ sudo ifconfig enp1s0 10.0.0.100/16      # netmask
                                           # of 16 bit for subnet
```

These configurations can be combined in a single command. For instance, to configure interface enp1s0 to use the static IP address 192.168.7.100, and the network mask 255.255.255.0:

```
$ sudo ifconfig enp1s0 192.168.7.100 netmask 255.255.255.0
```

mtu N

This parameter sets the MTU (maximum transfer unit) of an interface. This setting is used to limit the maximum packet size transferred by the interface. If you're unsure about it, you can safely leave this setting alone.

netmask address

Set the IP network mask for this interface. This value defaults to the

usual class A, B or C network mask (as derived from the interface IP address), but it can be set to any value.

What about DHCP (Dynamic Host Configuration Protocol)?

ifconfig can only assign a static IP address to a network interface. To assign a dynamic IP address using DHCP, use the "dhclient" command.

Dynamic Host Configuration Protocol (DHCP) is a client/server protocol that automatically provides an Internet Protocol (IP) host with its IP address and other related configuration information such as the subnet mask and default gateway. RFCs 2131 and 2132 define DHCP as an Internet Engineering Task Force (IETF) standard based on Bootstrap Protocol (BOOTP), a protocol with which DHCP shares many implementation details. DHCP allows hosts to obtain required TCP/IP configuration information from a DHCP server.

Examples :

Display details of all interfaces, including disabled interfaces:
We can see whether an interface is enabled or disabled by looking at the flags, in particular the UP and RUNNING keywords.

```
$ ifconfig -a
```

For a quick summary of the interfaces, with some brief network stats, we can use the -s option:

```
$ ifconfig -s
```

View network settings of an Ethernet adapter enp0s3:

```
$ ifconfig enp0s3
```

Enable enp0s3 interface:

```
$ ifconfig enp0s3 up
```

Disable enp0s3 interface:

```
$ ifconfig enp0s3 down
```

17.23 ping : ping command

```
$ ping rt-ed.co.il # We can see that IP of the site is 212.199.114.183
```

```
PING rt-ed.co.il (212.199.114.183) 56(84) bytes of data.
```

```
64 bytes from 212.199.114.183.static.012.net.il (212.199.114.183):
```

```
icmp_seq=2 ttl=57 time=13.3 ms
```

```
64 bytes from 212.199.114.183.static.012.net.il (212.199.114.183):
```

```
icmp_seq=3 ttl=57 time=36.1 ms
```

```
$ ping 192.168.1.12
```

```
PING 192.168.1.12 (192.168.1.12) 56(84) bytes of data.  
64 bytes from 192.168.1.12: icmp_seq=1 ttl=64 time=129 ms  
64 bytes from 192.168.1.12: icmp_seq=2 ttl=64 time=4.95 ms  
64 bytes from 192.168.1.12: icmp_seq=3 ttl=64 time=4.64 ms
```

Send ICMP ECHO_REQUEST packets to network hosts.

- Ping host:
 ping {{host}}
- Ping a host only a specific number of times:
 ping -c {{count}} {{host}}
- Ping host, specifying the interval in seconds between requests (default is 1 second):
 ping -i {{seconds}} {{host}}
- Ping host without trying to lookup symbolic names for addresses:
 ping -n {{host}}
- Ping host and ring the bell when a packet is received (if your terminal supports it):
 ping -a {{host}}
- Also display a message if no response was received:
 ping -0 {{host}}

17.24 hostname : system's hostname

hostname command in Linux is used to obtain the DNS (Domain Name System) name and set the system's hostname or NIS (Network Information System) domain name.

A hostname is a name given to a computer and attached to the network. Its main purpose is to uniquely identify over a network.

```
$ hostname  
    amit-vm-mint21  
$ hostname -i  
    127.0.1.1
```

17.25 wget : network downloader

GNU Wget is a free utility for non-interactive download of files from the Web.

It supports HTTP, HTTPS, and FTP protocols, as well as retrieval through HTTP proxies.

Wget is non-interactive, meaning that it can work in the background, while the user is not logged on.

This allows you to start a retrieval and disconnect from the system, letting Wget finish the work. By contrast, most of the Web browsers require constant user's presence, which can be a great hindrance when transferring a lot of data.

Wget provides a number of options allowing you to download multiple files, resume downloads, limit the bandwidth, recursive downloads, download in the background, mirror a website, and much more.

Mirror a site:

```
wget -m http://lwn.net
```

Mirror a site (-r : recursive):

```
wget -r -np http://www.xml.com/ldd/chapter/book/
```

#recursively download an online book

#for offline access. -np : no parent

18 TextToSpeech

18.1 TextToSpeech..apps

say : converts text to audible speech using the GNUstep speech engine.

```
sudo apt-get install gnustep-gui-runtime  
say "hello"
```

festival : General multi-lingual speech synthesis system.

```
sudo apt-get install festival  
echo "hello" | festival --tts
```

spd-say : sends text-to-speech output request to speech-dispatcher

```
sudo apt-get install speech-dispatcher  
spd-say "hello"
```

espeak : is a multi-lingual software speech synthesizer.

```
sudo apt-get install espeak  
espeak "hello"
```

Example to see voices :

```
espeak --voices=variant # to see available languages  
espeak -compile=en-scottish
```

18.2 spd-say : text-to-speech output

See: <https://askubuntu.com/questions/501910/how-to-text-to-speech-output-using-command-line>

Install :
`sudo apt-get install speech-dispatcher`
`spd-say "hello"`

See `spd-say`

`-r, --rate`
Set the rate of the speech (between -100 and +100, default: 0)

`-p, --pitch`
Set the pitch of the speech (between -100 and +100, default: 0)

`-i, --volume`
Set the volume (intensity) of the speech (between -100 and +100, default: 0)

`-l, --language`
Set the language (ISO code)

`-t, --voice-type`
Set the preferred voice type (male1, male2, male3, female1, female2 female3, child_male, child_female)

`-L, --list-synthesis-voices`
Get the list of synthesis voices

```
spd-say -t Robert "Hello, how do you do "  
spd-say -t Pablo "Hello, how do you do "  
spd-say -t female2 "Hello, how do you do "
```

19 Appendix.

19.1 LaTeX : editor

Bookmarks addition was based on following explanation :
<https://michaelgoerz.net/notes/pdf-bookmarks-with-latex.html>

```
sudo apt install texlive-full # install full LaTeX editor
```

To add bookmarks do following steps:

1. `sudo apt install texlive-full # install LaTeX editor`
2. `cd ~/Documents/docs/c_for_embedded_pdf`

3. touch c_for_embed.tex
4. geany & c_for_embed.tex # prepare LaTeX document with bookmarks definitions
5. pdflatex c_for_embed.tex # this command generates linux_admin.pdf with desired bookmarks :)

Note : the latex document can be edited with texmaker

see : <https://artofproblemsolving.com/wiki/index.php/LaTeX>

To enlarge interface font run following command
from terminal window :

```
$ texmaker -dpiscale 1
```

19.2 Shell..builtin..commands : shell built in commands

See :

https://www.gnu.org/software/bash/manual/html_node/Shell-Builtin-Commands.html

Shell Builtin Commands

Builtin commands are contained within the shell itself.

When the name of a builtin command is used as the first word of a simple command (see Simple Commands), the shell executes the command directly, without invoking another program.

Builtin commands are necessary to implement functionality impossible or inconvenient to obtain with separate utilities.

Example:

```
$ type -a time
time is a shell keyword # time is built in inside the shell
time is /usr/bin/time   # also time is in this location
time is /bin/time       # also time is in this location
```

19.3 shift+PgUp/PgDn : page scroll in terminal

Page scroll in terminal

19.4 niceness : process priority

To my knowledge, the Linux kernel does not change the niceness of a process, and I fail to see why it would since it doesn't have to to lower the priority of a process.

The niceness is an information given to the kernel, telling it how nice that process is willing to be.

The kernel scheduler is free to take this information into account the way it wants in order to change the priority of a process, it doesn't need to change its value.

On the other hand, in user land, there are daemons like `AND` whose task is to renice processes according to rules set up by the admin. Do you have such a daemon installed on your server?

Niceness values range from -20 (highest priority - most favorable to the process) to 19 (lowest priority - least favorable to the process).

Niceness can be changed with command `renice` for the running process.

The `NI` field in `ps` command shows the nice value of the process. The nice value determines the priority of the process. The higher the value, the lower the priority--the "nicer" the process is to other processes. The default nice value is 0 on Linux workstations.

19.5 VT : virtual terminal

Linux systems come with a bunch of virtual terminals, or VTs. Your graphical user interface on Ubuntu runs on VT2, and VT3 to VT6 allow you to login via command line.

When the graphics is stuck, you can still access the console window.

To access consoles: `[Ctrl][Alt][F3]/[F4]/[F5]`

It can be used to work with terminal `tty3/tty4/tty5` only. In the text console, you can try to kill the guilty application

Once this is done, you can go back to the graphic session by pressing `[Ctrl][Alt][F2]` (depends on distribution)

19.6 ctrl+shift+arrowup/down : scroll in terminal

Scroll in terminal window

19.7 Additional...shells

Different shells :

`sh` #bourne shell

`#bourne again shell` : `bash`

`sch` # c shell , less recommended

`zsh` # another shell

19.8 Special...files

Runs automation scripts every time the terminal runs:

`.bashrc` - remote control

Runs automation scripts every time before the program run:

`.profile`

19.9 inode : Create files or set access/modif. times

<https://en.wikipedia.org/wiki/Inode>

- The inode (index node) is a data structure in a Unix-style file system that describes a file-system object such as a file or a directory.
- Each inode stores the attributes and disk block locations of the object's data. File-system object attributes may include metadata (times of last change, access, modification), as well as owner and permission data

19.10 Disc..Partitions

<code>/</code>	root directory
<code>/bin/</code>	Basic, essential system commands
<code>/boot/</code>	Bootloader, Kernel images, initrd (initial RAM disk) and configuration files
<code>/dev/</code>	Files representing devices
<code>/etc/</code>	System configuration files(as registry)
<code>/home/</code>	User directories
<code>/lib/</code>	Basic system shared libraries
<code>/lost+found</code>	Corrupt files the system tried to recover
<code>/media</code>	Mount points for removable media (<code>/media/usbdisk</code> , <code>/media/cdrom</code>)
<code>/mnt/</code>	Mount points for temporarily mounted filesystems
<code>/opt/</code>	Specific tools installed by the sysadmin (for all users)
<code>/proc/</code>	Access to system information (list of processes)
<code>/root/</code>	root user home directory

/sbin/ superuser only commands

 /sys/ System and device controls (cpu frequency,
 device power, etc.

 /tmp/ temporary files

 /usr/ Regular user tools (not essential to the system)
 /usr/bin/, /usr/lib/, /usr/sbin...

 /usr/local/ Specific software installed by the sys-admin
 (often preferred to /opt/

 /var/ Data used by the system or system servers
 /var/log/, /var/spool/mail (incoming mail),
 /var/spool/lpd (print jobs)...

19.11 Globbing

Expansion :

~ : home directory

[], ? , *

!

\$

“

‘

* : any characters

? : exactly one character

```
echo ?.txt      # ? replace exactly one character
echo ???.txt
```

^ is for not :

```
$ echo [^0-9].txt
a.txt b.txt g.txt i.txt
```